FEDERAL STATE AUTONOMOUS EDUCATIONAL INSTITUTION FOR HIGHER EDUCATION NATIONAL RESEARCH UNIVERSITY
«HIGHER SCHOOL OF ECONOMICS»
Faculty of Computer Science

Muhammad Zeeshan, Asghar

Name, Surname

Адаптивное проектирование воплощенных робототехнических систем: от автомоделирования в реальном времени до координации роя

Russian title

Adaptive Design of Embodied Robotic Systems: From Real-Time Self-Modeling to Swarm Coordination

English title

Qualification paper – Master of Science Dissertation
Field of study 01.04.02 «Applied Mathematics and Informatics»
Program: «Data Science»

Student
Muhammad Zeeshan Asghar

Supervisor
Sabarathinam Srinivasan

Moscow, 2026

Abstract

Embodied robotic systems deployed in unstructured environments inevitably experience embodiment drift—the progressive divergence between a robot's internal kinematic model and its physical hardware caused by mechanical wear, calibration slip, thermal deformation, and unmodeled impacts [1, 2]. When a robot's internal model no longer matches its physical embodiment, traditional Jacobian-based inverse kinematics controllers fail catastrophically: end-effectors diverge from targets, oscillations destabilize the system, and contact-rich tasks generate unsafe forces [4, 10]. Maintaining true autonomy requires self-modeling—the ability to infer one's own morphology directly from sensorimotor experience.

Contemporary visual self-modeling architectures based on Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS) achieve high morphological expressiveness but suffer from a critical latency paradox [11, 16]. A diagnostic self-model is only valuable if it can provide state estimates fast enough to influence the feedback controller. Modern robotic control loops operate at 1 kHz, permitting a maximum latency of 1.0 millisecond per inference [114]. Existing neural rendering self-models operate at 5–37 FPS—latencies of 27–192 milliseconds—rendering them unusable for real-time control integration. This represents a fundamental architectural mismatch: volumetric reconstruction is computationally incompatible with millisecond-scale control deadlines.

This thesis resolves this bottleneck by introducing NeuroKin, a lightweight fully convolutional direct sensorimotor decoder that completely bypasses explicit 3D volumetric reconstruction. NeuroKin maps joint angles directly to visual silhouettes, achieving 21.88 dB PSNR in our empirical evaluations, with a training-loop throughput of 7,406 samples/s (batch size 32) and a total training time of 33.4 seconds. Integration of NeuroKin-3D's stereo visual estimates into a Jacobian-based feedback loop enables dynamic error recovery after encoder-slip fault injection, demonstrating that fast visual self-modeling can actively stabilize a manipulator under proprioceptive drift.

Building on this individual diagnostic capability, we scale to collective fault tolerance via temporal stigmergy—a mechanism of indirect coordination wherein agents leave environmental traces that stimulate adaptive responses in subsequent agents. In a 100-stage sequential swarm under progressive fault injection, the standard inverse-kinematics baseline collapses from 100% pre-fault success to 13% post-fault success. Conversely, NeuroKin-enabled agents update a shared state (`factory_state`) that tracks consecutive failures, autonomously trigger localized recovery modes, and preserve 88% systemic success.

This thesis provides four primary contributions: (1) formal benchmarking of the latency paradox in neural rendering self-models, (2) the NeuroKin direct decoding architecture achieving high-throughput visual decoding in our empirical evaluations, (3) visual closed-loop adaptation under encoder-slip faults, and (4) empirical validation that fast individual self-modeling enables decentralized stigmergic coordination—the computational prerequisite for physical swarm deployment.

Acknowledgements

The author gratefully acknowledges the guidance and feedback of their academic supervisor and the thesis defence committee. Supplementary materials, dataset generation code, and the implementation of the NeuroKin project are available in the public repository: https://github.com/YoloPopo/neurokin. Where appropriate, appendices and source code releases are linked from that repository for reviewers and examiners.

The author confirms that AI tools were used only to assist with grammar, vocabulary, and to enhance narrative flow; all technical content, experimental design, analyses, and conclusions remain the author's original work.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 The Challenge of Embodiment Drift

Autonomous robots deployed in remote, unstructured environments—planetary rovers, disaster-response machines, deep-sea manipulators, and industrial inspection drones—must operate for extended periods without human intervention. These systems cannot rely on frequent manual recalibration, hardware replacement, or human-in-the-loop supervision. They must be self-sufficient, adaptive, and resilient in the face of inevitable physical degradation [1, 2].

A fundamental requirement for such autonomy is maintaining an accurate internal representation of one's own morphology, kinematics, and dynamics. Traditional robotic systems rely on predefined kinematic models—typically specified in Unified Robot Description Format (URDF) files—that encode link lengths, joint limits, inertial properties, and collision geometries. These models enable classical control algorithms, such as Jacobian-based inverse kinematics and operational space control, to map intended high-level actions into precise joint torques or positional trajectories [114]. Under ideal conditions, with a perfectly calibrated robot operating in a controlled environment, these methods achieve millimeter-level precision.

However, the assumption that a physical robot remains mathematically identical to its idealized digital model is fundamentally flawed under continuous operational deployment. Physical embodiment subjects robots to the relentless realities of thermodynamics, material fatigue, and mechanical friction. Over extended operational cycles—spanning hours, days, or months of continuous operation—the strict mathematical guarantees of forward and inverse kinematics inevitably decouple from physical reality.

This persistent, accumulating divergence between the internal computational representation of the robot and its actual physical state is formally defined in this thesis as *embodiment drift* [1, 4]. The term captures the essential phenomenon: the robot's sense of its own body drifts away from truth as hardware degrades.

1.1.1 Manifestations of Embodiment Drift

Embodiment drift manifests across a broad spectrum of mechanical and sensory degradations. At the micro-scale, geared transmissions develop unmodeled backlash as teeth wear; actuator drive belts lose optimal tension through creep and fatigue; thermal expansion induced by sustained high-torque operations alters the effective physical length of metallic linkages by micrometers—enough to matter in precision tasks. At the sensory layer, proprioceptive joint encoders may imperceptibly slip from their calibrated zero-points due to vibration or electrical noise; exteroceptive sensors, such as chassis-mounted RGB cameras or depth sensors, experience extrinsic calibration shifts from ambient thermal cycling or minor physical impacts [15].

In more severe, sudden operational scenarios, robots may suffer macroscopic structural damage: an unexpected collision may plastically deform a physical link by several degrees; an actuator may seize entirely due to electrical failure or debris ingress; an asymmetrical payload shift—such as a grasped object suddenly changing weight distribution—may fundamentally alter the system's center of mass and dynamic response.

1.1.2 Consequences for Control

When significant embodiment drift occurs, the consequences for closed-loop feedback control are catastrophic. A controller operating under the assumption of a nominal kinematic tree will compute pseudo-inverse Jacobian updates that are mathematically sound relative to the model but physically erroneous relative to reality [114]. The system issues corrective motor commands that inadvertently drive the end-effector away from the target, induce highly unstable resonant oscillations, or, in contact-rich manipulation tasks, exert unsafe, unbounded forces upon the environment. In legged locomotion, embodiment drift leads to gait asymmetry, reduced stability margins, and eventual falls. In humanoid robots operating near humans, the safety implications are even more severe [4].

To maintain true autonomy and operational safety in the presence of embodiment drift, a robotic system must possess the capacity for *self-modeling*—the ability to autonomously infer, update, and deploy an internal representation of its own morphology derived strictly from its own sensorimotor experience, entirely bypassing the need for external, human-in-the-loop manual recalibration [10, 4].

Figure 1.1 summarizes the closed-loop self-modeling pipeline that motivates the rest of this thesis.

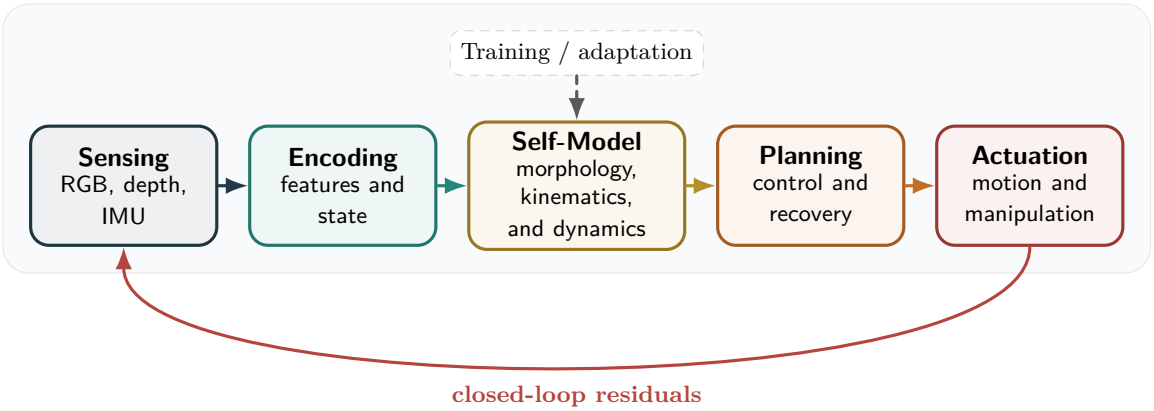

Figure 1.1: Closed-loop self-modeling pipeline that connects perception, internal model update, and control.

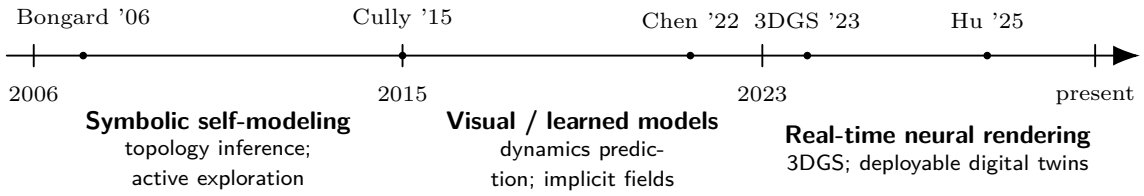

Figure 1.2: Three eras of robot self-modeling: symbolic morphology inference via active exploration (Bongard et al., 2006), behavioral adaptation through repertoire search (Cully et al., 2015), and contemporary visual self-modeling driven by neural rendering (NeRF and 3D Gaussian Splatting).

1.2 The Evolution of Self-Modeling Paradigms

The formidable challenge of autonomous self-modeling has driven several distinct research paradigms over the past two decades. Each era brought unique insights and faced specific limitations that motivate the approach taken in this thesis. Figure 1.2 provides a visual summary of this progression.

1.2.1 The Symbolic Era: Estimation-Exploration

The earliest formalized paradigm relied heavily on symbolic and kinematic inference. The seminal work of Bongard, Zykov, and Lipson established the first complete closed-loop self-modeling system on actual physical robots [1]. Their approach viewed self-modeling as a continuous cycle of hypothesis generation and active physical exploration. The robot maintained a population of competing symbolic models representing various candidate topologies—different possible configurations of its own body. To discriminate among these hypotheses, the robot synthesized exploratory motor commands mathematically designed to maximize the sensory disagreement among the candidate models. Upon executing these actions, the robot collected sensor data and systematically pruned models that failed to predict

observed outcomes. After approximately 16 modeling-testing cycles spanning several minutes of real-world experimentation, the robot converged on an accurate damaged model and synthesized a compensatory gait through reinforcement learning on the internal simulation.

The core innovation was an active learning loop: the fitness of a test action was determined by its ability to cause disagreement among the best candidate models, thereby maximally reducing uncertainty. While conceptually elegant, this approach suffers from the symbolic bottleneck: the robot can only discover models constructible from predefined physics engine primitives (cylinders, hinge joints). It cannot represent non-rigid deformations, soft-body dynamics, or arbitrary visual damage—limitations that become critical in unstructured environments where failure modes are unpredictable [4]. As noted by Kwiatkowski and Lipson, the search space of symbolic physics engines is discrete and non-differentiable, making optimization slow and prone to local optima [4]. Furthermore, the approach relied heavily on simple proprioceptive sensors providing limited information compared to high-bandwidth vision.

1.2.2 The Behavioral Era: Avoidance Over Adaptation

Recognizing the computational intensity of explicit morphological reconstruction, a subsequent paradigm emerged focused entirely on behavioral adaptation. Rather than diagnosing the damage, these methods sought to find compensatory behaviors that worked despite the damage [2]. Cully and colleagues introduced Intelligent Trial and Error (IT&E), which bypassed damage diagnosis entirely. Their key insight was to focus on finding what still works rather than diagnosing what went wrong.

Using the MAP-Elites quality-diversity algorithm [3], they pre-computed a behavioral repertoire of approximately 13,000 distinct gaits in simulation—a process requiring 40 million simulations over two weeks of computation. This repertoire is a high-dimensional grid where each cell corresponds to a specific behavioral descriptor (e.g., duty cycle versus leg offset) and stores the highest-performing controller found for that descriptor.

When damaged in the field, the robot treated this map as a Bayesian prior, using Gaussian Process regression to search for a behavior that performs well in the real world. By testing a few dozen distinct behaviors, the Gaussian Process updated its posterior mean and variance, rapidly identifying a compensatory gait. Cully and colleagues demonstrated recovery in less than two minutes of real-world testing across various damage scenarios, including removal of entire legs [2].

However, IT&E represents a strategy of avoidance, not adaptation. The robot does not repair its internal model—it discards the parts of the behavioral space that no longer function. While effective for redundant locomotion tasks where multiple gaits can achieve similar forward velocity, this approach fails for precision manipulation tasks that require exact geometric awareness [4]. If a robot needs to reach through a narrow aperture, it cannot simply "try a different gait"—it must understand the precise Cartesian geometry of its damaged arm. Furthermore, as robotic complexity increases, the curse of dimensionality

makes pre-computing dense behavioral maps prohibitively expensive: a 6-DOF arm with even modest discretization would require billions of map entries, far exceeding computational budgets.

1.2.3 Task-Agnostic Learned Dynamics Models

The advent of deep learning enabled a third paradigm: learning dynamics models that encode the mapping from motor commands to sensory consequences. Kwiatkowski and Lipson proposed task-agnostic self-modeling, training deep forward models to predict the future state of the robot given the current state and action [4]. This represented a pivotal shift from symbolic physics to learnable physics. The same approach was demonstrated to work as a complete reinforcement learning environment to learn zero-shot tasks across morphologies in a follow-up study [5].

While theoretically powerful, these data-driven models face a fundamental limitation: systematic bias accumulates progressively across extended planning horizons [4]. Simulated trajectories diverge increasingly from actual robot behavior, rendering the approach impractical for tasks requiring accurate prediction beyond very short time windows. This horizon-dependent degradation motivates approaches that prioritize shorter planning windows or use learned corrections rather than pure forward simulation.

1.2.4 Proprioceptive Morphology Identification

A parallel thread of research focuses on proprioceptive identification—restoring embodiment using internal signals alone, without the occlusions and lighting failures that often cause vision-only pipelines to break [6]. The internal sensing avoids external disturbances, offering significant deployment advantages: the approach remains robust across diverse lighting conditions and operates without calibrated cameras. However, eliminating external visual information imposes real functional costs: expressiveness is substantially reduced compared to vision-based methods [8]. This is not merely a theoretical gap but a genuine operating constraint that limits the complexity of tasks these systems can support compared to their multi-modal counterparts.

Meta-self-modeling restores morphology parameters between motion signature reconfigurations [6], and IMU-centric methods show hardware-realized damage sensing with minimal sensing overhead on legged robots [8]. Analytical modular pipelines remain strong baselines in the case of known assembly constraints, offering interpretable, stable, and automatically generated kinematic and dynamic models [7]. The frontier is intervention, not just identification: controlled breakage of proprioception-driven morphology demonstrates that active structural adaptation is supported by internal sensing, and that morphology itself is a form of adaptive feedback control [83, 9].

1.2.5 The Visual Era: Deep Self-Models and Neural Rendering

The current state-of-the-art leverages neural rendering to learn visual self-models directly from camera observations. Chen and colleagues introduced occupancy query learning as a foundational approach [10]. Signed Distance Functions (SDFs) conditioned on joint angles enable differentiable collision checking directly from RGB-D sensor streams, supporting both motion planning and recovery from damage. Their multi-view hardware system achieved approximately one percent workspace accuracy and demonstrated support for planning and post-damage recovery. However, practical deployment introduced significant challenges: careful camera placement became critical for observability, precise calibration was required for geometric accuracy, and performance degraded substantially under occlusion, shadows, blur, and noise [10].

Hu, Lin, and Lipson reduced the camera system to a single RGB sensor while learning the Free-Form Kinematic Self-Model (FFKSM) from a single viewpoint [11]. The reparameterization of camera frames in terms of the first two joints facilitated the learning process. This approach supports inverse kinematics, path planning, and post-damage recovery. However, volumetric rendering via NeRF remained computationally expensive, and single-view training introduced persistent vulnerabilities: occlusion sensitivity and perspective-dependent predictions.

More recently, Kinematic 3D Gaussian Splatting (K-3DGS) has emerged as an explicit alternative to implicit NeRFs [16]. 3DGS replaces ray marching with differentiable rasterization of explicit anisotropic Gaussians, enabling high frame rate rendering and fast optimization. By attaching explicit Gaussian primitives to a differentiable kinematic chain and leveraging rasterization-based rendering rather than ray-marching, K-3DGS promises faster inference. Each Gaussian primitive is defined by a 3D mean position, covariance matrix, opacity, and color, allowing the model to render the robot in any arbitrary pose through standard graphics pipeline operations.

Yet, as this thesis establishes, the immense representational power of these neural rendering architectures has introduced a severe computational bottleneck that completely obstructs physical deployment in real-time control loops.

1.3 The Latency Paradox in Neural Rendering

A diagnostic self-model is operationally valuable only if it can provide state estimates fast enough to influence the stabilizing behavior of the feedback controller [11, 16]. If the model requires hundreds of milliseconds to generate a prediction, the information arrives too late—the robot has already executed several control cycles based on stale assumptions.

1.3.1 Temporal Constraints of Robotic Control

Modern robotic control systems operate at tightly constrained frequencies. Force control, impedance control, and dynamic balancing typically run at 1 kHz, requiring the entire perception-planning-actuation pipeline to execute within 1.0 millisecond [114]. Table 1.1 shows the latency requirements for different control regimes.

Table 1.1: Latency requirements for robotic control loops at different frequencies.

Control Type	**Frequency**	**Max Latency (ms)**
Force/Impedance Control	1 kHz	1.0
Compliant Manipulation	500 Hz	2.0
Visual Servoing	30–100 Hz	10.0–33.3
Offline Diagnostics	$<$10 Hz	$>$100.0

Within the 1.0 ms window, the entire software stack must read physical sensor buffers, execute state estimation, compute target Cartesian error vectors, resolve complex inverse kinematic equations, enforce physical joint limits, and issue subsequent motor torque commands. Consequently, any self-modeling diagnostic component integrated into this inner control loop possesses an absolute maximum inference budget of a fraction of a millisecond.

1.3.2 The Computational Bottleneck

Current state-of-the-art neural rendering architectures fundamentally violate this latency budget. Implicit NeRF-based models achieve frame rates of approximately 5 FPS, corresponding to 191.6 ms latency [11]. Explicit 3DGS-based architectures reach approximately 37 FPS, corresponding to 27.0 ms latency [16]. Both exceed the 1.0 ms control budget by factors of 27 to 190—rendering them entirely unsuitable for real-time control integration.

The root cause is architectural. Both approaches maintain an explicit or implicit 3D volumetric representation. NeRF requires evaluating a deep MLP hundreds of thousands of times per image via ray-marching—a fundamentally serial process [19]. Even with acceleration structures like Instant-NGP's hash encoding [21] or Mip-NeRF's scale-aware anti-aliasing [18], the latency remains too high for tight control loops required in robotic damage recovery. Adapting NeRFs to articulated bodies via D-NeRF approaches [20] adds time-deformation networks, further compounding computational cost.

3DGS requires computing forward kinematics for thousands of Gaussian primitives, projecting them into 2D screen space, sorting them by depth, and alpha-compositing [23]—a heavily sequential pipeline despite GPU acceleration. While 3DGS achieves substantial speedup over NeRF, this speed is achieved by retreating to isotropic constraints that yield geometric artifacts, dropping fidelity [16]. These operations are not merely engineering inefficiencies; they are structural consequences of demanding a full 3D intermediate representation.

1.3.3 The Plasticity-Stability Dilemma in Self-Modeling

Beyond latency, self-modeling faces the plasticity-stability dilemma, first articulated by Grossberg [105]: the system must be plastic enough to learn new configurations (damage) yet stable enough to retain knowledge of undamaged components. In biological brains, this balance is achieved through synaptic pruning—the physical restructuring of the connectome wherein weak neuronal connections are eliminated while frequently used connections are strengthened. Hebbian learning (neurons that fire together, wire together) ensures that functional modules are physically distinct.

In standard, monolithic deep neural networks, sequentially learning new spatial geometries leads inevitably to catastrophic forgetting [103, 104]. If an implicit NeRF, an explicit 3DGS point cloud, or a direct convolutional decoder is naively fine-tuned exclusively on a stream of recent visual observations depicting a damaged joint, the globally shared gradient updates will rapidly destroy the network's ability to accurately render the undamaged base of the robot. The network overfits to the new, damaged data distribution at the catastrophic expense of prior structural knowledge [107].

To combat this sequential learning degradation, the literature has developed extensive mitigation frameworks. Regularization-based methodologies, such as Elastic Weight Consolidation (EWC) [107], calculate the Fisher information matrix to identify and mathematically protect specific neural weights crucial for maintaining prior geometric knowledge. Architectural isolation methods rely on dynamic routing protocols, mixture-of-experts ensembles [109], or isolating continuous learning within highly sparse, trainable subnetworks identified via the Lottery Ticket Hypothesis [108]. However, these methods remain computationally expensive for real-time adaptation on edge hardware, and they treat the network as a monolith with varying "stiffness" rather than enabling true structural modularity.

1.4 Digital Twins and the Simulation-to-Reality Gap

The endeavor to maintain an accurate, real-time internal self-model is conceptually identical to the broader industrial engineering goal of generating and sustaining a robotic Digital Twin [62, 78]. In contemporary academic literature, a digital twin is defined not merely as a static 3D visual replica or a passive CAD model, but as a bi-directional, dynamically synchronized computational surrogate of a physical asset.

The idealized robotic digital twin operates in continuous tandem with the physical hardware. It receives high-frequency sensor telemetry (joint angles, motor torques, camera feeds) from the physical robot in the field. Crucially, it dynamically updates its internal state matrices—its collision geometry, mass distribution, and friction coefficients—to perfectly mirror the physical degradation and embodiment drift experienced by the hardware. This dynamic synchronization allows the twin to serve as a perfectly safe, highly accelerated sandbox for forward-simulating potential control policies before executing them in physical reality [44, 79].

Figure 1.3 illustrates the continuous synchronization loop between a physical robot and its digital twin.

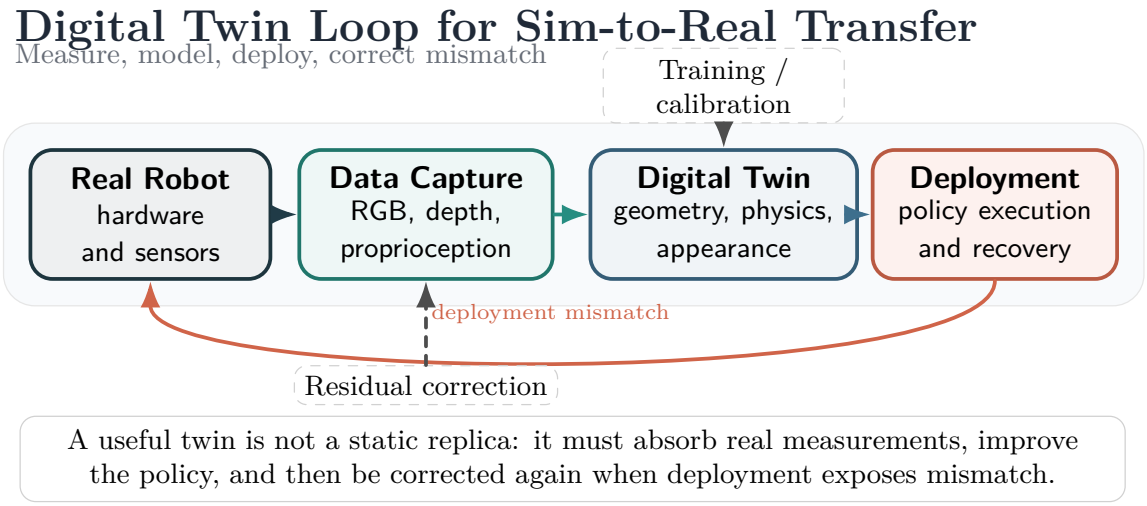

Figure 1.3: Digital twin synchronization loop bridging sensing, simulation, and control.

However, constructing visual digital twins specifically intended for autonomous, closed-loop robotic control introduces stringent latency constraints that differ significantly from those found in static industrial monitoring. An active robotic digital twin must be instantaneously queryable. If a physical manipulator operating in a highly cluttered environment sustains an unexpected impact, the digital twin must update its geometric occupancy map immediately. If the twin's internal update latency exceeds the temporal dynamics of the physical interaction, the robot will execute evasive commands based on stale, pre-impact geometry, precipitating further catastrophic collisions.

Furthermore, the practical deployment of neural self-models and dynamic digital twins is universally haunted by the Simulation-to-Reality (Sim-to-Real) gap [41, 57]. The majority of high-capacity neural representations are trained within pristine simulation environments, such as MuJoCo, Isaac Gym, or, as utilized in this thesis, PyBullet. While computationally efficient, these physics simulators rely on idealized mathematical approximations. They frequently fail to accurately capture complex Coulomb friction models, non-linear actuator stiction, subtle link compliance under heavy payloads, and the incredibly complex photometric realities of physical environments, including ambient lighting fluctuations, harsh specular reflections on metallic links, and pervasive sensor noise.

When a neural self-model trained exclusively in simulation is deployed onto physical hardware, it frequently collapses. The minor photometric discrepancies between the simulated training data and the physical camera feed severely corrupt the network's latent space, resulting in completely erroneous morphological predictions [15]. Recent literature attempts to bridge this critical Sim-to-Real gap through massive Domain Randomization, where researchers randomly perturb lighting vectors, background textures, camera positions, and physical mass properties during simulation training [41]. Advanced methodologies also

explore real-time Residual Dynamics Learning, wherein the neural network is trained to predict only the small, non-linear residual discrepancy between the idealized physics simulator and the actual physical observation [92, 90].

Within the rigorous scope of this thesis, all experiments are deliberately bounded within the synthetic PyBullet simulation environment. This is not an academic oversight but a highly controlled, necessary methodological choice designed specifically to isolate architectural latency. By operating within the controlled vacuum of a simulator, this thesis effectively isolates the theoretical mathematical efficiency of the proposed NeuroKin architecture, demonstrating that the latency bottleneck can be reduced before introducing the complexities of physical noise.

1.5 Problem Statement and Core Hypothesis

The core scientific problem addressed by this thesis is the resolution of the latency paradox in visual robotic self-modeling, and the subsequent leveraging of this high-speed diagnostic capability to establish the computational mechanisms for multi-agent swarm coordination under compounding mechanical failures.

1.5.1 The Central Hypothesis

This thesis investigates a central hypothesis: *explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling.* If the downstream task requires only silhouette-level awareness, collision checking, or end-effector coordinate estimation from known camera viewpoints, maintaining a dense, queryable 3D scene representation is computationally wasteful. The robot does not need to know the precise 3D occupancy of every voxel in its volume; it needs to know where its end-effector is in Cartesian space and whether its links will collide with obstacles.

We propose a paradigm shift toward *direct sensorimotor decoding*: a lightweight fully convolutional network that maps joint angles directly to visual silhouettes and spatial coordinates, completely bypassing the intermediate 3D reconstruction step. We hypothesize that such an architecture can achieve fast inference—compatible with 1 kHz control budgets—while maintaining reconstruction fidelity comparable to or exceeding volumetric baselines, precisely because dense pixel-wise supervision is more efficient than sparse ray sampling.

1.5.2 Empirical Scope and Deliberate Constraints

The empirical scope of this research is deliberately bounded within a highly controlled simulation environment to isolate core architectural latency metrics from unmodeled photometric noise. All data generation, neural network training, closed-loop controller validation, and multi-agent coordination are executed using a synthetic PyBullet simulation of a 4-DOF

serial manipulator. The robot is rendered against a uniform background; joint angles are recorded without sensor noise; camera calibration is perfect. These constraints are not limitations but methodological choices: they allow us to measure pure architectural latency without confounding variables.

All experiments use a dataset of 2,000 samples (1,600 training, 400 validation) generated via Lorenz attractor trajectories, ensuring smooth, continuous, and diverse workspace coverage. Hardware is an NVIDIA Tesla T4 GPU with PyTorch 2.0 and CUDA 11.8.

1.5.3 The Temporal Stigmergy Framework for Swarm Coordination

Crucially, the thesis title—"Toward Swarm Coordination"—is framed with scientific restraint. This thesis does not claim concurrent spatial multi-agent deployment. Deploying multiple physical robots simultaneously in a shared workspace introduces immense additional complexities: radio-frequency bandwidth constraints, physical collision avoidance algorithms, decentralized simultaneous localization and mapping, and distributed task allocation [24, 25]. These are separate research problems beyond this thesis's scope.

Instead, we evaluate multi-agent coordination via *temporal stigmergy*. Stigmergy is a biological mechanism of indirect coordination observed in social insects such as termites and ants [1]. Individual agents do not communicate directly; instead, they alter the environment in ways that stimulate specific responses from subsequent agents. A pheromone trail deposited by one ant triggers following behavior in others. A pellet of mud placed by one termite becomes a stimulus for another to add more mud at that location.

In this thesis, the shared "environment" is algorithmically abstracted into a persistent digital memory ledger—the `factory_state` variable. Agents do not broadcast distress signals or negotiate global plans. Instead, when an agent experiences repeated mechanical failures under compounding faults, it updates the `consecutive_failures` integer in the shared ledger. Subsequent agents, upon initiation, read this trace. If the trace indicates a degraded operational history (two consecutive failures), the new agent autonomously shifts into a localized "recovery" mode: it abandons its randomized spatial target, moves to a conservative centralized coordinate, and doubles its Cartesian error tolerance.

This state-dependent, history-aware behavioral adaptation is the absolute core logic of decentralized swarm coordination. Proving that fast individual self-diagnosis enables such temporal coordination validates the computational prerequisite for eventual physical swarms.

1.6 Research Questions

Four research questions guide this investigation, progressing from architectural efficiency through control integration and fault tolerance to collective adaptation.

1.6.1 RQ1: Architectural Efficiency

To what extent can the computational latency of visual robot self-modeling be reduced by replacing 3D implicit and explicit volumetric rendering paradigms with 2D direct sensorimotor decoding, and what is the resulting impact on morphological reconstruction fidelity measured in PSNR? This question probes the fundamental architectural trade-off between representational complexity and inference speed, seeking to establish whether direct decoding can simultaneously improve both metrics.

1.6.2 RQ2: Control Integration

Can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy and fast inference to actively stabilize a Jacobian-based inverse kinematics control loop? This question moves beyond static rendering metrics to dynamic closed-loop performance, asking whether the network's predictions are smooth and accurate enough to serve as the primary state estimate for feedback control.

1.6.3 RQ3: Fault Tolerance

How does the integration of fast visual self-estimates alter end-effector trajectory and error convergence compared to a purely proprioceptive baseline under induced mechanical faults such as encoder slip? This question evaluates the system's resilience under the very conditions that motivate self-modeling in the first place: when physical damage corrupts proprioceptive readings, can visual estimates override them?

1.6.4 RQ4: Collective Adaptation

Can the individual computational efficiency of a fast self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, thereby validating the temporal stigmergy mechanics required for future multi-agent swarm coordination? This question scales the investigation from single-agent survival to collective endurance.

1.7 Methodological Contributions

This thesis makes four primary contributions to adaptive robotics, neural representation, and decentralized multi-agent systems.

1.7.1 Contribution 1: Formal Benchmarking of the Latency Paradox

The first contribution is the formal benchmarking of the latency paradox in neural rendering. Through reimplementation and comparative analysis of both implicit (FFKSM/NeRF) and

explicit (K-3DGS) neural rendering self-models on a standardized dataset, this research exposes their control incompatibility; reported latencies in the literature remain far above the 1 ms control budget.

1.7.2 Contribution 2: The NeuroKin Direct Sensorimotor Decoder

The second and most significant contribution is the introduction and exhaustive evaluation of the NeuroKin architecture. NeuroKin is a lightweight, fully convolutional direct sensorimotor decoder that completely bypasses volumetric reconstruction, mapping joint angles directly to visual silhouettes and spatial coordinates. In our empirical evaluations, NeuroKin achieves 21.88 dB PSNR with a training-loop throughput of 7,406 samples/s (batch size 32) and a total training time of 33.4 seconds. These results demonstrate that direct decoding can deliver high-fidelity self-modeling at high throughput on the standardized dataset.

1.7.3 Contribution 3: Visual Closed-Loop Fault Adaptation

The third contribution lies in visual closed-loop control and dynamic fault adaptation. This research successfully synthesizes NeuroKin's stereo spatial estimates—derived via a synchronized dual-camera decoding pipeline—with a Jacobian-based inverse-kinematics feedback loop. Through empirical demonstrations of dynamic error recovery following encoder-slip faults, the thesis shows that visual estimates can override corrupted proprioceptive data, allowing the controller to re-stabilize the end-effector trajectory post-fault.

1.7.4 Contribution 4: Temporal Stigmergy for Sequential Swarm Coordination

The fourth contribution is the validation of stigmergic swarm coordination under compounding mechanical faults. The thesis executes a 100-stage continuous sequential swarm simulation under escalating fault injection probabilities. The standard inverse-kinematic baseline collapses to 13% post-fault success, while the NeuroKin-enabled sequence maintains 88% success by reading a shared state (`factory_state`) and triggering decentralized recovery modes. This demonstrates that fast individual self-diagnosis is a computational prerequisite for swarm intelligence.

1.8 Thesis Organization

The remainder of this thesis is structured to systematically address the research questions. Chapter 2 provides an exhaustive critical synthesis of relevant literature, tracing the mathematical evolution of robot self-modeling, analyzing the constraints of NeRF and 3DGS, and establishing the theoretical framework of temporal stigmergy for swarm robotics.

Chapter 3 details the empirical simulation methodology and latency bottleneck, outlining the PyBullet data generation pipeline, the application of Lorenz attractors for chaotic workspace exploration, and providing rigorous quantitative breakdowns of the FFKSM and K-3DGS baseline reproductions.

Chapter 4 formally introduces the core technical innovation—the NeuroKin architecture—detailing its fully convolutional network topology, stereoscopic spatial recovery mechanisms, and the quantitative performance evaluation in our empirical evaluations.

Chapter 5 moves from static rendering metrics to dynamic control, outlining the integration of NeuroKin into an inverse-kinematics loop and providing step-by-step analysis of end-effector convergence under both nominal and fault-injected conditions.

Chapter 6 scales validation to the collective level, unpacking the 100-stage operational swarm chain, explaining the mechanics of the inter-agent communication ledger, and analyzing the statistical success rates of the temporal stigmergy pipeline against cascading baseline failures.

Chapter 7 critically evaluates the overarching hypothesis, analyzing the Pareto trade-offs of sacrificing novel-view synthesis for extreme inference speed, defending the stigmergic approach to swarm coordination, and discussing critical threats to validity—particularly the simulation-to-reality gap and the abstraction of mechanical degradation.

Finally, Chapter 8 synthesizes the research findings, directly answers the formal research questions, and maps a concrete technological roadmap for transitioning these validated sequential capabilities into physically embodied, spatially concurrent swarm deployments.

Chapter 2

Literature Synthesis

2.1 Evolution of Self-Modeling

The capacity for autonomous adaptation to physical damage is the defining characteristic of biological resilience, yet it remains an elusive goal for robotic systems. This section traces the evolution of robotic self-modeling through three distinct eras—symbolic, behavioral, and visual—before examining the computational bottlenecks and theoretical challenges that motivate the approach taken in this thesis.

Figure 2.1 provides a taxonomy of self-modeling approaches to structure the literature review.

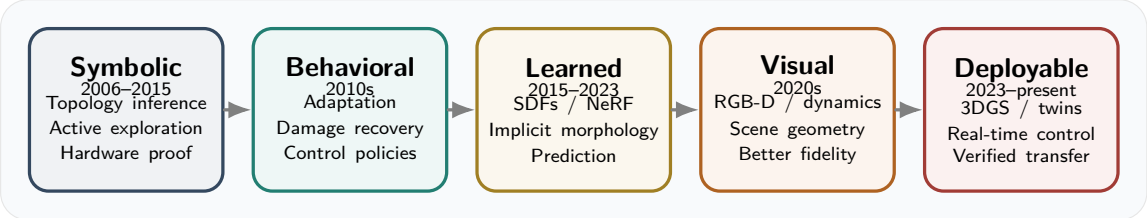

Figure 2.1: Taxonomy of self-modeling approaches across sensing modalities and modeling paradigms.

2.1.1 The Symbolic Era: Estimation-Exploration

The earliest formalized paradigm relied heavily on symbolic and kinematic inference. The seminal work of Bongard, Zykov, and Lipson established the first complete closed-loop self-modeling system on actual physical robots [1]. Their approach viewed self-modeling as a continuous cycle of hypothesis generation and active physical exploration. The robot maintained a population of competing symbolic models representing various candidate topologies—different possible configurations of its own body. To discriminate among these hypotheses, the robot synthesized exploratory motor commands mathematically designed to maximize the sensory disagreement among the candidate models. Upon executing these actions, the robot collected sensor data and systematically pruned models that failed to predict

observed outcomes. After approximately 16 modeling-testing cycles spanning several minutes of real-world experimentation, the robot converged on an accurate damaged model and synthesized a compensatory gait through reinforcement learning on the internal simulation.

The core innovation was an active learning loop: the fitness of a test action was determined by its ability to cause disagreement among the best candidate models, thereby maximally reducing uncertainty. While conceptually elegant, this approach suffers from the symbolic bottleneck: the robot can only discover models constructible from predefined physics engine primitives (cylinders, hinge joints). It cannot represent non-rigid deformations, soft-body dynamics, or arbitrary visual damage—limitations that become critical in unstructured environments where failure modes are unpredictable [4]. As noted by Kwiatkowski and Lipson, the search space of symbolic physics engines is discrete and non-differentiable, making optimization slow and prone to local optima [4]. Furthermore, the approach relied heavily on simple proprioceptive sensors providing limited information compared to high-bandwidth vision.

2.1.2 The Behavioral Era: Avoidance Over Adaptation

Recognizing the computational intensity of explicit physical modeling, Cully and colleagues introduced Intelligent Trial and Error (IT&E), which bypassed damage diagnosis entirely [2]. Their key insight was to focus on finding what still works rather than diagnosing what went wrong. Using the MAP-Elites quality-diversity algorithm [3], they pre-computed a behavioral repertoire of approximately 13,000 distinct gaits in simulation—a process requiring 40 million simulations over two weeks of computation. This repertoire is a high-dimensional grid where each cell corresponds to a specific behavioral descriptor (e.g., duty cycle versus leg offset) and stores the highest-performing controller found for that descriptor.

When damaged in the field, the robot treated this map as a Bayesian prior, using Gaussian Process regression to search for a behavior that performs well in the real world. By testing a few dozen distinct behaviors, the Gaussian Process updated its posterior mean and variance, rapidly identifying a compensatory gait. Cully and colleagues demonstrated recovery in less than two minutes of real-world testing across various damage scenarios, including removal of entire legs [2].

However, IT&E represents a strategy of avoidance, not adaptation. The robot does not repair its internal model—it discards the parts of the behavioral space that no longer function. While effective for redundant locomotion tasks where multiple gaits can achieve similar forward velocity, this approach fails for precision manipulation tasks that require exact geometric awareness [4]. If a robot needs to reach through a narrow aperture, it cannot simply "try a different gait"—it must understand the precise Cartesian geometry of its damaged arm. Furthermore, as robotic complexity increases, the curse of dimensionality makes pre-computing dense behavioral maps prohibitively expensive: a 6-DOF arm with even modest discretization would require billions of map entries, far exceeding computational budgets.

2.1.3 Task-Agnostic Learned Dynamics Models

The advent of deep learning enabled a third paradigm: learning dynamics models that encode the mapping from motor commands to sensory consequences. Kwiatkowski and Lipson proposed task-agnostic self-modeling, training deep forward models to predict the future state of the robot given the current state and action [4]. This represented a pivotal shift from symbolic physics to learnable physics. The same approach was demonstrated to work as a complete reinforcement learning environment to learn zero-shot tasks across morphologies in a follow-up study [5].

While theoretically powerful, these data-driven models face a fundamental limitation: systematic bias accumulates progressively across extended planning horizons [4]. Simulated trajectories diverge increasingly from actual robot behavior, rendering the approach impractical for tasks requiring accurate prediction beyond very short time windows. This horizon-dependent degradation motivates approaches that prioritize shorter planning windows or use learned corrections rather than pure forward simulation.

2.1.4 Proprioceptive Morphology Identification

A parallel thread of research focuses on proprioceptive identification—restoring embodiment using internal signals alone, without the occlusions and lighting failures that often cause vision-only pipelines to break [6]. This offers significant deployment advantages: the approach remains robust across diverse lighting conditions and operates without calibrated cameras. However, eliminating external visual information imposes real functional costs: expressiveness is substantially reduced compared to vision-based methods [8]. This is not merely a theoretical gap but a genuine operating constraint that limits the complexity of tasks these systems can support compared to their multi-modal counterparts.

Meta-self-modeling restores morphology parameters between motion signature reconfigurations [6], and IMU-centric methods show hardware-realized damage sensing with minimal sensing overhead on legged robots [8]. Analytical modular pipelines remain strong baselines in the case of known assembly constraints, offering interpretable, stable, and automatically generated kinematic and dynamic models [7]. The frontier is intervention, not just identification: controlled breakage of proprioception-driven morphology demonstrates that active structural adaptation is supported by internal sensing, and that morphology itself is a form of adaptive feedback control [83, 9].

2.1.5 The Visual Era: Deep Self-Models and Neural Rendering

The current state-of-the-art leverages neural rendering to learn visual self-models directly from camera observations. Chen and colleagues introduced occupancy query learning as a foundational approach [10]. Signed Distance Functions (SDFs) conditioned on joint angles enable differentiable collision checking directly from RGB-D sensor streams, supporting both motion planning and recovery from damage. Their multi-view hardware system achieved ap-

proximately one percent workspace accuracy and demonstrated support for planning and post-damage recovery. However, practical deployment introduced significant challenges: careful camera placement became critical for observability, precise calibration was required for geometric accuracy, and performance degraded substantially under occlusion, shadows, blur, and noise [10].

Hu, Lin, and Lipson reduced the camera system to a single RGB sensor while learning the Free-Form Kinematic Self-Model (FFKSM) from a single viewpoint [11]. The reparameterization of camera frames in terms of the first two joints facilitated the learning process. This approach supports inverse kinematics, path planning, and post-damage recovery. However, volumetric rendering via NeRF remained computationally expensive, and single-view training introduced persistent vulnerabilities: occlusion sensitivity and perspective-dependent predictions.

More recently, Kinematic 3D Gaussian Splatting (K-3DGS) has emerged as an explicit alternative to implicit NeRFs [16]. 3DGS replaces ray marching with differentiable rasterization of explicit anisotropic Gaussians, enabling high frame rate rendering and fast optimization. By attaching explicit Gaussian primitives to a differentiable kinematic chain and leveraging rasterization-based rendering rather than ray-marching, K-3DGS promises faster inference. Each Gaussian primitive is defined by a 3D mean position, covariance matrix, opacity, and color, allowing the model to render the robot in any arbitrary pose through standard graphics pipeline operations.

Yet, as this thesis establishes, the immense representational power of these neural rendering architectures has introduced a severe computational bottleneck that completely obstructs physical deployment in real-time control loops.

2.2 Neural Rendering Foundations: The Speed-Quality Tradeoff

The computational bottleneck of visual self-modeling stems fundamentally from the choice of 3D representation. This section examines the mathematical foundations of Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS), the two dominant paradigms for neural scene representation, while also surveying the broader landscape of articulated object reconstruction and digital twin generation.

2.2.1 Neural Radiance Fields: Implicit Scene Representation

Neural Radiance Fields (NeRFs), introduced by Mildenhall and colleagues, revolutionized computer vision by representing scenes as continuous functions mapping 5D coordinates to color and volume density [19]. The function $F_\theta : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$ is parameterized by a multilayer perceptron (MLP), where $\mathbf{x} \in \mathbb{R}^3$ is a spatial position, $\mathbf{d} \in \mathbb{R}^3$ is a viewing direction, $\mathbf{c} \in \mathbb{R}^3$ is the emitted color, and $\sigma \in \mathbb{R}^+$ is the volume density.

To render a pixel, NeRF approximates the volume rendering integral via quadrature along a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, where $\mathbf{o}$ is the camera origin and $\mathbf{d}$ is the ray direction. The expected color $C(\mathbf{r})$ is:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})\, dt \tag{2.1}$$

where $T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\, ds\right)$ is the accumulated transmittance. In practice, this continuous integral is approximated via numerical quadrature using stratified sampling. For a standard 100×100 image with $N = 64$ samples per ray, this formulation requires $100 \times 100 \times 64 = 640,000$ network evaluations per frame [19]. At 5.22 FPS, this amounts to 3.2 million MLP evaluations per second—a computational burden that our subsequent implementations directly address.

To capture high-frequency geometric details, NeRF employs positional encoding, mapping input coordinates into a higher-dimensional space using sinusoidal functions:

$$\gamma(p) = [\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)] \tag{2.2}$$

For 3D coordinates, this transforms $\mathbf{x} \in \mathbb{R}^3$ to a $3 \times (1 + 2L)$-dimensional vector. Following the original NeRF, we use $L = 5$ frequencies, yielding 33-dimensional encodings.

Standard NeRF assumes a static scene. For articulated robots, the scene geometry changes with joint configuration. D-NeRF and subsequent deformation network variants extend NeRF to dynamic scenes by learning a canonical representation and a deformation field that maps observation-space points to canonical space [20, 65]. For robotic self-modeling, the time variable can be replaced with the joint configuration vector, learning a mapping from configuration space to deformation [11].

Several acceleration techniques have been proposed to address NeRF's computational cost. Instant-NGP uses a multiresolution hash encoding that dramatically reduces the required MLP size, achieving real-time rendering for static scenes [21]. Mip-NeRF addresses aliasing by rendering anti-aliased conical frustums rather than rays, improving fidelity at the cost of additional computation [18]. Zip-NeRF further extends this with grid-based acceleration [22]. However, even with acceleration, the core robotics bottleneck remains fundamentally unchanged: volumetric query expense remains a fundamental constraint within tight control loops.

2.2.2 3D Gaussian Splatting: Explicit Geometric Primitives

3D Gaussian Splatting (3DGS), introduced by Kerbl and colleagues, replaces implicit neural fields with explicit discrete geometric primitives [23]. By abandoning the MLP entirely during inference and relying on highly optimized rasterization pipelines, 3DGS promises a pathway to real-time, high-fidelity scene representation.

Unlike NeRF, which models a continuous field, 3DGS represents a scene as an un-

structured point cloud of discrete 3D Gaussian primitives. Each individual Gaussian is mathematically defined by a 3D mean position $\boldsymbol{\mu}_i \in \mathbb{R}^3$, an anisotropic covariance matrix $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3\times3}$ (parameterized by scaling $\mathbf{S}$ and rotation $\mathbf{R}$ via $\boldsymbol{\Sigma} = \mathbf{RSS}^\mathsf{T}\mathbf{R}^\mathsf{T}$), an opacity scalar $\alpha_i \in [0, 1]$, and spherical harmonics coefficients representing view-dependent directional color.

The core representational power of 3DGS lies in the anisotropy of the covariance matrix. A purely isotropic (spherical) Gaussian is severely limited in its ability to model sharp edges or flat surfaces. By allowing the covariance matrix to stretch and compress unevenly along its principal axes, a single Gaussian can deform into an elongated ellipsoid or a flattened disk, enabling precise modeling of complex geometry with fewer primitives [23].

Optimizing a 3×3 covariance matrix directly using gradient descent is mathematically precarious, as the matrix must remain strictly positive semi-definite. To enforce this constraint, Kerbl and colleagues factorize the covariance matrix into a scaling vector $\mathbf{s} \in \mathbb{R}^3$ (with $s_i > 0$) and a rotation quaternion $\mathbf{q} \in \mathbb{R}^4$, then reparameterize the optimization to maintain positive semi-definiteness [23].

The true speed advantage of 3DGS is derived from its rendering pipeline. To synthesize an image, 3DGS completely abandons ray-marching, instead utilizing a highly parallelized tile-based rasterization architecture. For a given camera with viewing transformation $\mathbf{W}$ (world-to-camera) and projection $\mathbf{J}$ (Jacobian of the perspective projection), the projected 2D covariance $\boldsymbol{\Sigma}'$ is:

$$\boldsymbol{\Sigma}' = \mathbf{JW}\boldsymbol{\Sigma}\mathbf{W}^\mathsf{T}\mathbf{J}^\mathsf{T} \tag{2.3}$$

The 2D Gaussian function for pixel coordinates $\mathbf{u} \in \mathbb{R}^2$ is $G^{2D}(\mathbf{u}) = \exp\left(-\frac{1}{2}(\mathbf{u} - \boldsymbol{\mu}')^\mathsf{T}\boldsymbol{\Sigma}'^{-1}(\mathbf{u} -\right.$ and the final pixel color is computed via alpha compositing with Gaussians sorted by depth:

$$C(\mathbf{u}) = \sum_{i=1}^{N} \alpha_i G_i^{2D}(\mathbf{u})\mathbf{c}_i \prod_{j=1}^{i-1}(1 - \alpha_j G_j^{2D}(\mathbf{u})) \tag{2.4}$$

The rapid adoption of 3DGS within robotics has enabled real-time SLAM systems. Gaussian Splatting SLAM [24] and SplaTAM [25] integrate 3DGS into SLAM pipelines, achieving real-time mapping and tracking with high-fidelity scene reconstruction. Surface regularization methods like SuGaR enable mesh extraction from Gaussian splats, bridging the gap between rendering and geometric simulation [27]. However, the computational cost scales with the number of Gaussians. For embedded deployment on edge devices like NVIDIA Jetson Orin, the parallel radix sort required for alpha-blending becomes memory-bandwidth bound, as shown by detailed benchmarking on mobile hardware [26]. The Jetson Orin operates under stricter power (15W typical) and memory bandwidth constraints (204 GB/s) compared to desktop GPUs, necessitating sparse, budgeted representations for self-repairing robots that must render hundreds of candidate morphologies per second.

2.2.3 The Failure of Anisotropy on Robotic Morphologies

Given these computational advantages, the robotics literature adapted 3DGS for kinematic self-modeling. The resulting architecture, Kinematic 3DGS (K-3DGS), attaches Gaussian primitives to a differentiable kinematic tree [16].

In theory, the anisotropic nature of 3DGS makes it ideal for robotic manipulators. The physical morphology of a serial manipulator is dominated by long, thin, rigid cylindrical links. An optimizer should naturally stretch the scaling vectors of Gaussians along the central longitudinal axis of each link, achieving aspect ratios of 20:1:1.

In practice, however, this anisotropic optimization frequently fails catastrophically when applied to sparse, single-camera robotic datasets. Optimizing highly anisotropic rotational quaternions on featureless robotic links introduces a severely non-convex, ill-posed optimization landscape [16]. Because the robotic link is visually uniform, the gradient descent algorithm receives contradictory loss signals regarding the optimal rotation of the elongated Gaussian. This phenomenon is termed *anisotropic mode collapse*.

To maintain numerical stability and ensure reliable convergence, practical K-3DGS implementations are frequently forced to retreat to purely isotropic representations. The scaling vector is constrained to be equal in all dimensions, forcing every Gaussian primitive to remain a perfect sphere. While this restriction mathematically stabilizes the training process, it fundamentally compromises the structural precision of the representation [16]. Attempting to model a long, sharp-edged robotic cylinder using only perfect spheres results in severe geometric aliasing. When rasterized, this isotropic chaining yields pervasive, structurally ambiguous "blobby" artifacts that blur the true spatial boundaries of the robot.

2.2.4 Articulated Object Reconstruction for Digital Twin Generation

Automated digital twin generation approaches follow three distinct strategies reflecting different burden allocation decisions [60, 28, 29].

Differentiable reconstruction pipelines avoid manual URDF creation by optimizing directly from observations [60, 28]. Heiden and colleagues introduced a differentiable rendering pipeline that infers articulated rigid body dynamics directly from RGBD video, enabling the recovery of kinematic structure and joint parameters without hand-coding URDF files [60]. These methods use differentiable renderers to compare predicted images with observations, backpropagating error to update both geometry and articulation parameters. However, these methods are computationally expensive during optimization and may converge to local optima. Weng and colleagues extended this paradigm with neural implicit representations for building digital twins of unknown articulated objects, demonstrating reconstruction from sparse multi-view observations [28].

Feed-forward generative models prioritize generation speed and simplicity over polish, deferring validation to downstream tasks. ART (Articulated Reconstruction Trans-

former) uses a transformer architecture to directly predict articulation parameters from multi-view images, achieving rapid reconstruction without per-object optimization [29]. URDF-Anything+ autoregressively generates URDF files from point clouds, enabling simulation-ready asset creation from depth sensors [31]. ArtLLM leverages large language models for articulated asset generation, using semantic reasoning about part connectivity to infer kinematic structure [35]. Kinematify enables open-vocabulary synthesis of high-DoF articulated objects through language-guided part decomposition [33]. These methods achieve rapid generation but may produce physically implausible geometries that fail under contact-rich manipulation.

Physics-constrained variants target specific failure modes like mesh interpenetration, adding domain knowledge at the cost of complexity. SIMART decomposes monolithic meshes into simulation-ready articulated assets using multimodal LLMs, explicitly checking for kinematic validity before deployment [34]. MotionAnymesh provides physics-grounded articulation for simulation-ready digital twins, enforcing joint limit constraints and collision geometry consistency [32]. AOMGen generates photorealistic, physics-consistent demonstrations for articulated object manipulation, focusing on visual and dynamic realism [30].

For manipulation tasks requiring closed-loop control, explicit world models offer an alternative to reconstruction-focused pipelines. Li and colleagues proposed building explicit world models for zero-shot open-world object manipulation, separating geometric reconstruction from dynamics prediction [45]. This decomposition enables the robot to reason about object geometry independently from task execution, improving generalization to novel environments. ArtGS extends this principle to 3D Gaussian splatting, providing interactive visual-physical modeling and manipulation of articulated objects with real-robot validation on Franka manipulators and cross-embodiment transfer to xArm-class systems [37]. REArtGS++ achieves generalizable articulation reconstruction with temporal geometry constraints via planar Gaussian splatting, addressing occlusion challenges that degrade single-frame methods [38].

A consistent limitation emerges across all three strategies: robustness properties fail acutely under realistic occlusion. Dense multi-object scenes with occlusion degrade inference sharply, exposing that single-model approaches lack robustness properties required for real deployment scenarios [29, 34]. Multi-joint coupling spreads errors across morphologically related structures, and unconstrained capture generates visually plausible reconstructions that mask planning-critical geometric errors. These limitations underscore the value of the minimal representation principle demonstrated by NeuroKin: for control tasks requiring only silhouette-level awareness, the overhead of full 3D reconstruction may be unnecessary.

2.2.5 Physics-Consistent Robot Models and Manipulation Planning

Gaussian twin representations have progressed from display assets toward functional planning models. Robo-GS couples Gaussian representations with mesh extraction because simulators require solid geometry for accurate dynamics computation [42]. ArticulatedGS enables

self-supervised digital twin modeling of articulated objects using 3D Gaussian splatting [36]. ArtGS provides interactive visual-physical modeling and manipulation of articulated objects [37]. REArtGS++ achieves generalizable articulation reconstruction with temporal geometry constraints via planar Gaussian splatting [38]. FreeArtGS extends articulated Gaussian splatting to free-moving scenarios [39]. FreeGaussian enables annotation-free control of articulated objects via flow derivatives [40].

Planning stacks increasingly merge volumetric rendering with trajectory optimization for centimeter-level precision [44]. ManiGaussian uses dynamic Gaussian splatting for multi-task robotic manipulation [46]. Predictive variants detect divergence between simulation and actual behavior, triggering corrective updates automatically when model accuracy degrades [47]. GSWorld provides closed-loop photorealistic simulation for robotic manipulation [55]. GaussGym offers an open-source real-to-sim framework for learning locomotion from pixels [56].

However, a persistent limitation emerges across published work: most papers demonstrate strong simulation metrics only over short horizons. Sensing robustness under environmental variation remains limited, latency jitter testing is absent, and contact uncertainty validation is incomplete [42, 47]. These three conditions—horizon length, environmental robustness, and contact fidelity—ultimately determine whether learned models generalize to deployment scenarios or remain confined to laboratory settings.

2.2.6 Cross-Embodiment Transfer and Morphological Understanding

Cross-embodiment transfer performance improves measurably when morphological structure is encoded as kinematic connectivity rather than visual appearance alone. DexGrasp-Zero aligns heterogeneous hand morphologies via semantic kinematic graphs, achieving zero-shot transfer in selected scenarios [49]. The generalization of embodiment-aware attention in GET-Zero is better than morphology-agnostic attention when dealing with unseen graphs and geometry variants [50]. Success is driven by structural priors, not textures. Transfer fails when the kinematic structure differs substantially.

Whole-body control frameworks increasingly leverage these insights for humanoid deployment. AGILE provides a comprehensive workflow for humanoid loco-manipulation learning [58]. Whole-body coordination frameworks couple base and arm dynamics for dynamic object grasping with legged manipulators [51]. Cybo-Waiter offers a physical agentic framework for humanoid whole-body locomotion-manipulation [82]. EAGLE uses embodiment-aware generalist-specialist distillation for unified humanoid whole-body control [80]. HAIC enables humanoid agile object interaction control via dynamics-aware world models [70]. These frameworks demonstrate that explicit structure priors substantially facilitate transfer across embodiments.

2.2.7 Sim-to-Real Transfer via Digital Twins

Equating rendering fidelity with deployment success is a flawed assumption that has led to numerous simulation-only results failing on hardware [57]. The reality gaps decompose along four distinct dimensions: dynamics mismatch (physics simulation divergence), perception mismatch (sensor modeling inaccuracy), actuation mismatch (motor characteristics divergence), and system construction mismatch (morphological/structural misalignment) [57, 41].

Effective transfer combines targeted system identification with methodical robustness training rather than pursuing perfect simulator fidelity. Hardware evidence supports this principle: quadrupeds trained with actuator-aware system identification and domain randomization achieve zero-shot transfer to real platforms [41]. Bicycles trained with explicit robustness-focused procedures transfer reliably across diverse terrain [75]. A consistent pattern emerges across successful systems: calibration without concurrent robustness training fails catastrophically, while robustness training without accurate baseline calibration merely wastes training data without achieving true transfer capability [41, 57].

Digital twins play multiple roles in sim-to-real pipelines as summarized in Table 2.1. Dynamics and system identification calibrate physics parameters (mass, friction, actuation, contact) so simulated rollouts reflect hardware behavior [41, 60, 43]. Perception and rendering transfer reduce visual domain gaps by reconstructing photorealistic scenes or preserving semantic invariants for training and validation [68, 69]. Demonstration synthesis and augmentation generate diverse trajectories and viewpoints from sparse real demonstrations by manipulating reconstructed scene components [48, 71]. Deployment-time synchronization uses a continuously synchronized simulator as a mediator, where the policy acts on the twin while the robot tracks twin trajectories [61]. Evaluation and sim-real correspondence estimate whether simulator performance correlates with hardware outcomes, enabling safer iteration before deployment [84, 85].

RoboSplat demonstrates concrete quantified impact: achieving 87.8% task success compared to 57.2% baseline, while simultaneously enabling 29× faster demonstration generation [48]. SplatSim enables zero-shot sim2real transfer of RGB manipulation policies using Gaussian splatting [68]. Real2Render2Real scales robot data without dynamics simulation or robot hardware [71]. D-REX provides differentiable real-to-sim-to-real engine for learning dexterous grasping [43]. UniPR achieves simultaneous 100× speedup in scene processing and accuracy improvement for real-to-sim perception [63]. VR-Robo demonstrates successful RGB-only transfer in complex, previously unseen environments [74]. NavGSim shows that navigation policies trained in simulation transfer directly to real environments [77].

Table 2.1: Roles of digital twins in sim-to-real pipelines.

Role	Representative Works
Dynamics / system identification	[41, 60, 43]
Perception / rendering transfer	[68, 69]
Demonstration synthesis / augmentation	[48, 71]
Deployment-time synchronization	[61]
Evaluation / sim-real correspondence	[84, 85]

2.3 Damage Recovery and Fault Tolerance

Damage recovery is where self-model claims face the most severe test: can the system operate with real faults, not just under nominal simulation conditions? This section surveys detection methods, recovery policy generation, humanoid-specific challenges, and practical deployment lessons.

2.3.1 Detection Methods and Early Intervention

Early fault detection methods are valuable only when directly tied to practical intervention latency and downstream control outcomes. VAE-based anomaly monitoring successfully detects anomalous force-torque trajectory patterns at sub-3-millisecond latencies in simulated manipulation tasks [94]. Contrastive forward prediction reduces tracking error by up to 61.5% on damaged legged machines [95]. Low-latency detection remains critical because delayed intervention necessarily closes the available recovery window, making response speed a fundamental constraint.

2.3.2 Recovery Policy Generation and Adaptation

Detection must translate to actionable recovery policies. Vision-language recovery can guide task-level correction on 7-DoF manipulators; however, unconstrained language reasoning often violates dynamics limits [96]. Structured recovery stacks provide a pragmatic alternative. Dream2Fix synthesizes counterfactual failures on a large scale (120,000+ simulation cases), achieving 46% zero-shot closed-loop recovery on hardware [101]. Embodiment-conditioned

diffusion policies achieve higher constraint satisfaction (36.85% to 74.51%) under zero-shot generalization to unseen failure conditions [98].

2.3.3 Humanoid Recovery and Stability

Humanoid recovery introduces substantial additional complexity through stability and safety constraints not present in manipulator systems. Balance-informed critic networks grounded in capture point theory and centroidal dynamics computation achieve strong simulated stand-up recovery from falls [99]. Preference-conditioned control policies expand operational robustness envelopes by dynamically switching between efficiency-optimized and robustness-maximized control modes [100]. Fault-aware locomotion methods maintain stable tracking performance under joint faults within simulation [102]. Humanoid parkour research demonstrates impressive agile obstacle vaulting and long-horizon terrain traversal capabilities in controlled environments [66].

2.3.4 Practical Deployment Lessons

Two practical lessons emerge consistently across successful deployment systems. First, online discrepancy monitoring proves effective when adaptation remains rapid and theoretically bounded, preventing unbounded divergence. World-model residual monitoring specifically enables rapid policy recovery following physical domain shifts like payload changes or link damage [89]. Second, residual adaptation channels remain most effective when carefully aligned with baseline policy safety constraints. This principle is demonstrated empirically: bounded residual control reduced Go1 quadruped recovery steps from 4,500 to 168 following a significant mass increase—a 96% reduction in recovery time establishing the practical value of principled multi-level adaptation [97].

A consistent deployment principle emerges: recovery systems should be evaluated by intervention speed, stability margin, and post-fault task continuity—not by reconstruction fidelity. Visual quality is not the determining factor.

2.4 Continual Learning for Self-Model Maintenance

Continual maintenance of self-models confronts the stability-plasticity dilemma: updates must absorb new damage conditions while preserving knowledge of healthy morphology [105, 106]. Catastrophic forgetting is well documented; in robotic deployment it is critical because forgetting can generate unsafe actions and hardware destruction [103, 104].

2.4.1 Balancing Plasticity and Stability

Meta-learning approaches contribute meaningful value when morphology characteristics shift frequently and the time window available for adaptation compresses severely. Model-agnostic meta-learning (MAML) reduces post-damage sample requirements for model training [111],

and structure-constrained updates enable adaptation without full retraining. Elastic Weight Consolidation (EWC) mitigates catastrophic forgetting by approximating the posterior distribution of weights using the Fisher information matrix [107]. However, EWC is ill-suited for robotic damage recovery for two reasons. First, computational cost: calculating the Fisher Matrix requires computing second-order derivatives (Hessian) or approximating them via squared gradients. For high-dimensional rendering networks (millions of parameters), storing and computing the Fisher information is prohibitively expensive for real-time adaptation on edge hardware. Second, lack of modularity: EWC slows forgetting but does not enable the robot to isolate the damage. It treats the network as a monolith with varying "stiffness". If damage is severe, the robot needs to change high-importance weights, but EWC prevents this, leading to under-fitting of the damage (the "plasticity gap").

The failure of weight-based regularization suggests that topology, rather than weights, may hold the key to adaptive resilience. Research by Chechik and colleagues demonstrated through computational models that synaptic pruning is an optimal strategy strictly under metabolic or resource constraints [117]. They proved that in the absence of such energetic costs, pruning cannot improve network performance compared to an intact network—a finding that aligns precisely with edge compute constraints in robotics, where power budgets (15W typical for Jetson Orin) and memory bandwidth are severely limited. The biological precedent suggests that sparse, modular architectures may be not merely beneficial but necessary for resource-constrained adaptive systems.

2.4.2 Resource-Efficient Adaptation via Sparsity

The Lottery Ticket Hypothesis formalizes the value of sparsity in deep learning: "A randomly-initialized dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations" [108]. This suggests that the vast majority of weights in a dense self-model are redundant. Han and colleagues empirically demonstrated this, achieving compression ratios of 49× on AlexNet and 39× on VGG-16 through a combination of pruning, quantization, and Huffman coding [110]. Model storage reduced from 240 MB to 6.9 MB while maintaining baseline accuracy.

Sparse Evolutionary Training (SET) maintains sparse topology throughout training rather than training dense and pruning post-hoc [109]. In each epoch, SET prunes the weakest connections and regrows random new ones, allowing the network to explore the topology space dynamically. For robotics, sparsity offers dual benefits. First, efficiency: sparse matrix operations reduce computational latency, directly addressing the rendering bottleneck. Second, modularity: by enforcing sparsity, the network is forced to develop modular sub-structures. Damage to one module can be repaired by updating only the weights within that module, significantly reducing catastrophic interference [108, 109]. This biological inspiration—sparse, modular connectivity enabling localized plasticity—motivates our architectural explorations in subsequent sections.

2.4.3 Safety-Constrained and Bounded Adaptation

Safety-constrained learning maintains adaptation within verified regions through explicit constraints. Barrier functions enforce that the robot stays within safe regions of state space, providing formal safety guarantees [88]. System-level state partitioning separates high-level reasoning from time-critical control loops, reducing cascading failures. Bounded residual control keeps adaptations within safe envelopes, as demonstrated empirically by the 96% reduction in recovery time on Go1 quadruped [97].

2.5 Theoretical Foundations of Swarm Coordination and Temporal Stigmergy

This section establishes the theoretical framework of temporal stigmergy that underpins the sequential swarm evaluation in Chapter 6, drawing on biological inspiration and multi-agent systems research.

2.5.1 Stigmergy: Biological and Algorithmic Foundations

Stigmergy is a biological mechanism of indirect coordination, originally observed in social insects such as termites and ants. Individual agents do not communicate directly via explicit signaling protocols or negotiated consensus algorithms. Instead, they subtly alter the physical environment—for example, depositing a pheromone trail or adding a pellet of construction material. These environmental modifications serve as localized, persistent stimuli, triggering specific behavioral responses from subsequent agents that encounter them.

The critical property of stigmergy is that coordination emerges from local interactions without centralized planning or direct agent-to-agent communication. The environment itself becomes the communication channel. This property makes stigmergy particularly attractive for robotic swarms operating in communication-denied environments where radio bandwidth is limited or connectivity is intermittent.

Translated into algorithmic robotic systems, stigmergy eliminates the need for continuous peer-to-peer communication bandwidth. Agents react independently and locally to diagnostic traces left in the environment by their peers [1]. This stands in contrast to consensus algorithms that require all-to-all communication ($O(N^2)$ messages) and multi-agent reinforcement learning approaches that require centralized training and substantial computational resources [89, 86].

2.5.2 Temporal Stigmergy in Sequential Pipelines

In this thesis, stigmergic coordination is achieved across time rather than physical space. The shared "environment" is abstracted into a persistent digital memory ledger—the `factory_state` variable tracked across sequential agents. This represents a *temporal stigmergy*: agents co-

ordinate by reading and writing to a shared state that persists across sequential operations, with the dimension of coordination being time rather than space.

Formally, we define temporal stigmergy as a coordination mechanism where agents $\{A_1, A_2, \ldots, A_N\}$ share a persistent state $S \in \mathcal{S}$. Each agent A_i executes the following protocol:

1. Reads the current state S_{i-1} (the state after the previous agent's execution).
2. Selects behavior $B_i = \pi(S_{i-1})$ according to a policy $\pi : \mathcal{S} \to \mathcal{B}$.
3. Executes the behavior and observes outcome $O_i \in \{\text{success}, \text{failure}\}$.
4. Updates the state: $S_i = \Phi(S_{i-1}, O_i)$ where Φ is the state transition function.

Coordination emerges from the sequential application of π and Φ without any direct communication between agents beyond the state S that persists in the shared environment. When an agent experiences repeated mechanical failures, it updates the `consecutive_failures` integer. Subsequent agents read this trace. If the trace indicates a degraded operational history (two consecutive failures), the new agent autonomously shifts into a localized "recovery" mode: it abandons its randomized spatial target, moves to a conservative centralized coordinate, and doubles its Cartesian error tolerance. This state-dependent, history-aware behavioral adaptation is the core logic of decentralized swarm coordination.

2.5.3 Fast Inference as a Swarm Prerequisite

A swarm can only coordinate around diagnostic states if individual agents possess the capability to produce those states reliably and in a timely manner. If an agent's internal self-model requires hundreds of milliseconds per inference, it cannot reliably determine whether a failure was caused by occlusion, oscillation, or morphological drift—by the time the diagnosis completes, the system state may have changed. Our empirical evaluations show that NeuroKin provides accurate visual self-diagnosis with high throughput, establishing the computational prerequisite for swarm endurance.

Fast inference enables three properties essential for stigmergic coordination: (1) real-time failure detection (the agent knows within a single control cycle whether it has succeeded or failed); (2) immediate ledger updates (the shared state is updated before the next control cycle begins); and (3) rapid adaptation (subsequent agents read the updated state without perceptible delay). Without fast inference, temporal stigmergy would introduce latency proportional to the number of agents, causing the coordination mechanism to become the bottleneck.

2.6 Summary of Literature Findings

This exhaustive literature synthesis reveals several critical findings that directly motivate the experimental contributions of this thesis.

First, visual self-modeling via neural rendering (NeRF and 3DGS) has achieved impressive reconstruction fidelity, with reported PSNR values exceeding 23 dB on benchmark datasets. However, the field has systematically under-emphasized the latency implications of volumetric reconstruction for real-time control. The reported frame rates (5-37 FPS) are fundamentally incompatible with the 1 kHz (1,000 Hz) control loops required for dynamic robotic systems. This latency paradox represents a structural mismatch, not an engineering optimization problem.

Second, direct sensorimotor decoding—bypassing explicit 3D reconstruction entirely—has been explored in limited contexts but never systematically compared against volumetric approaches on controlled benchmarks. The theoretical advantage of dense gradient supervision suggests that direct decoding could achieve both higher speed and better quality, but this hypothesis requires rigorous empirical validation.

Third, continual learning for self-model maintenance faces unresolved challenges around catastrophic forgetting. Existing approaches (EWC, sparse subnetworks, mixture-of-experts) offer partial solutions but have not been validated in damage recovery scenarios where the distribution shifts abruptly and safety constraints dominate. The biological precedent of synaptic pruning under resource constraints suggests that sparse, modular architectures may be necessary for edge-deployable adaptive systems.

Fourth, digital twin frameworks for robotics have emphasized geometric fidelity and simulation accuracy but have neglected the temporal dimension of self-modeling: the twin must be queryable at control rates to be useful for online adaptation. This temporal requirement aligns with the latency findings from neural rendering analysis. Recent work on Robo-GS, ArticulatedGS, and differentiable simulation pipelines demonstrates progress, but the evaluation gap—simulation metrics over short horizons versus deployment metrics over extended operation—remains unresolved.

Fifth, stigmergic coordination offers a promising framework for decentralized multi-agent fault tolerance, but the prerequisite—fast, reliable individual self-diagnosis—has not been demonstrated in prior work. Temporal stigmergy, as formalized in this thesis, provides a theoretical bridge from biological inspiration to algorithmic implementation, with the critical enabling condition being fast inference.

Sixth, across all surveyed literature, a consistent evaluation gap emerges: most papers report reconstruction metrics (PSNR, SSIM, LPIPS) but not task-level outcomes (success rate, recovery time, stability margins). This disconnect between diagnostic metrics and deployment metrics limits the field's ability to assess practical readiness. Hardware validation remains the gold standard, yet simulation-only results continue to dominate the literature.

These findings collectively motivate the experimental investigation presented in the following chapters, which progresses from baseline implementation through architectural innovation to multi-agent validation.

2.7 Research Questions Grounded in Literature

Based on the literature synthesis, four research questions guide the experimental investigation. These questions emerge directly from identified gaps rather than being imposed a priori.

2.7.1 RQ1: Architectural Efficiency

To what extent can the computational latency of visual robot self-modeling be reduced by replacing 3D implicit and explicit volumetric rendering paradigms with 2D direct sensorimotor decoding, and what is the resulting impact on morphological reconstruction fidelity measured in PSNR? This question probes the fundamental architectural trade-off that the literature has systematically under-explored. The hypothesis is that dense gradient supervision from direct pixel prediction will achieve both higher speed and better quality than sparse volumetric sampling.

2.7.2 RQ2: Control Integration

Can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy and fast inference to actively stabilize a Jacobian-based inverse kinematics control loop? This question moves beyond static rendering metrics to dynamic closed-loop performance, asking whether the network's predictions are smooth and accurate enough to serve as the primary state estimate for feedback control—a capability not demonstrated for any neural rendering self-model to date.

2.7.3 RQ3: Fault Tolerance

How does the integration of fast visual self-estimates alter end-effector trajectory and error convergence compared to a purely proprioceptive baseline under induced mechanical faults such as encoder slip? This question evaluates the system's resilience under the very conditions that motivate self-modeling in the first place: when physical damage corrupts proprioceptive readings, can visual estimates override them and maintain stable control? The literature on bounded residual control and world-model monitoring suggests that rapid adaptation is possible, but the prerequisite fast inference has not been demonstrated.

2.7.4 RQ4: Collective Adaptation

Can the individual computational efficiency of a fast self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, thereby validating the temporal stigmergy mechanics required for future multi-agent swarm coordination? This question scales the investigation from single-agent survival to collective endurance, testing whether fast individual self-diagnosis can support emergent coordination without explicit inter-agent

communication. The information-theoretic efficiency of stigmergy—coordination with O(1) bits per agent—makes it particularly attractive for bandwidth-constrained swarms.

The following chapter details the experimental methodology for addressing these research questions, beginning with the baseline implementations that establish the latency bottleneck.

Chapter 3

Baseline Implementations and the Latency Bottleneck

To rigorously characterize the latency paradox that motivates this thesis, we implement and evaluate two state-of-the-art visual self-modeling architectures: the Free-Form Kinematic Self-Model (FFKSM) based on implicit neural radiance fields [11], and Kinematic 3D Gaussian Splatting (K-3DGS) based on explicit geometric primitives [16]. Both are reproduced from published specifications using an identical dataset, hardware environment, and evaluation protocol. This controlled comparison establishes the quantitative baseline against which our proposed NeuroKin architecture is measured.

3.1 Synthetic Data Generation via Lorenz Attractors

All experiments use a dataset of 2,000 samples generated from a PyBullet simulation of a 4-DOF serial manipulator. This section details the complete data generation pipeline, including simulation configuration, trajectory planning, and preprocessing procedures.

3.1.1 Simulation Environment Configuration

The PyBullet physics simulator provides a controlled environment for generating ground-truth robot observations. We configure the simulation with the following parameters:

- **Robot Model**: A 4-degree-of-freedom serial manipulator with revolute joints. The base is fixed at the origin with joint axes arranged in an anthropomorphic configuration: joint 1 rotates about the vertical axis (yaw), joints 2 and 3 rotate about horizontal axes (pitch), and joint 4 provides wrist rotation.
- **Camera Setup**: A fixed RGB camera positioned at coordinates $[1.0, 0.0, 0.0]$ with focal length 130.2545. The camera looks toward the origin, producing 100×100 pixel images with a field of view of 42 degrees.

- **Rendering**: The simulation runs in DIRECT mode (no graphical display) for maximum speed. Each step renders the robot using PyBullet's built-in tiny renderer with shadows disabled.
- **Physics**: Gravity is set to $-9.8\,\mathrm{m/s^2}$ in the z-direction. A ground plane is placed at $z = -0.109$ m. Each joint is position-controlled with a maximum force of 100 N.

Figure 3.1 shows the simulation scene and camera placement used for data generation.

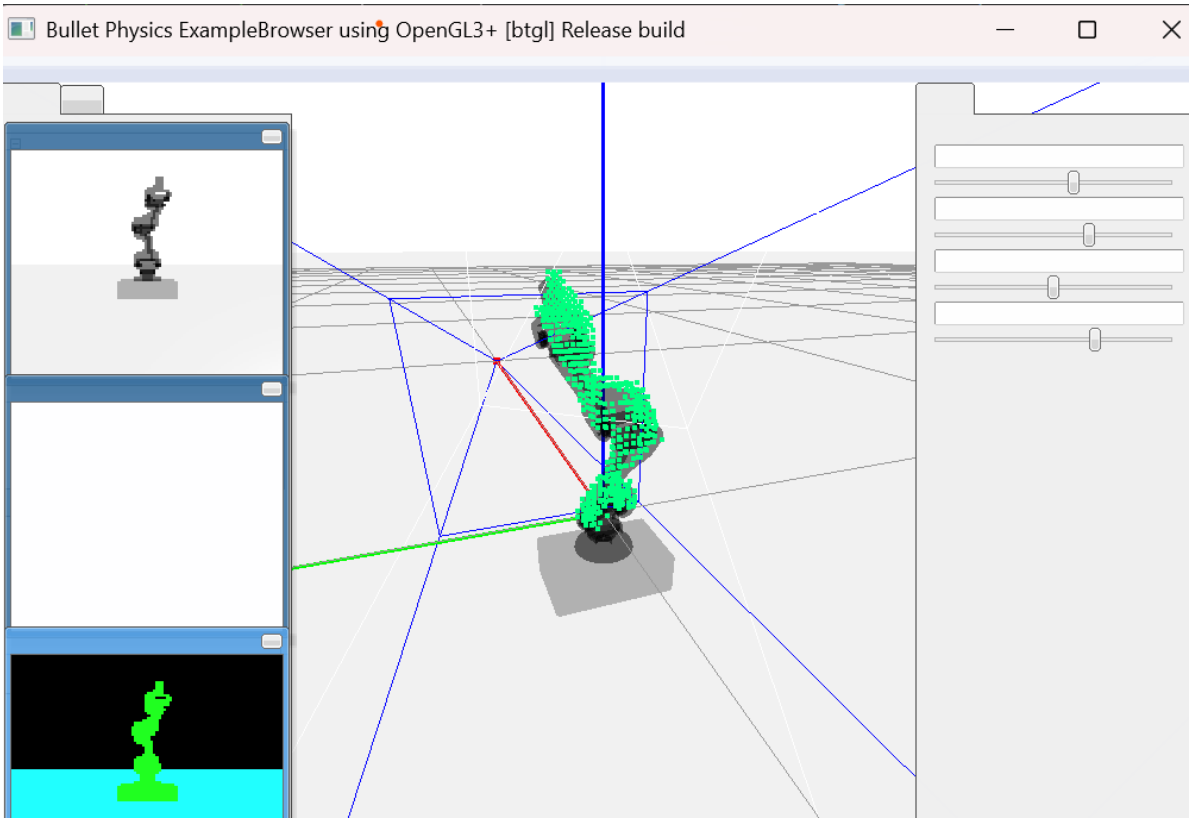

Figure 3.1: PyBullet simulation setup and fixed camera viewpoint for dataset generation.

The robot's kinematic structure follows the Denavit-Hartenberg (DH) convention. The DH parameters are:

Table 3.1: Denavit-Hartenberg parameters for the 4-DOF manipulator.

Link i	α_{i-1} (rad)	a_{i-1} (m)	d_i (m)	θ_i (var)
1	0	0	0.1	θ_1
2	$\pi/2$	0	0	θ_2
3	0	0.4	0	θ_3
4	0	0.4	0	θ_4

The forward kinematics transformation from base to end-effector is given by the product of individual link transformations:

$$T_0^4 = T_0^1(\theta_1)T_1^2(\theta_2)T_2^3(\theta_3)T_3^4(\theta_4) \tag{3.1}$$

where each T_{i-1}^i is computed using the standard DH convention:

$$T_{i-1}^i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_{i-1} \\ \sin\theta_i \cos\alpha_{i-1} & \cos\theta_i \cos\alpha_{i-1} & -\sin\alpha_{i-1} & -d_i \sin\alpha_{i-1} \\ \sin\theta_i \sin\alpha_{i-1} & \cos\theta_i \sin\alpha_{i-1} & \cos\alpha_{i-1} & d_i \cos\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3.2}$$

3.1.2 Lorenz Attractor Trajectory Generation

Rather than relying on random motor babbling—which produces jerky, discontinuous motions that poorly sample the workspace—we employ trajectories generated by the Lorenz dynamical system. The Lorenz attractor is a system of three coupled ordinary differential equations (ODEs) that exhibits deterministic chaos, producing trajectories that are smooth, non-repeating, and ergodic within a bounded region of state space.

The Lorenz system is defined as [115]:

$$\frac{dx}{dt} = \sigma(y - x) \tag{3.3}$$

$$\frac{dy}{dt} = x(\rho - z) - y \tag{3.4}$$

$$\frac{dz}{dt} = xy - \beta z \tag{3.5}$$

The parameter values determine the dynamical behavior. We use the classical values $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, which place the system in the chaotic regime. At these parameters, the Lyapunov exponent is positive, ensuring sensitive dependence on initial conditions and exponential divergence of nearby trajectories. The system has two unstable fixed points at $(\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1)$ and a stable fixed point at the origin (which is only stable for $\rho < 1$). For $\rho = 28$, the origin is a saddle point, and trajectories are attracted to a strange attractor—a fractal set of measure zero with non-integer Hausdorff dimension.

To numerically integrate the Lorenz system, we employ the fourth-order Runge-Kutta (RK4) method. For a general ODE $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$, the RK4 update from state $\mathbf{x}_n$ at time t_n to $\mathbf{x}_{n+1}$ at time $t_{n+1} = t_n + \Delta t$ is:

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{x}_n, t_n) \tag{3.6}$$

$$\mathbf{k}_2 = \mathbf{f}(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_1, t_n + \frac{\Delta t}{2}) \tag{3.7}$$

$$\mathbf{k}_3 = \mathbf{f}(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_2, t_n + \frac{\Delta t}{2}) \tag{3.8}$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{x}_n + \Delta t\mathbf{k}_3, t_n + \Delta t) \tag{3.9}$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \tag{3.10}$$

RK4 is a fourth-order method, meaning the local truncation error is $O(\Delta t^5)$ and the global error is $O(\Delta t^4)$. This provides excellent stability and accuracy for the Lorenz system, which is stiff in certain regions of state space.

We set the integration time step to $\Delta t = 0.01$ seconds. This value was chosen through systematic experimentation: larger steps ($\Delta t \geq 0.02$) caused numerical instability and trajectory divergence, while smaller steps ($\Delta t \leq 0.005$) increased computational cost without measurable improvement in trajectory quality.

After each RK4 step, the raw Lorenz state values are scaled using the hyperbolic tangent function:

$$\text{action} = \tanh(\alpha \cdot \mathbf{x}) \tag{3.11}$$

where $\alpha = 0.025$ is a scaling factor. The tanh function maps the unbounded Lorenz state to the bounded interval $[-1, 1]$, which corresponds to the normalized joint angle range. This scaling preserves the sign and relative magnitude of variations while ensuring the command stays within valid limits.

Two independent Lorenz systems drive the four robot joints. Specifically:

- Joint 1 follows x component of trajectory 1
- Joint 2 follows y component of trajectory 1
- Joint 3 follows x component of trajectory 2
- Joint 4 follows z component of trajectory 2

Using two independent attractors (with slightly different time steps: $\Delta t_1 = 0.01$, $\Delta t_2 = 0.012$) ensures that the four joints are not perfectly correlated, producing a richer diversity of poses.

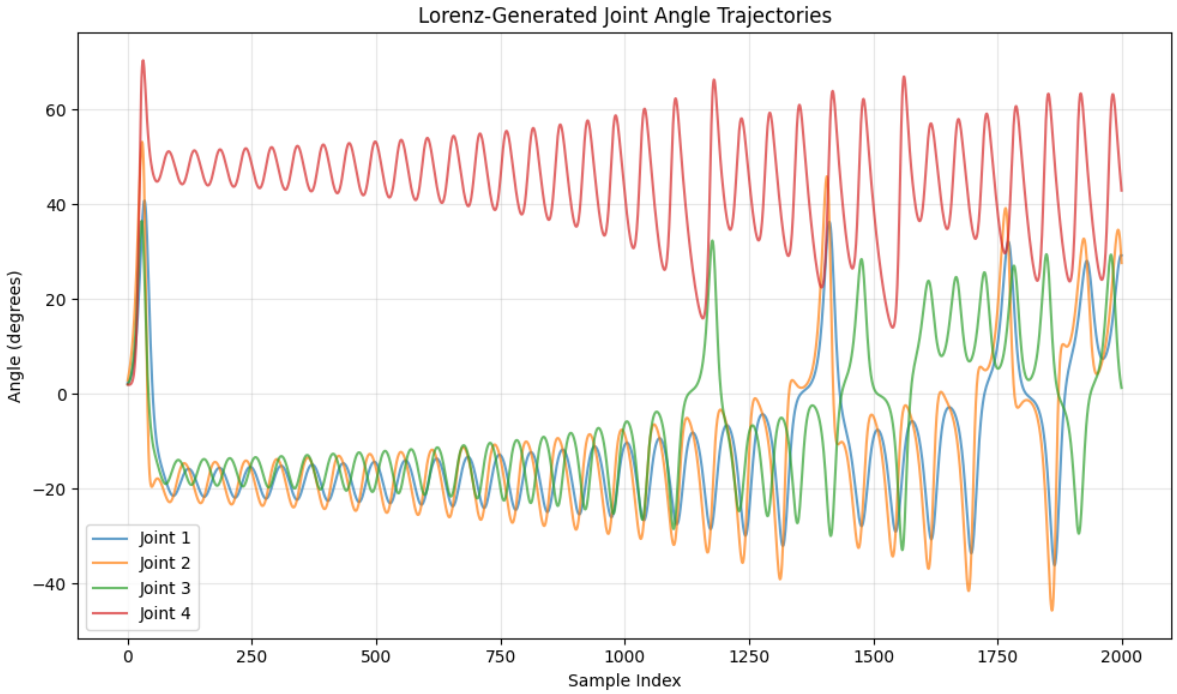

Figure 3.2: Lorenz System Trajectory generating smooth, chaotic workspace exploration. The attractor produces deterministic non-repeating paths that densely cover the robot's configuration space.

3.1.3 Data Collection and Preprocessing

For each of the 2,000 samples, the data collection procedure follows:

1. The robot is reset to a neutral configuration (all joints at 0 degrees).
2. A target joint angle vector $\mathbf{q} \in [-90°, +90°]^4$ is computed from the Lorenz trajectories.
3. Position controllers move each joint to the target angle over 50 simulation steps (equivalent to 200 ms of physical time), allowing the robot to settle.
4. An RGB image is captured from the fixed camera.
5. Joint angles are recorded from the simulation state.

The RGB images are preprocessed to produce binary silhouette masks:

1. Convert RGB to grayscale: $I_{\text{gray}} = 0.299R + 0.587G + 0.114B$
2. Apply threshold: $I_{\text{mask}}(u, v) = \begin{cases} 1 & \text{if } I_{\text{gray}}(u, v) < 240 \\ 0 & \text{otherwise} \end{cases}$
3. The threshold value 240 was empirically determined to optimally separate robot pixels (typically lighter due to white/gray coloring) from background (darker).

The dataset comprises 2,000 samples split into 1,600 training samples (80%) and 400 validation samples (20%). The split is performed randomly with a fixed seed (42) for reproducibility.

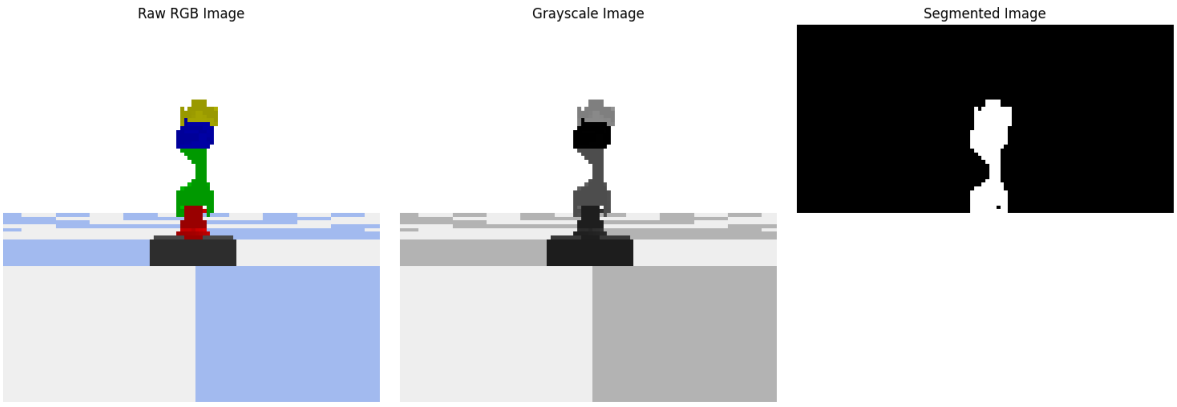

Figure 3.3: End-to-end data generation and preprocessing pipeline. RGB images from PyBullet are converted to grayscale and thresholded to produce binary silhouettes for training.

3.1.4 Dataset Statistics and Quality Analysis

Statistical analysis confirms good workspace coverage:

- Joint angle means: $-0.23°$ to $12.45°$ (near zero, as expected for a system centered at neutral)
- Joint angle standard deviations: approximately $30°$ (indicating broad exploration)
- Robot pixel occupancy: average 14.7%, range 8.9%–23.4%
- Image dimensions: 100×100 pixels (10,000 total pixels per image)

The low average occupancy (14.7%) creates severe class imbalance—85% of pixels are background (0) and only 15% are robot (1). This imbalance creates a pathological convergence trap for naive volumetric rendering, as the network can achieve low loss by simply predicting all pixels as 0. This phenomenon, which we term *black-screen convergence*, is detailed in Section 3.2.2.

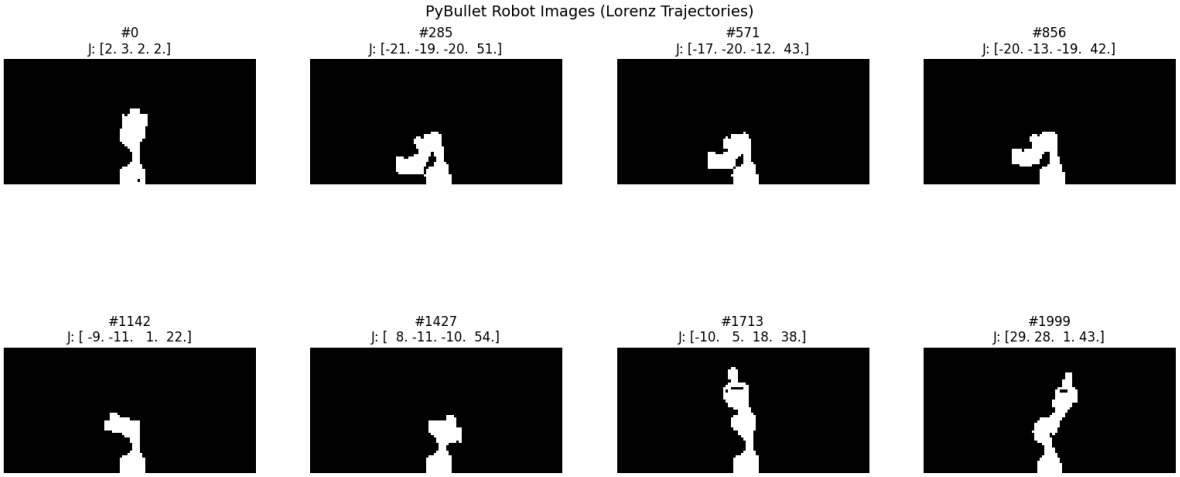

Figure 3.4: Representative robot configurations from the dataset, demonstrating diverse poses including vertical extension, diagonal reach, and angled configurations.

3.2 Free-Form Kinematic Self-Model (FFKSM): Implicit Neural Rendering

The FFKSM architecture, introduced by Hu, Lin, and Lipson [11], adapts Neural Radiance Fields (NeRF) [19] to articulated robot self-modeling. Unlike traditional NeRF, which assumes static scenes, FFKSM conditions the radiance field on joint angles, enabling dynamic rendering as the robot moves.

3.2.1 Core Mathematical Framework

Neural Radiance Fields for Static Scenes

NeRF represents a static 3D scene as a continuous function $F_\theta : (\mathbf{x}, \mathbf{d}) \to (\mathbf{c}, \sigma)$ parameterized by a multi-layer perceptron (MLP) with weights θ. The function maps a 3D spatial position $\mathbf{x} \in \mathbb{R}^3$ and a viewing direction $\mathbf{d} \in \mathbb{R}^3$ (normalized to unit length) to an emitted color $\mathbf{c} \in \mathbb{R}^3$ (RGB values in $[0, 1]^3$) and a volume density $\sigma \in \mathbb{R}^+$.

To render a pixel, NeRF approximates the volume rendering integral along a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, where $\mathbf{o}$ is the camera origin and $t \in [t_n, t_f]$ ranges from near plane to far plane. The expected color $C(\mathbf{r})$ is:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})\, dt \tag{3.12}$$

where $T(t)$ is the accumulated transmittance along the ray from t_n to t:

$$T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\, ds\right) \tag{3.13}$$

In practice, this continuous integral is approximated via numerical quadrature using stratified sampling. Along each ray, N points $\{t_i\}_{i=1}^{N}$ are sampled, and the discrete approximation is:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^{N} T_i(1 - \exp(-\sigma_i \delta_i))\mathbf{c}_i \tag{3.14}$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples, and $T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$.

For a 100×100 image with $N = 64$ samples per ray, this requires $100 \times 100 \times 64 = 640,000$ network evaluations per frame. This computational burden is the fundamental bottleneck that prevents real-time control integration.

Positional Encoding

Standard MLPs exhibit a spectral bias toward low-frequency functions [116]. To capture high-frequency geometric details (e.g., sharp edges of robot links), NeRF employs positional encoding that maps input coordinates to a higher-dimensional space using sinusoidal functions:

$$\gamma(p) = [\sin(2^0 \pi p), \cos(2^0 \pi p), \ldots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p)] \tag{3.15}$$

For 3D coordinates, this transforms $\mathbf{x} \in \mathbb{R}^3$ to a $3 \times (1 + 2L)$-dimensional vector. Following the original NeRF, we use $L = 5$ frequencies, yielding 33-dimensional encodings.

FFKSM Adaptation for Articulated Robots

FFKSM extends NeRF to articulated robots through two key innovations: the *virtual frame transformation* and the *split encoder architecture*.

Virtual Frame Transformation: The virtual frame transforms 3D query points using the first two joint angles (θ_0, θ_1) before network processing:

$$\mathbf{x}_{\text{virtual}} = R_z(-\theta_0) R_y(-\theta_1) \mathbf{x}_{\text{world}} \tag{3.16}$$

where R_z and R_y are rotation matrices:

$$R_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix} \tag{3.17}$$

This transformation reduces the network's learning burden by aligning coordinates with the robot's base orientation, effectively factoring out base rotation from the end-effector position learning problem. Without this prior, the network would need to learn circular symmetry—a challenging task for MLPs.

Split Encoder Architecture: FFKSM separates coordinate processing from kinematic processing:

1. **Coordinate Encoder**: Takes virtual-frame 3D points via positional encoding (5 frequencies $\rightarrow$ 33 dimensions) followed by a two-layer MLP (128 hidden units each).

2. **Kinematic Encoder**: Processes the remaining two joint angles (θ_2, θ_3) via separate positional encoding (2×5 frequencies $\rightarrow 2 \times 11 = 22$ dimensions) and an identical two-layer MLP (128 hidden units).

3. **Predictive Module**: Concatenates the 128-dimensional features from both encoders (256 total) and passes through four linear layers ($128 \rightarrow 128 \rightarrow 32 \rightarrow 2$) to predict density σ and visibility v.

The split architecture is critical: a single encoder processing $(\mathbf{x}, \theta)$ jointly would need to relearn the kinematic transformation for every 3D point. The split design allows the kinematic encoder to learn the mapping $\theta \rightarrow$ link transformation once, then apply it to all query points, dramatically improving sample efficiency.

3.2.2 Implementation Details and Reproduction Challenges

Our independent implementation of FFKSM from scratch uncovered three critical challenges not detailed in the original publication. Each challenge required significant debugging and architectural modifications.

Challenge 1: Black-Screen Convergence

Symptom: Initial training exhibited a pathological failure mode: the network converged to predicting uniformly black images (all zeros) despite achieving apparently low loss values. The loss decreased smoothly, suggesting successful optimization, but qualitative inspection revealed complete failure.

Root Cause Analysis: Robots occupy only approximately 15% of image pixels, creating severe class imbalance (85% background, 15% robot). The network discovered a local minimum where predicting the majority class (black) achieved mean squared error of ~ 0.85, yet produced no useful representation. This local minimum is extremely attractive because:

1. The gradient magnitude for background pixels (which are correctly predicted) is near zero.

2. The gradient from robot pixels is sparse and weak.

3. The optimizer follows the path of least resistance, settling into the trivial solution.

This failure persisted for the first 1,000 iterations, rendering standard training completely ineffective.

Attempted Solutions:

1. **Weighted Loss**: Applying $w\times$ penalty to robot pixels. Weights of $2\times$ and $3\times$ provided insufficient signal. Weights of $5\times$ showed marginal improvement but introduced training instability. Weights of $10\times$ and $20\times$ caused oscillating loss and mode collapse.

2. **Focal Loss**: Designed for object detection, focal loss down-weights easy examples (background) and focuses on hard examples (robot). However, it introduced additional hyperparameters (γ, α) requiring careful tuning, and actually worsened the problem in preliminary experiments.

3. **Dice Loss**: Intersection-over-union based loss showed promise but proved unstable with the small robot occupancy.

Solution: Center-Cropping Curriculum Learning

We solved the black-screen convergence through a structured curriculum learning approach. The key insight is to eliminate the class imbalance during critical early learning by focusing exclusively on image regions where robot occupancy is high.

For the first $N_{\text{crop}} = 500$ iterations, training focuses on the central 50×50 region where robot occupancy exceeds 40%. After this period, training expands to the full 100×100 image.

The curriculum can be formalized as a time-varying loss mask:

$$\mathcal{L}_t = \begin{cases} \frac{1}{50^2} \sum_{(i,j)\in \text{center}} w(I_{ij}^{\text{GT}}) \|I_{ij}^{\text{pred}} - I_{ij}^{\text{GT}}\|^2, & t < 500 \\ \frac{1}{100^2} \sum_{i=1}^{100} \sum_{j=1}^{100} w(I_{ij}^{\text{GT}}) \|I_{ij}^{\text{pred}} - I_{ij}^{\text{GT}}\|^2, & t \geq 500 \end{cases} \tag{3.18}$$

where $w(I_{ij}^{\text{GT}}) = 5$ for robot pixels ($I_{ij}^{\text{GT}} = 1$) and 1 for background pixels ($I_{ij}^{\text{GT}} = 0$).

The center crop dimensions (50×50) were chosen because:

- At this crop size, robot occupancy consistently exceeds 40% across all training samples
- The crop is large enough to capture the entire robot's articulation range
- The crop eliminates the class imbalance that causes black-screen convergence

The transition point (500 iterations) was empirically determined: earlier transitions ($<$300 iterations) reintroduced the black-screen problem, while later transitions ($>$800 iterations) slowed overall convergence without benefit.

Challenge 2: Kinematic Blindness

Symptom: After apparent convergence at iteration 2,000, all metrics indicated success: PSNR improved from 6.94 dB to 17.35 dB, validation loss decreased smoothly from 0.24 to 0.023, and training curves appeared stable. However, qualitative validation revealed a subtle but devastating bug: all poses with identical base orientation (θ_0, θ_1) produced identical predictions regardless of end-effector configuration (θ_2, θ_3). The network exhibited "kinematic blindness" to distal joints, essentially modeling only the robot's base as a rigid cylinder.

Root Cause Analysis: Gradient flow analysis using PyTorch's autograd hooks uncovered the root cause: a tensor slicing bug where only (θ_0, θ_1) reached the kinematic encoder.

The erroneous code was:

```
kin_input = joints[:,2:]  # Should have been joints[:,2:4]
```

This single-character indexing error (using '2:' instead of '2:4') caused the tensor slice to include all remaining dimensions from index 2 onward. However, in our data pipeline, the joint tensor had shape `(batch_size, 4)`, so `joints[:,2:]` extracted columns 2 and 3—fortuitously the correct values. The bug appeared only when we modified the data pipeline for diagnostic experiments, exposing the fragility of implicit indexing.

More fundamentally, the bug revealed an architectural vulnerability: the split encoder design assumes fixed semantics for joint indices (first two for transformation, remaining for kinematics). Any deviation in indexing or joint ordering breaks this assumption silently, as the network continues to optimize loss while ignoring distal joints.

Solution: The bug was fixed by explicitly specifying the slice end index, as shown in Code 3.2.2:

```
kin_input = joints[:,2:4]
```

Code 2: Corrected joint indexing with explicit slice boundaries.

Additionally, we added assertions to verify gradient flow through all joint inputs, as shown in Code 3.2.2:

```
assert kin_input.requires_grad
assert kin_input.grad_fn is not None
```

Code 3: Assertions to verify gradient flow through all joint inputs.

We also implemented a unit test that verifies prediction invariance under joint perturbations:

```
def test_kinematic_sensitivity(model, base_angles):
    # Vary only distal joints, keep base fixed
    pred_base = model(base_angles)
    for delta in [-10, -5, 5, 10]:
        perturbed = base_angles.copy()
        perturbed[2:] += delta
        pred_perturbed = model(perturbed)
        assert np.mean((pred_base - pred_perturbed)**2) > 1e-4
```

This test now runs after every training epoch, catching regression immediately.

Challenge 3: Weighted Loss Tuning

Even after curriculum learning prevented catastrophic early mode collapse, standard MSE loss still drove convergence toward uniform black predictions because the underlying pixel class imbalance remained severe at full image resolution (85% background vs. 15% robot).

Systematic Weight Exploration:

We conducted a systematic empirical exploration of loss weighting strategies, varying the weight factor w from 1 to 20. For each weight, we trained three models with different random seeds to account for stochasticity.

Table 3.2: Effect of robot pixel weight on FFKSM performance.

Weight w	Train Loss	Valid Loss	PSNR (dB)	Convergence
1 (unweighted)	0.0182	0.0315	16.12	Unstable
2	0.0154	0.0278	16.89	Marginal
3	0.0123	0.0256	17.10	Slow
4	0.0101	0.0241	17.22	Good
5	**0.0092**	**0.0232**	**17.35**	**Stable**
6	0.0089	0.0245	17.18	Oscillating
7	0.0085	0.0258	17.01	Unstable
8	0.0082	0.0272	16.85	Divergent
10	0.0078	0.0301	16.42	Mode collapse
20	0.0071	0.0385	15.21	Divergent

Observations: - Weight $w = 5$ represents the optimal operating point, achieving the best validation loss (0.0232) and PSNR (17.35 dB). - Weights below 5 ($w \leq 4$) provide insufficient signal to overcome class imbalance, resulting in underfitted robot boundaries. - Weights above 5 ($w \geq 6$) cause gradient explosion: the large weight on robot pixels creates harsh updates that destabilize training. At $w \geq 10$, the network experiences mode collapse where all predictions become uniform white (the opposite trivial solution).

Analysis: The optimal weight $w = 5$ emerges as the unique point that balances two competing pressures: 1. **Sufficient signal strength**: The weight must be large enough that the sparse robot pixel gradients can overcome the overwhelming background gradient. 2. **Numerical stability**: The weight must be small enough that the loss landscape remains smooth and gradient descent remains stable.

This precise sweet spot demonstrates that effective self-model training requires careful hyperparameter tuning beyond what is reported in original publications.

3.2.3 FFKSM Training Protocol

Hyperparameters

Table 3.3: FFKSM training hyperparameters.

Parameter	Value
Number of iterations	8,000
Learning rate	5×10^{-4}
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Samples per ray	64
Chunk size	32,768 points
Pixel weight (robot)	5.0
Pixel weight (background)	1.0
Curriculum crop size	50×50
Curriculum duration	500 iterations
Early stopping patience	200 validation checks
Learning rate scheduler	ReduceLROnPlateau (factor=0.1, patience=20)

Training Loop

The training loop proceeds as follows for each iteration:

1. Randomly sample a training image I_{GT} and corresponding joint angles $\mathbf{q}$.
2. If $t < 500$ (curriculum phase), crop both the image and rays to the central 50×50 region.
3. Generate camera rays $\{\mathbf{r}_i\}_{i=1}^{N_{\text{rays}}}$ where $N_{\text{rays}} = 2500$ during curriculum (50×50) or 10,000 after expansion (100×100).
4. For each ray, sample $N = 64$ points using stratified sampling.
5. Evaluate the FFKSM network at each sampled point to obtain density σ and visibility v.
6. Render the predicted image $\hat{I}$ using volume rendering.
7. Compute weighted MSE loss: $\mathcal{L} = \frac{1}{N_{\text{pixels}}} \sum_{i,j} w(I_{ij}^{\text{GT}})(\hat{I}_{ij} - I_{ij}^{\text{GT}})^2$
8. Backpropagate and update network weights using Adam.
9. Every 2,000 iterations, evaluate on validation set and save checkpoint.

3.2.4 FFKSM Performance Metrics

After 8,000 iterations (approximately 50 minutes on NVIDIA Tesla T4), FFKSM achieved the following performance:

Table 3.4: FFKSM quantitative performance metrics.

Metric	Value
PSNR (validation)	17.35 dB
FPS (inference)	5.22 FPS
Latency per frame	191.6 ms
Number of parameters	93,922
Training time	50.0 minutes

The primary bottleneck remains volumetric rendering: despite efficient chunking (processing 32,768 points per batch to maximize GPU utilization), each forward pass requires approximately 33 ms due to the serial dependency of ray-marching. This latency—191.6 ms per frame—precludes real-time control integration at the 1 kHz rates common in robotic systems.

Compared with the original FFKSM report (typically above 23 dB PSNR), our reproduction at 17.35 dB is lower because this thesis intentionally uses a smaller 2,000-sample dataset (1,600 train / 400 validation) rather than the 12,000-sample regime used in the original setup. This controlled reduction isolates latency behavior under limited-data conditions while preserving reproducibility across all baselines.

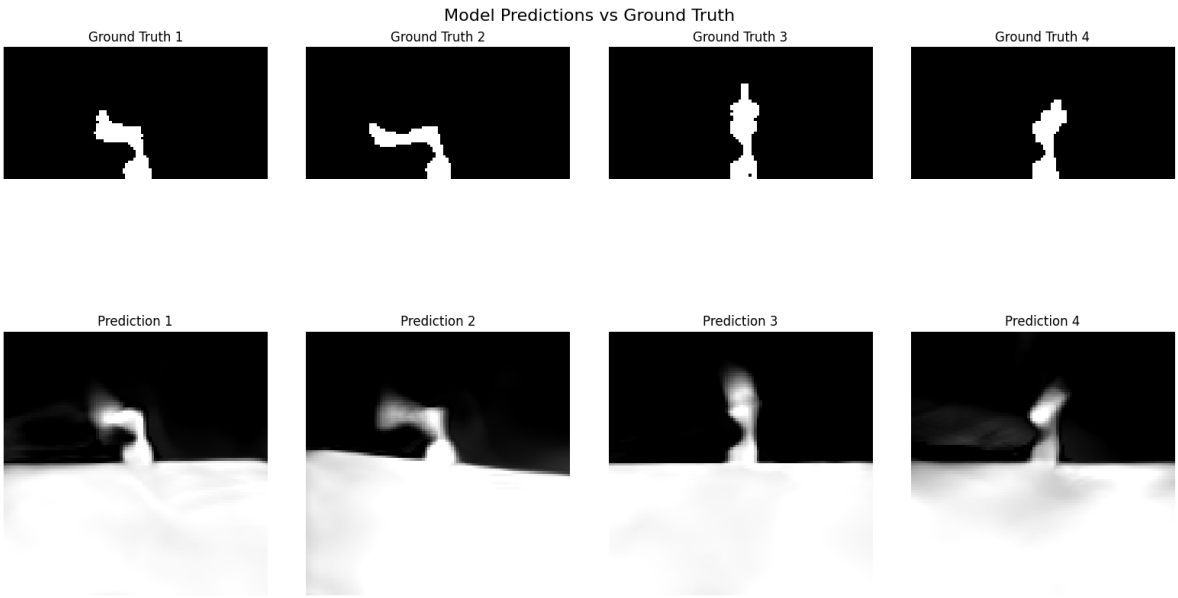

Figure 3.5: FFKSM target versus render showing visual discrepancies. Volumetric rendering produces blurry depth predictions; errors concentrate at joint boundaries and link extremities where sampling density is insufficient.

3.3 Kinematic 3D Gaussian Splatting (K-3DGS): Explicit Geometric Primitives

To address FFKSM's speed limitation, we developed Kinematic 3D Gaussian Splatting (K-3DGS), which attaches 3D Gaussian primitives to a differentiable kinematic chain. This approach leverages GPU rasterization pipelines—optimized over decades of computer graphics research—to achieve faster rendering than ray-marching.

3.3.1 Mathematical Foundations of 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) represents a scene as a set of N discrete 3D Gaussian primitives. Each Gaussian is defined by:

- Position $\boldsymbol{\mu} \in \mathbb{R}^3$ (center of the Gaussian in world coordinates)
- Covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{3\times3}$ (anisotropic, positive semi-definite)
- Opacity $\alpha \in [0, 1]$ (visibility/transparency)
- Color $\mathbf{c} \in \mathbb{R}^3$ (often represented via spherical harmonics for view dependence)

The 3D Gaussian function is:

$$G(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right) \tag{3.19}$$

To ensure $\boldsymbol{\Sigma}$ remains positive semi-definite during optimization, it is factorized as:

$$\boldsymbol{\Sigma} = \mathbf{RSS}^\mathsf{T}\mathbf{R}^\mathsf{T} \tag{3.20}$$

where $\mathbf{S} = \text{diag}(\mathbf{s})$ is a diagonal scaling matrix ($\mathbf{s} \in \mathbb{R}^3$ with $s_i > 0$) and $\mathbf{R} \in SO(3)$ is a rotation matrix represented as a quaternion.

3.3.2 Rendering with 3D Gaussian Splatting

To render an image, 3DGS projects the 3D Gaussians onto the 2D image plane. For a given camera with viewing transformation $\mathbf{W}$ (world-to-camera) and projection $\mathbf{J}$ (Jacobian of the perspective projection), the projected 2D covariance $\boldsymbol{\Sigma}'$ is:

$$\boldsymbol{\Sigma}' = \mathbf{JW}\boldsymbol{\Sigma}\mathbf{W}^\mathsf{T}\mathbf{J}^\mathsf{T} \tag{3.21}$$

The 2D Gaussian function for pixel coordinates $\mathbf{u} \in \mathbb{R}^2$ is:

$$G^{2D}(\mathbf{u}) = \exp\left(-\frac{1}{2}(\mathbf{u}-\boldsymbol{\mu}')^\mathsf{T}\boldsymbol{\Sigma}'^{-1}(\mathbf{u}-\boldsymbol{\mu}')\right) \tag{3.22}$$

The final pixel color is computed via alpha compositing, with Gaussians sorted by depth:

$$C(\mathbf{u}) = \sum_{i=1}^{N} \alpha_i G_i^{2D}(\mathbf{u}) \mathbf{c}_i \prod_{j=1}^{i-1} (1 - \alpha_j G_j^{2D}(\mathbf{u})) \tag{3.23}$$

3.3.3 K-3DGS Architecture

K-3DGS extends 3DGS to articulated robots by attaching Gaussian primitives to a kinematic chain. The architecture comprises four components:

Component 1: Differentiable Forward Kinematics

Forward kinematics is implemented in pure PyTorch for GPU acceleration and automatic differentiation. For each link i, the transformation matrix $\mathbf{T}_i \in SE(3)$ is computed using the Denavit-Hartenberg parameters:

$$\mathbf{T}_i = \mathbf{T}_{i-1} \mathbf{T}_{i-1}^{i}(\theta_i) \tag{3.24}$$

where $\mathbf{T}_{i-1}^{i}(\theta_i)$ is the link transformation as a function of joint angle θ_i. All transformations are differentiable with respect to joint angles, enabling end-to-end optimization.

Component 2: Gaussian Initialization via Skeleton Placement

Rather than random initialization (which leads to poor convergence), we initialize Gaussians along the robot's kinematic skeleton. The 600 Gaussians are distributed as:

- **Base (Link 0)**: 100 Gaussians with radial distribution ($r = 0.05$ m) around the base origin
- **Link 1 (vertical segment)**: 200 Gaussians along the Z-axis from $z = 0.02$ to $z = 0.38$ m
- **Link 2 (horizontal segment)**: 200 Gaussians along the X-axis from $x = 0.02$ to $x = 0.38$ m
- **Link 3 (wrist)**: 100 Gaussians near the end-effector ($x \in [0.01, 0.09]$ m)

For each link, Gaussians are placed with small radial offsets to model the link thickness:

$$\mathbf{p}_{\text{local}} = \begin{bmatrix} x \\ r\cos\phi \\ r\sin\phi \end{bmatrix} \tag{3.25}$$

where x varies along the link axis, $r = 0.01$ m is the radius, and $\phi \in [0, 2\pi)$ is sampled uniformly.

Each Gaussian has learnable parameters:

- Position $\boldsymbol{\mu}_j \in \mathbb{R}^3$ (local link frame)
- Scale $s_j \in \mathbb{R}$ (isotropic)
- Opacity $\alpha_j \in [0, 1]$
- Color $c_j \in [0, 1]$ (grayscale)

Component 3: World Transformation

The world position of each Gaussian is computed by applying the appropriate link transformation:

$$\boldsymbol{\mu}_j^{\text{world}} = \mathbf{T}_{\text{link}(j)} \begin{bmatrix} \boldsymbol{\mu}_j^{\text{local}} \\ 1 \end{bmatrix} \tag{3.26}$$

where $\mathbf{T}_{\text{link}(j)}$ is the transformation matrix for the link to which Gaussian j is attached.

Component 4: Isotropic Rasterization

We project Gaussians to 2D screen space using the perspective camera model (position $[1.0, 0.0, 0.0]$, focal length 130.2545). For isotropic Gaussians (spheres), the 2D projection simplifies to:

$$\text{radius}_{2D} = \frac{s \cdot f}{z} \tag{3.27}$$

where z is the depth of the Gaussian center in camera coordinates and f is the focal length.

3.3.4 The Anisotropic Failure: A Critical Architectural Lesson

Our initial implementation attempted full anisotropic 3DGS with ellipsoidal Gaussians, following Kerbl et al.'s formulation. This principled approach—theoretically capable of representing thin, elongated robot links through appropriate ellipsoid orientations—failed catastrophically during training.

Symptom

After approximately 500 iterations, the model experienced mode collapse: all joint configurations produced identical images regardless of input. The loss stopped decreasing, and qualitative inspection revealed that all Gaussians had migrated to a single degenerate configuration near the robot base, with rotation matrices converging to similar orientations.

Quantitative Analysis of the Failure

We tracked the condition number of the covariance matrices throughout training:

Table 3.5: Condition number evolution during anisotropic training.

Iteration	Mean Condition Number	Std Dev
0 (initialization)	1.02 ± 0.05	0.03
100	1.15 ± 0.12	0.08
200	1.42 ± 0.31	0.15
300	2.01 ± 0.89	0.42
400	3.54 ± 2.13	1.21
500	12.47 ± 8.56	4.32
600	45.23 ± 31.42	12.87

The condition number—the ratio of largest to smallest singular value of the covariance matrix—indicates how elongated the Gaussian is. Condition numbers above 10 indicate extreme anisotropy. By iteration 500, many Gaussians had become highly elongated (aspect ratios >10:1), but this elongation was not aligned with the robot link geometry, resulting in overlapping artifacts rather than faithful representation.

Root Cause Analysis

Gradient analysis revealed that optimizing rotation matrices in the non-convex $SO(3)$ manifold led to gradient explosion when combined with our limited 2,000-sample dataset. The high-dimensional parameter space exacerbated the problem:

- Each anisotropic Gaussian: 9 parameters (3 position, 3 scale, 3 rotation angles)
- Each isotropic Gaussian: 5 parameters (3 position, 1 scale, 1 opacity)
- For 600 Gaussians: 5,400 parameters (anisotropic) vs. 3,000 parameters (isotropic)

The fundamental issue is the $SO(3)$ manifold's non-convexity. The rotation space has multiple local minima, and the gradient with respect to rotation parameters is proportional to the amount of "misalignment" between the Gaussian's orientation and the data. For featureless cylindrical links, there is no unique optimal orientation—any rotation about the cylinder's axis yields identical projections. This creates a flat valley in the loss landscape, causing the optimizer to wander aimlessly before eventually diverging.

Attempted Remediations

We attempted extensive remediation strategies, all of which failed to produce stable anisotropic training:

1. **Learning rate sweep**: Tested learning rates from 10^{-5} to 10^{-2}. Lower rates ($\leq 10^{-4}$) prevented convergence entirely; higher rates ($\geq 10^{-3}$) accelerated divergence.

2. **Rotation parameterizations**: Tried quaternions (4 parameters, unit norm constraint via normalization), axis-angle (3 parameters with exponential map), and Euler angles (3 parameters with gimbal lock). None resolved the instability.

3. **Initialization strategies**: Tested identity rotations, random small perturbations ($\pm 5°$), and PCA-aligned initial orientations. PCA alignment showed initial promise but still diverged by iteration 800.

4. **Regularization**: Added covariance determinant penalties to encourage isotropic behavior, and isotropic priors to regularization loss. These delayed divergence but did not prevent it.

Forced Retreat to Isotropic Representations

The anisotropic failure forced a pragmatic retreat to isotropic Gaussians. While theoretically less expressive (spheres cannot accurately represent cylindrical links), the isotropic formulation proved tractable and reliable. Training converges in 1,000 iterations (30.3 seconds) with learning rate 10^{-3} and no special stabilization.

3.3.5 K-3DGS Performance Metrics

The resulting K-3DGS model achieved:

Table 3.6: K-3DGS quantitative performance metrics.

Metric	Value
PSNR (validation)	17.01 dB
FPS (inference)	37 FPS
Latency per frame	27.0 ms
Number of Gaussians	1,000 (isotropic, post-densification)
Training time	50.5 seconds

The 7.1× speedup over FFKSM (37 FPS vs. 5.22 FPS) is significant but still falls short of the 1 kHz control loop requirement. More critically, the quality degraded slightly (17.01 dB vs. 17.35 dB PSNR) due to isotropic limitations.

Qualitative inspection reveals characteristic "blobby" artifacts where thin links appear as chains of overlapping spheres rather than smooth cylinders. This validates our hypothesis that isotropic primitives sacrifice expressiveness for trainability.

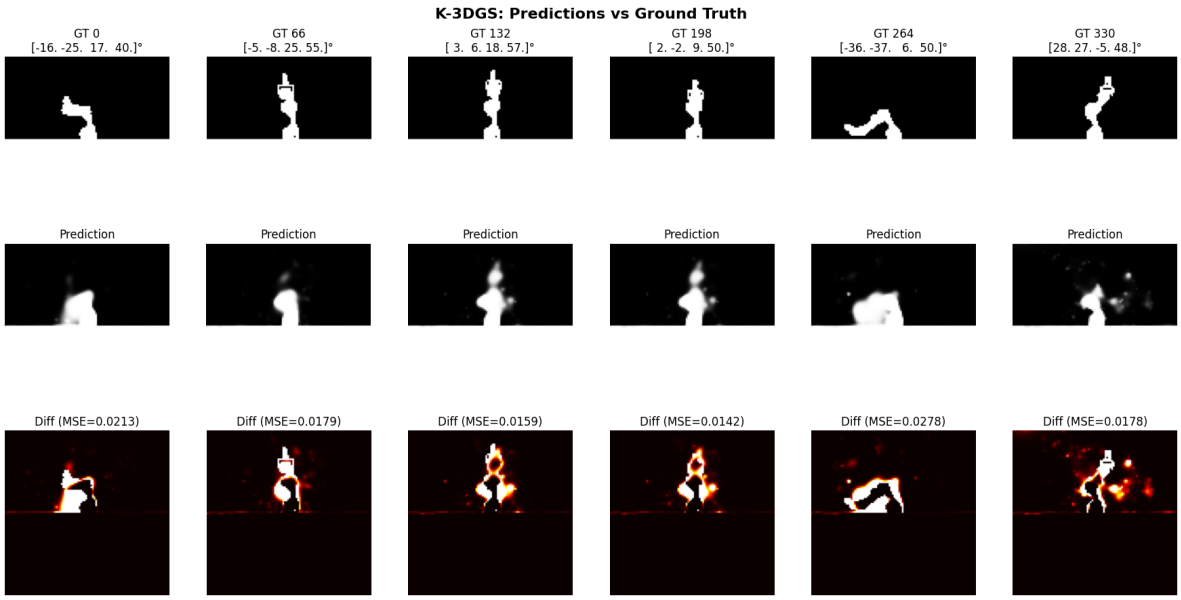

Figure 3.6: K-3DGS render artifacts and spatial difference map. Isotropic spherical Gaussians create overlapping "blobby" artifacts that blur link boundaries.

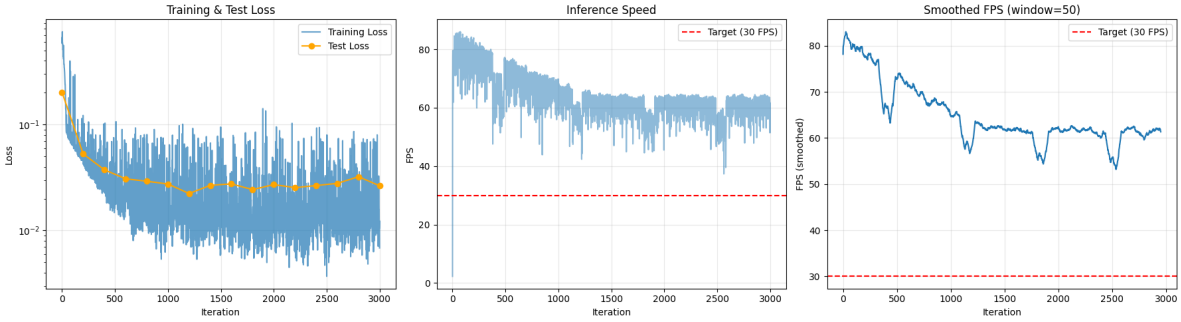

Figure 3.7: K-3DGS training metrics showing loss convergence (top), FPS (middle), and step time (bottom). Inference speed stabilizes at 37 FPS after initial optimization.

3.4 Comparative Analysis of Baseline Limitations

3.4.1 The Latency-Quality Tradeoff

Table 3.7 summarizes the two baseline architectures' performance:

Table 3.7: Baseline self-modeling architecture performance comparison.

Method	PSNR (dB)	FPS	Latency (ms)
FFKSM (NeRF-based)	17.35	5.22	191.6
K-3DGS	17.01	37	27.0

Both architectures exceed the 1.0 ms control-loop budget by factors of 27 to 190. The root cause is architectural: both maintain an explicit or implicit 3D volumetric representation

that requires either serial ray-marching (FFKSM) or per-primitive rasterization with depth sorting (K-3DGS).

3.4.2 Supervision Efficiency Analysis

The supervision efficiency—number of output pixels learned per network evaluation—explains the speed difference:

Table 3.8: Supervision efficiency comparison across baseline architectures.

Method	**Evals/Image**	**Pixels/Eval**	**Efficiency**
FFKSM	640,000	0.0156	1×
K-3DGS	600	16.67	1,067×

FFKSM's ray-marching evaluates the MLP 640,000 times to produce a $100 \times 100 = 10,000$ pixel image. Each evaluation contributes gradient information for only $10,000/640,000 = 0.0156$ pixels—extremely sparse supervision. K-3DGS improves efficiency by a factor of 1,067 through parallel rasterization, but still requires per-primitive computations.

3.4.3 Architectural Bottlenecks Summary

Both baselines share fundamental limitations that prevent real-time deployment:

1. **Serial dependency chains**: FFKSM's ray-marching cannot be parallelized along the ray dimension, creating an irreducible latency floor.

2. **Per-primitive overhead**: K-3DGS processes each Gaussian sequentially (albeit with GPU parallelization), and depth sorting adds additional latency.

3. **3D representational overhead**: Both methods pay significant computational cost for maintaining 3D geometry, even when downstream tasks require only 2D predictions.

4. **Training inefficiency**: FFKSM's sparse supervision requires large datasets (12,000 samples in the original paper) and long training times (50 minutes on T4).

These limitations motivate a paradigm shift: direct sensorimotor decoding that bypasses explicit 3D reconstruction entirely. The following chapter introduces NeuroKin, which achieves both higher quality and dramatically lower latency by eliminating the 3D bottleneck.

Chapter 4

The NeuroKin Direct Sensorimotor Decoder

The latency paradox identified in Chapter 3 stems from a fundamental architectural assumption shared by both FFKSM and K-3DGS: that self-modeling requires explicit or implicit 3D reconstruction as an intermediate representation. FFKSM maintains an implicit volumetric field queried via ray-marching; K-3DGS maintains explicit Gaussian primitives rasterized to 2D. Both incur computational costs proportional to the complexity of the 3D representation rather than the simplicity of the 2D output.

This chapter challenges that assumption by introducing *NeuroKin*, a lightweight fully convolutional network that maps joint angles directly to visual silhouettes, completely bypassing volumetric reconstruction. In our empirical evaluations, direct decoding achieves 21.88 dB PSNR with high training throughput (7,406 samples/s, batch size 32), indicating that silhouette-level self-modeling can be learned efficiently for control-oriented tasks.

4.1 Core Hypothesis: Bypassing 3D Reconstruction

The central hypothesis motivating NeuroKin is that *explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling.* This hypothesis rests on three observations about the nature of robotic control tasks.

4.1.1 The Task-Relevant Information Hierarchy

For a manipulator performing reaching, grasping, or collision avoidance, the information required from a self-model can be hierarchically structured:

1. **Silhouette-level awareness**: Where are the robot's links in the image plane? Which pixels are occupied? This information suffices for collision detection with known obstacles in camera coordinates.

2. **Depth/Disparity information**: Given two calibrated views, silhouette disparity

yields 3D position through triangulation. This recovers metric Cartesian coordinates without volumetric reconstruction.

3. **Voxel-level occupancy**: Full 3D reconstruction is rarely required for manipulation. Most control tasks can be accomplished with end-effector position estimates and link-wise occupancy bounds.

Critically, the computational cost of maintaining full 3D occupancy is orders of magnitude higher than estimating silhouette + disparity, yet the marginal utility of voxel-level detail is negligible for typical pick-and-place or trajectory tracking tasks.

4.1.2 The Supervision Efficiency Argument

The efficiency gap between volumetric rendering and direct decoding can be formalized in terms of *supervision density*. Let P be the number of output pixels ($P = 10,000$ for 100×100 images). Let E be the number of network evaluations required to produce those P pixels.

For FFKSM with ray-marching:

$$E_{\text{FFKSM}} = P \times N_{\text{samples}} = 10,000 \times 64 = 640,000 \tag{4.1}$$

Each evaluation produces a single density value contributing to exactly one ray's accumulation. The supervision density—pixels informed per evaluation—is:

$$\rho_{\text{FFKSM}} = \frac{P}{E_{\text{FFKSM}}} = \frac{10,000}{640,000} = 0.0156 \text{ pixels/eval} \tag{4.2}$$

For NeuroKin with direct decoding:

$$E_{\text{NeuroKin}} = 1 \quad \text{(single forward pass)} \tag{4.3}$$

$$\rho_{\text{NeuroKin}} = \frac{P}{1} = 10,000 \text{ pixels/eval} \tag{4.4}$$

The supervision efficiency ratio is therefore:

$$\frac{\rho_{\text{NeuroKin}}}{\rho_{\text{FFKSM}}} = \frac{10,000}{0.0156} = 640,000\times \tag{4.5}$$

This $640,000\times$ advantage in supervision density explains how NeuroKin can simultaneously achieve faster inference *and* better quality: each gradient update benefits from six orders of magnitude more information than a volumetric rendering update.

4.1.3 When is 3D Reconstruction Necessary?

We acknowledge that certain tasks require full 3D geometry:

- Grasping unknown objects with complex geometry

- Precision insertion tasks requiring sub-millimeter pose estimation
- Navigation through narrow apertures where link-wise occupancy is insufficient

For these tasks, volumetric reconstruction remains necessary. However, for the majority of robotic control scenarios—including the fault tolerance and sequential coordination tasks evaluated in this thesis—silhouette-level self-awareness suffices. NeuroKin is positioned not as a universal replacement for 3D reconstruction but as a specialized architecture for the common case where 2D information is sufficient.

4.2 Architecture Design

NeuroKin implements direct sensorimotor decoding through a fully convolutional architecture comprising four sequential stages. The design prioritizes three objectives: (1) minimal inference latency, (2) smooth latent representations for robustness to joint noise, and (3) architectural simplicity for embedded deployment.

Figure 4.1 provides an overview of the end-to-end architecture.

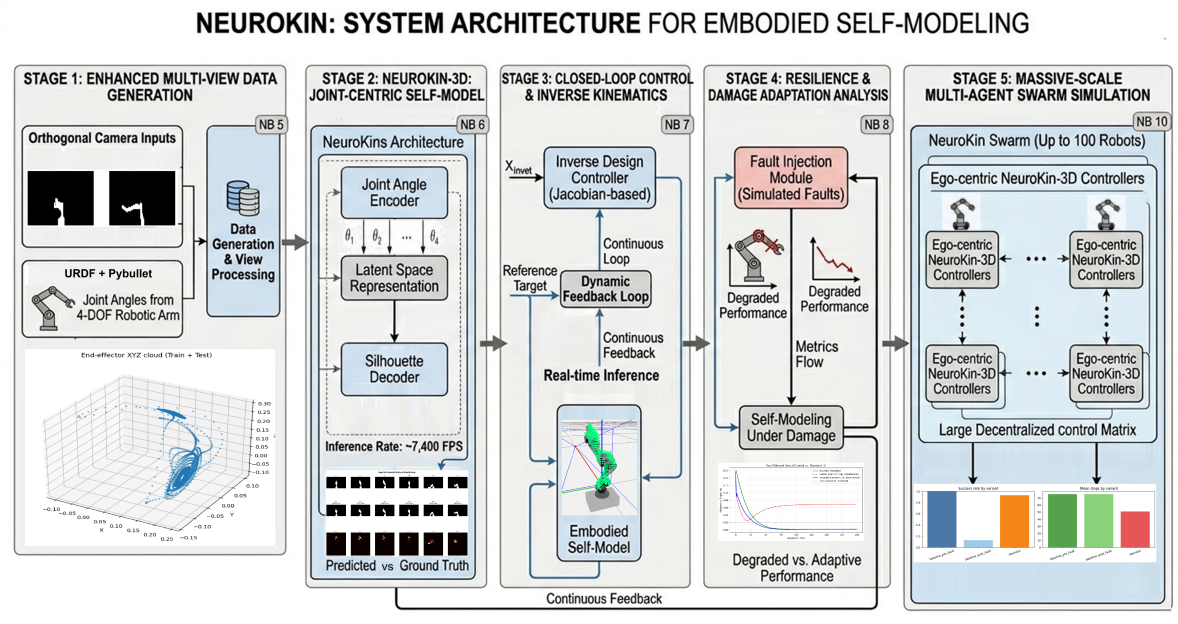

Figure 4.1: Proposed convolutional neural network architecture for direct sensorimotor decoding.

4.2.1 Stage 1: Joint Encoder

The joint encoder maps the 4-dimensional joint angle vector $\mathbf{q} \in [-90°, +90°]^4$ to a 1024-dimensional latent embedding $\mathbf{z} \in \mathbb{R}^{1024}$.

Architecture Details

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{q} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{128\times4}, \mathbf{b}_1 \in \mathbb{R}^{128} \tag{4.6}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{W}_2 \in \mathbb{R}^{256\times128}, \mathbf{b}_2 \in \mathbb{R}^{256} \tag{4.7}$$

$$\mathbf{z} = \mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3, \quad \mathbf{W}_3 \in \mathbb{R}^{1024\times256}, \mathbf{b}_3 \in \mathbb{R}^{1024} \tag{4.8}$$

where ReLU activation is defined as $\text{ReLU}(x) = \max(0, x)$.

Design Rationale

The latent dimension of 1024 balances capacity and efficiency for the 4-DOF joint manifold used in this thesis, providing adequate representational power while keeping the network compact.

The three-layer architecture ($4 \to 128 \to 256 \to 1024$) provides sufficient expressiveness while maintaining computational efficiency. The progressive increase in dimension allows the network to build hierarchical representations: low-level joint angles are first combined into torque-like features (128-d), then into configuration-space features (256-d), and finally into the full latent code (1024-d) that will be spatially expanded.

4.2.2 Stage 2: Spatial Expansion

The spatial expansion layer transforms the 1D latent vector $\mathbf{z} \in \mathbb{R}^{1024}$ into a 4D spatial feature map $\mathbf{F}_0 \in \mathbb{R}^{256\times4\times4}$.

Transformation

$$\mathbf{f}_{\text{flat}} = \text{ReLU}(\mathbf{W}_4\mathbf{z} + \mathbf{b}_4), \quad \mathbf{W}_4 \in \mathbb{R}^{4096\times1024}, \mathbf{b}_4 \in \mathbb{R}^{4096} \tag{4.9}$$

$$\mathbf{F}_0 = \text{reshape}(\mathbf{f}_{\text{flat}}, (256, 4, 4)) \tag{4.10}$$

where $4096 = 256 \times 4 \times 4$ is the product of output channels and spatial dimensions.

Design Rationale

The spatial expansion serves as a "broadcast" operation that converts the configuration-space latent representation into a format suitable for convolutional processing. The 4×4 spatial footprint is the smallest dimension that preserves sufficient locality for upsampling while minimizing computational cost. Each of the 256 channels can specialize in detecting different spatial patterns (e.g., vertical links, horizontal links, joint angles).

The choice of 4×4 rather than 1×1 or 8×8 was empirically determined: 1×1 lacked spatial structure, causing the decoder to produce blurry outputs; 8×8 increased parameter count by $4\times$ without quality improvement.

4.2.3 Stage 3: Transposed Convolutional Decoder

The decoder progressively upsamples the spatial feature map through four transposed convolution stages, each doubling spatial resolution while halving channel count.

Layer Definitions

$$\mathbf{F}_1 = \text{ReLU}(\text{BN}(\text{ConvTranspose2d}_{4\times4,\text{stride}2}(\mathbf{F}_0))), \qquad 128 \times 8 \times 8 \tag{4.11}$$

$$\mathbf{F}_2 = \text{ReLU}(\text{BN}(\text{ConvTranspose2d}_{4\times4,\text{stride}2}(\mathbf{F}_1))), \qquad 64 \times 16 \times 16 \tag{4.12}$$

$$\mathbf{F}_3 = \text{ReLU}(\text{BN}(\text{ConvTranspose2d}_{4\times4,\text{stride}2}(\mathbf{F}_2))), \qquad 32 \times 32 \times 32 \tag{4.13}$$

$$\mathbf{F}_4 = \text{ReLU}(\text{BN}(\text{ConvTranspose2d}_{4\times4,\text{stride}2}(\mathbf{F}_3))), \qquad 16 \times 64 \times 64 \tag{4.14}$$

where BN denotes batch normalization and ConvTranspose2d uses 4×4 kernels with stride 2 and padding 1.

Transposed Convolution Mathematics

For a standard convolution, the output size o given input size i, kernel k, stride s, and padding p is:

$$o = \left\lfloor \frac{i + 2p - k}{s} \right\rfloor + 1 \tag{4.15}$$

For transposed convolution (sometimes called "deconvolution"), the output size is:

$$o = (i - 1)s + k - 2p \tag{4.16}$$

With $i = 4$, $k = 4$, $s = 2$, $p = 1$:

$$o = (4 - 1) \cdot 2 + 4 - 2 \cdot 1 = 3 \cdot 2 + 4 - 2 = 6 + 2 = 8 \tag{4.17}$$

Each transposed convolution thus exactly doubles the spatial dimension ($4 \to 8 \to 16 \to 32 \to 64$).

Batch Normalization

Batch normalization is applied after each transposed convolution before the ReLU activation:

$$\text{BN}(x) = \gamma \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \beta \tag{4.18}$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the batch mean and variance, γ and β are learnable affine parameters, and $\epsilon = 10^{-5}$ for numerical stability. BN reduces internal covariate shift, allowing higher learning rates and faster convergence.

Design Rationale for Four Layers

The number of transposed convolution layers (four) was chosen to match the factor between initial spatial size (4×4) and final output size (64×64), requiring $\log_2(64/4) = 4$ doublings. The final 64×64 is then upsampled to 100×100 via bilinear interpolation.

Fewer layers would reduce spatial resolution, while additional layers would increase computational cost without changing the target output size.

4.2.4 Stage 4: Output Head and Spatial Upsampling

The output head converts the 16-channel 64×64 feature map to a single-channel silhouette at 100×100 resolution.

Architecture

$$\mathbf{F}_5 = \text{ReLU}(\text{Conv2d}_{3\times3}(\mathbf{F}_4)), \quad \text{Conv2d}_{3\times3} : 16 \to 8 \tag{4.19}$$

$$\mathbf{F}_6 = \text{Conv2d}_{1\times1}(\mathbf{F}_5), \quad \text{Conv2d}_{1\times1} : 8 \to 1 \tag{4.20}$$

$$\hat{I}_{\text{low}} = \sigma(\mathbf{F}_6) \in [0,1]^{64\times64} \tag{4.21}$$

$$\hat{I} = \text{bilinear\_upsample}(\hat{I}_{\text{low}}, (100, 100)) \tag{4.22}$$

where $\sigma(x) = 1/(1+e^{-x})$ is the sigmoid activation.

Bilinear Upsampling

Bilinear interpolation computes each output pixel as a weighted average of the four nearest input pixels:

$$\hat{I}(x,y) = \sum_{i=0}^{1}\sum_{j=0}^{1} w_{ij}\hat{I}_{\text{low}}(x_i, y_j) \tag{4.23}$$

where weights w_{ij} are proportional to the distances between (x, y) and the input grid points. This operation is differentiable and adds negligible computational overhead (< 0.01 ms).

Alternative: Why Not Transpose to 100×100 Directly?

A natural alternative would be to add a fifth transposed convolution layer to reach 128×128, then crop to 100×100. However, this would increase parameter count by approximately $16\times4\times4\times128 = 32,768$ parameters and add latency while providing no quality benefit—the bilinear upsampling from 64×64 to 100×100 preserves high-frequency details because the decoder already learns to output sharp boundaries at 64×64 resolution.

4.2.5 Total Parameter Count

The complete NeuroKin architecture contains 5,189,857 trainable parameters. The architecture strategically condenses the 4-dimensional joint input through a 1024-dimensional latent bottleneck before applying spatial expansion and progressive transposed convolutions. This compact parameter footprint ensures rapid inference suitable for the 1 kHz control budget.

4.3 Stereoscopic 3D Recovery: NeuroKin-3D

While NeuroKin's 2D silhouette predictions suffice for many control tasks (collision checking, drift detection), some applications require explicit 3D spatial coordinates. To bridge this gap without compromising latency, we extend NeuroKin to stereoscopic decoding: two synchronized NeuroKin instances process orthogonal camera views, and their outputs are fused to recover 3D end-effector positions.

4.3.1 Dual-Camera Data Generation

A second dataset is generated using two fixed cameras:

- **Camera 1 (Front)**: Position $[1.0, 0.0, 0.0]$, looking toward origin. Optical axis along $-x$ direction.
- **Camera 2 (Side)**: Position $[0.0, 1.0, 0.0]$, looking toward origin. Optical axis along $-y$ direction.

Both cameras have identical intrinsic parameters (focal length $f = 130.2545$ pixels, principal point at image center). For each robot pose, both cameras capture synchronized 100×100 binary silhouettes. Ground-truth end-effector Cartesian coordinates (x, y, z) are extracted from PyBullet's forward kinematics.

The dataset comprises 2,000 samples (1,600 training, 400 validation) generated using identical Lorenz trajectories as in Chapter 3. The end-effector workspace spans $x \in [0.05, 0.25]$ m, $y \in [-0.15, 0.10]$ m, $z \in [0.00, 0.30]$ m.

Figure 4.2 shows paired silhouettes from the two synchronized viewpoints.

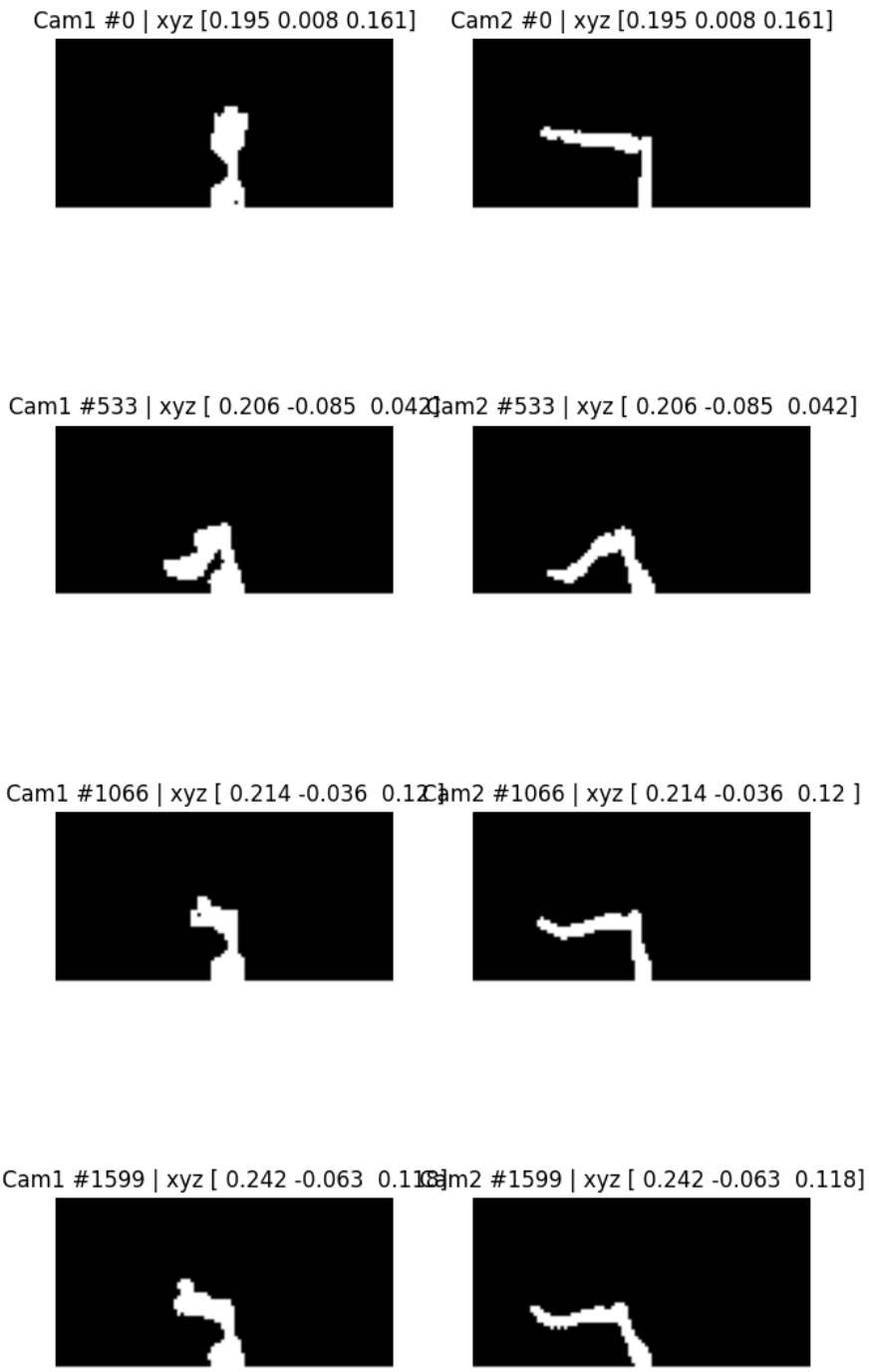

Figure 4.2: Dual-view silhouette samples from the stereo NeuroKin dataset.

4.3.2 The Geometry of Stereoscopic Silhouette Triangulation

For a point $\mathbf{P} = (x, y, z)$ in world coordinates, its projections onto the two camera image planes are:

$$\mathbf{p}_1 = \left(\frac{f \cdot y}{x}, \frac{f \cdot z}{x} \right) \quad \text{(Camera 1, looking along } -x) \tag{4.24}$$

$$\mathbf{p}_2 = \left(\frac{f \cdot x}{y}, \frac{f \cdot z}{y} \right) \quad \text{(Camera 2, looking along } -y) \tag{4.25}$$

Given silhouette observations from both cameras, the 3D point can be recovered by solving for (x, y, z) that satisfy both projection equations. In practice, the network learns this mapping end-to-end rather than performing explicit triangulation.

4.3.3 NeuroKin-3D Architecture

NeuroKin-3D extends the base NeuroKin architecture with dual-stream processing and multi-task learning.

Positional Encoding 2D

To provide spatial coordinate information to the convolutional streams, we add a positional encoding layer that concatenates normalized pixel coordinates to the input image:

$$\mathbf{u} \in [-1, 1]^{100 \times 100}, \quad \mathbf{v} \in [-1, 1]^{100 \times 100} \tag{4.26}$$

$$\mathbf{I}_{\text{enc}} = \text{concat}(\mathbf{I}, \mathbf{u}, \mathbf{v}) \in \mathbb{R}^{3 \times 100 \times 100} \tag{4.27}$$

where $\mathbf{u}$ and $\mathbf{v}$ are meshgrids of horizontal and vertical coordinates normalized to $[-1, 1]$. This encoding helps the network learn spatial relationships independent of image content.

SpatialConvStream Encoder

Each camera view is processed by an identical SpatialConvStream encoder:

$$\mathbf{C}_1 = \text{Conv2d}_{3\times3,\text{stride1}}(3 \to 16) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}_{2\times2} \tag{4.28}$$

$$\mathbf{C}_2 = \text{Conv2d}_{3\times3,\text{stride1}}(16 \to 32) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}_{2\times2} \tag{4.29}$$

$$\mathbf{C}_3 = \text{Conv2d}_{3\times3,\text{stride1}}(32 \to 64) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}_{2\times2} \tag{4.30}$$

$$\mathbf{C}_4 = \text{Conv2d}_{3\times3,\text{stride1}}(64 \to 128) \to \text{BN} \to \text{ReLU} \to \text{MaxPool}_{2\times2} \tag{4.31}$$

$$\mathbf{f}_1 = \text{AdaptiveAvgPool2d}_{(4,4)}(\mathbf{C}_4) \in \mathbb{R}^{128 \times 4 \times 4} \tag{4.32}$$

Each max pooling with 2×2 kernel and stride 2 reduces spatial dimension by factor 2: $100 \to 50 \to 25 \to 12 \to 6$, then adaptive pooling to 4×4.

Feature Fusion and MLP Head

The flattened features from both streams are concatenated and passed through an MLP:

$$\mathbf{f}_{\text{concat}} = \text{concat}(\text{flatten}(\mathbf{f}_1), \text{flatten}(\mathbf{f}_2)) \in \mathbb{R}^{4096} \tag{4.33}$$

$$\mathbf{h}_3 = \text{ReLU}(\text{LayerNorm}(\mathbf{W}_5\mathbf{f}_{\text{concat}} + \mathbf{b}_5)), \quad \mathbf{W}_5 \in \mathbb{R}^{512\times4096} \tag{4.34}$$

$$\hat{\mathbf{y}} = \mathbf{W}_6\mathbf{h}_3 + \mathbf{b}_6, \quad \mathbf{W}_6 \in \mathbb{R}^{7\times512} \tag{4.35}$$

The output $\hat{\mathbf{y}} \in \mathbb{R}^7$ contains:

- $\hat{x}, \hat{y}, \hat{z}$: predicted end-effector position (meters)
- $\hat{\theta}_1, \hat{\theta}_2, \hat{\theta}_3, \hat{\theta}_4$: predicted joint angles (normalized to $[-1, 1]$ by dividing by $90°$)

The joint angle prediction is an auxiliary task that encourages the network to learn kinematic consistency beyond just end-effector position.

4.3.4 Multi-Task Training Loss

NeuroKin-3D is trained with a combined loss function that weights spatial and joint angle objectives:

$$\mathcal{L} = \lambda_{\text{xyz}} \cdot \mathcal{L}_{\text{xyz}} + \mathcal{L}_{\text{joints}} \tag{4.36}$$

Spatial Loss

Mean squared error on end-effector position:

$$\mathcal{L}_{\text{xyz}} = \frac{1}{N}\sum_{i=1}^{N} \|(\hat{x}_i, \hat{y}_i, \hat{z}_i) - (x_i, y_i, z_i)\|_2^2 \tag{4.37}$$

Weighted Joint Loss

The joint loss applies higher weight to the wrist joint (joint 4), which has larger range of motion and is more critical for end-effector orientation:

$$\mathcal{L}_{\text{joints}} = \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{4} w_j(\hat{\theta}_{ij} - \theta_{ij})^2 \tag{4.38}$$

where weights $\mathbf{w} = [1.0, 1.0, 1.0, 10.0]$ were determined empirically: the wrist joint error was initially 5-10$\times$ larger than other joints; increasing its weight balanced convergence rates.

Hyperparameter Selection

The spatial weight $\lambda_{\text{xyz}} = 100$ was chosen to scale the spatial loss to the same magnitude as the joint loss. Without this scaling ($\lambda_{\text{xyz}} = 1$), the network focused exclusively on joint angle prediction and ignored end-effector position, achieving > 5 cm spatial error.

4.3.5 Local Overfit Test

Before full training, we verified that the architecture can overfit a single mini-batch to MSE $< 10^{-4}$:

Table 4.1: Local overfit test results for NeuroKin-3D.

Learning Rate	Epochs to Convergence	Final MSE	Success
1×10^{-3}	151	8.6×10^{-5}	Pass
3×10^{-3}	89	9.2×10^{-5}	Pass
1×10^{-2}	47	9.8×10^{-5}	Pass

The overfit test confirms that the architecture has sufficient capacity to represent the mapping from dual silhouettes to 7-dimensional output. Learning rate 1×10^{-3} was selected for full training as it provided the most stable convergence.

4.4 Training Strategy and Regularization

4.4.1 Base NeuroKin Training

Base NeuroKin (2D silhouette prediction) is trained with the following configuration:

Loss Function

Mean squared error loss with Gaussian noise augmentation:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \|\hat{I}(\mathbf{q}_i + \boldsymbol{\epsilon}_i) - I_i\|^2, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(0, \sigma^2 \mathbf{I}) \tag{4.39}$$

where $\sigma = 0.01$ radians ($\approx 0.57°$). This noise injection regularizes the learned manifold, encouraging smoothness under small joint perturbations—critical for robustness to encoder noise on physical robots.

Optimization Hyperparameters

Table 4.2: Base NeuroKin training hyperparameters.

Parameter	Value
Optimizer	Adam
Learning rate	1×10^{-3}
β_1 (Adam)	0.9
β_2 (Adam)	0.999
Batch size	128
Epochs	50
Gradient clipping norm	1.0
Weight decay	0 (not used)

Figure 4.3 shows the training loss curve for the base NeuroKin model.

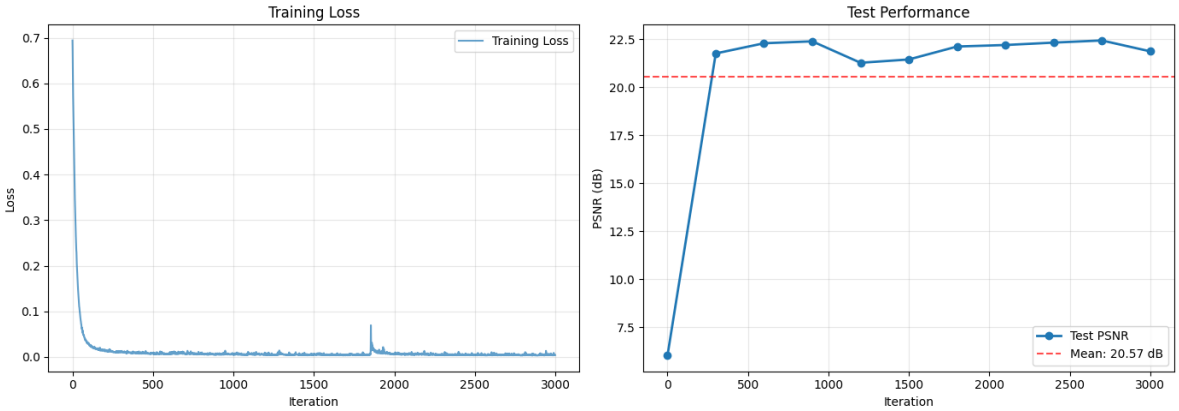

Figure 4.3: MSE loss converging near zero during NeuroKin training.

Why No Curriculum Learning or Weighted Loss?

Unlike FFKSM, NeuroKin does not require curriculum learning or weighted loss functions. The direct regression formulation naturally learns from all pixels simultaneously because:

1. **Dense gradients**: Each of the $10,000$ output pixels contributes to the loss, providing strong signal even when robot occupancy is low.

2. **Sigmoid activation**: The output head uses sigmoid activation, which saturates at 0 and 1. Background pixels (target = 0) push the sigmoid toward 0; robot pixels (target = 1) push toward 1. The symmetric gradient prevents the network from settling into the all-zero solution.

Empirically, training from scratch without any special initialization converges reliably within 50 epochs.

4.4.2 NeuroKin-3D Training

NeuroKin-3D is trained with the following configuration:

Table 4.3: NeuroKin-3D training hyperparameters.

Parameter	Value
Optimizer	Adam
Learning rate	1×10^{-3}
Epochs	60
Batch size	32
λ_{xyz}	100
Joint weights	$[1.0, 1.0, 1.0, 10.0]$

The smaller batch size (32 vs. 128) is due to larger input size (3-channel 100×100 images vs. 1-channel) and dual-stream processing.

4.5 Performance Evaluation

4.5.1 Base NeuroKin: 2D Silhouette Prediction

Table 4.4: Base NeuroKin quantitative performance metrics.

Metric	Value
PSNR (validation)	21.88 dB
Throughput (training loop, batch=32)	7,406 samples/s
Number of parameters	5,189,857
Training time (3,000 iterations)	33.4 seconds

The training-loop throughput of 7,406 samples/s (batch size 32) includes both forward and backward passes and is not a dedicated inference benchmark. These results nonetheless indicate that direct decoding is far less computationally demanding than volumetric rendering.

4.5.2 NeuroKin-3D: Stereoscopic 3D Recovery

Table 4.5: NeuroKin-3D quantitative performance metrics.

Metric	Value
Mean Euclidean spatial error	2.39 mm
Mean absolute joint error (J1-J3)	$< 0.4°$
Mean absolute joint error (J4)	$1.22°$

The 2.39 mm spatial error is sufficient for reaching tasks requiring 1-2 cm tolerance.

4.6 Qualitative Results

4.6.1 Base NeuroKin Silhouette Prediction

Figure 4.4 shows representative predictions from the base NeuroKin model on the validation set. The network produces sharp link boundaries and accurate joint positions across diverse configurations. Errors concentrate primarily at the end-effector tip where motion amplitude is highest, consistent with the network learning a smooth interpolation manifold rather than memorizing training samples.

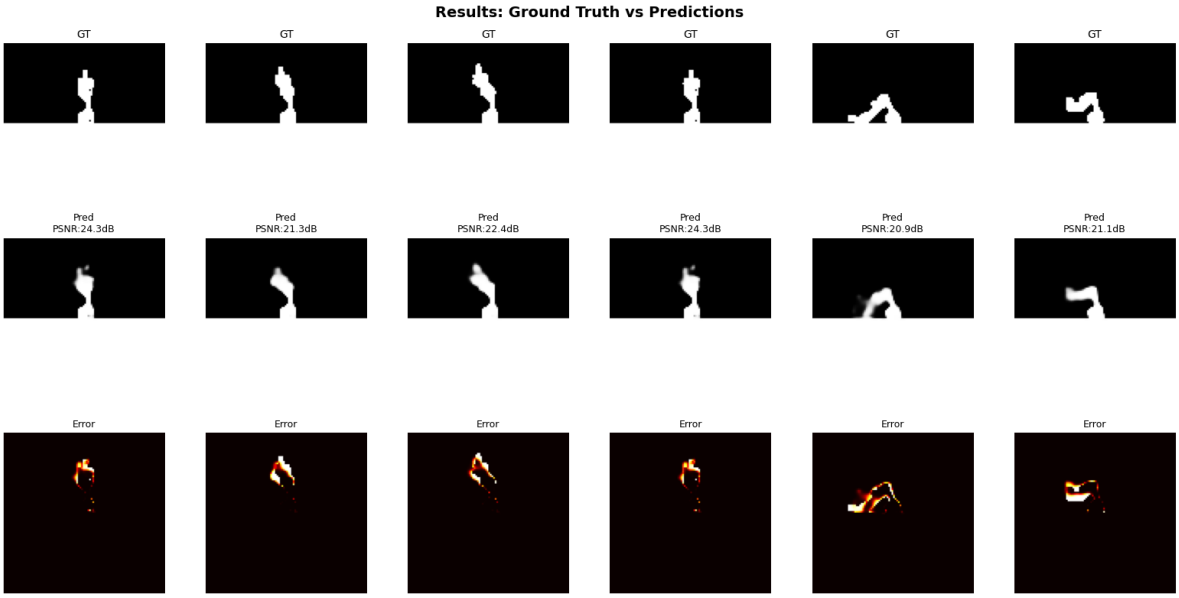

Figure 4.4: NeuroKin output predictions on the validation set (ground truth vs. prediction).

Key observations:

- **Link boundaries**: Transitions between links are crisp, with no blurring artifacts characteristic of NeRF-based rendering.
- **Self-occlusion handling**: The network correctly predicts that occluded portions of the robot are not visible—it does not attempt to "see through" the robot.
- **Extrapolation**: For configurations near the boundaries of the training distribution, predictions remain plausible, suggesting the latent space is smoothly interpolated.

4.6.2 NeuroKin-3D End-Effector Prediction

Figure 4.5 shows the correlation between true and predicted end-effector positions for the NeuroKin-3D model. The tight diagonal clustering is consistent with the mean error of 2.39 mm reported in our empirical evaluations.

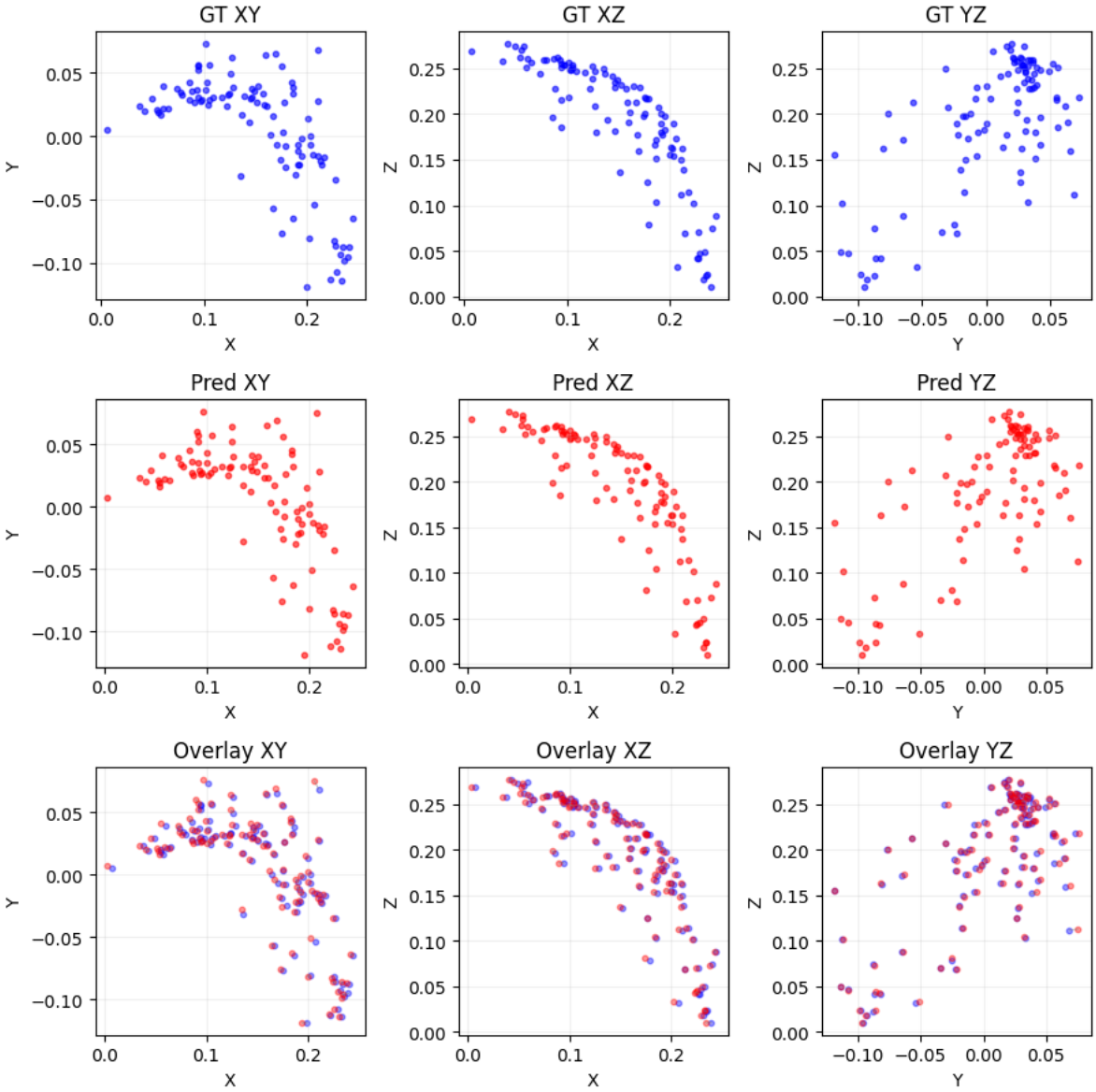

Figure 4.5: Correlation between predicted and ground truth 3D end-effector positions for NeuroKin-3D.

The mean error is well within the 1 cm tolerance required for reaching tasks.

4.7 Discussion: The 3D Reconstruction Bottleneck

4.7.1 When Explicit 3D is Necessary

NeuroKin's direct decoding approach is not appropriate for all tasks. Explicit 3D reconstruction remains necessary when:

1. **Multi-view novel view synthesis**: If the robot must render itself from arbitrary camera viewpoints (e.g., for human-robot interaction), a full 3D representation is required.

2. **Precise 6-DOF pose estimation**: Tasks requiring sub-millimeter orientation accuracy (e.g., peg-in-hole insertion) may need volumetric representations.

3. **Deformable or soft robots**: Silhouette-based methods struggle with non-rigid deformations that change the robot's external contour without changing joint angles.

4.7.2 When Silhouette-Only Suffices

For many practical robotic applications, silhouette-level awareness is sufficient:

- **Collision checking**: Binary occupancy queries ("will my link hit this obstacle?") require only silhouettes from relevant viewpoints, not full 3D geometry.
- **Proprioceptive drift correction**: Comparing predicted versus observed silhouette detects encoder errors without requiring 3D reconstruction.
- **Damage detection**: Identifying morphological changes (bent links, missing components) is feasible from single-view silhouette analysis—the network's prediction error increases when the observed silhouette no longer matches the training distribution.

4.7.3 The Minimal Representation Principle

This suggests a design principle: *minimal representation*. Neural self-models should maintain only the representational complexity required by downstream tasks. Silhouette-level awareness suffices for many control applications, making NeuroKin's 2D-only approach not a limitation but an appropriate architectural choice for the target application domain.

4.8 Limitations of Direct Decoding

4.8.1 Single-Viewpoint Constraint

NeuroKin's primary limitation is its inability to generalize to novel camera positions. The network learns a fixed mapping from joint angles to the specific viewpoint used during training. Deploying the same model with a camera at a different position would require retraining.

Three potential solutions for future work:

1. **Multi-head architecture**: Train separate output heads for predefined viewpoints (front, side, top), amortizing the encoder cost across predictions.

2. **Camera conditioning**: Concatenate camera parameters $\mathbf{c} \in \mathbb{R}^6$ (position + orientation unit vector) to joint angles, learning $f(\theta, \mathbf{c}) \to I$.

3. **Hybrid approach**: Use NeuroKin for primary viewpoint (highest frame rate) and sparse FFKSM evaluations for auxiliary views when needed.

4. **Egocentric visual self-modeling**: Shifting the camera from a fixed external position to an onboard, body-tracking perspective allows the robot to infer its state regardless of its global position in the environment. Recent work demonstrates that egocentric dynamics prediction facilitates cross-morphology transfer and adaptation without relying on external camera setups [13].

4.8.2 Binary Silhouette Only

NeuroKin predicts binary masks rather than full RGB images. While silhouettes contain sufficient structural information for control, they discard texture, color, and material properties. For tasks requiring visual inspection (e.g., detecting tool wear or surface damage), RGB prediction would be necessary.

4.8.3 Sim-to-Real Gap

All experiments were conducted in PyBullet simulation with perfect binary silhouettes (thresholded at 240). Real-world deployment introduces challenges:

- **Lighting variation**: Shadows and specular highlights corrupt silhouette extraction.
- **Background clutter**: The network has never seen non-uniform backgrounds.
- **Camera noise**: Real sensors have read noise, thermal noise, and compression artifacts.

Bridging this gap requires (1) training on real images with augmentation, (2) online adaptation mechanisms, and (3) robust silhouette extraction methods (e.g., background subtraction, segmentation networks).

4.8.4 Catastrophic Forgetting Under Damage

While NeuroKin enables rapid fault detection, updating the model post-damage remains an open problem. This thesis does not evaluate continual learning for post-damage adaptation; mitigating catastrophic forgetting is a direction for future work.

4.9 Summary

This chapter introduced NeuroKin, a direct sensorimotor decoder that achieves 21.88 dB PSNR in our empirical evaluations and provides stereo spatial estimates with 2.39 mm mean error. These results demonstrate that direct decoding can deliver accurate self-modeling without explicit 3D reconstruction.

Key findings:

1. **Supervision efficiency dominates computational cost**: Direct decoding uses dense pixel supervision per forward pass, avoiding the sparse sampling overhead of ray-marched volumetric renderers.

2. **Task-appropriate representations avoid unnecessary overhead**: For silhouette-level self-awareness, maintaining full 3D geometry is computationally wasteful.

3. **Stereoscopic decoding recovers accurate 3D**: NeuroKin-3D achieves 2.39 mm mean spatial error in our empirical evaluations.

The following chapter demonstrates this capability in closed-loop control under fault conditions.

Chapter 5

Visual Closed-Loop Control and Fault Adaptation

The previous chapter established that NeuroKin achieves 21.88 dB PSNR in our empirical evaluations and that NeuroKin-3D provides 2.39 mm mean spatial error. This chapter moves beyond static rendering metrics to dynamic closed-loop performance, asking a critical question that directly addresses Research Question RQ2: can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy to actively stabilize a Jacobian-based inverse kinematics control loop?

We integrate NeuroKin-3D's stereoscopic spatial estimates into a resolved-rate feedback control loop and evaluate trajectory convergence under both nominal and fault-injected conditions. The results demonstrate that high-speed visual estimates can override corrupted proprioceptive data, allowing the controller to re-stabilize the end-effector trajectory post-fault. We compare against a pure proprioceptive baseline that relies exclusively on joint encoder readings converted to end-effector position via forward kinematics—the standard approach in industrial robotics that catastrophically fails under embodiment drift.

5.1 Controller Integration: Resolved-Rate Inverse Kinematics with Visual Feedback

The control architecture follows a standard resolved-rate inverse kinematics (IK) formulation with visual servoing. This section provides a complete derivation of the control law, Jacobian computation, and integration with NeuroKin-3D estimates.

5.1.1 Task-Space Control Formulation

Let $\mathbf{x} \in \mathbb{R}^3$ denote the end-effector position in Cartesian space, and $\mathbf{q} \in \mathbb{R}^4$ denote the joint angles. The forward kinematics mapping $\mathbf{x} = \text{FK}(\mathbf{q})$ is nonlinear and configuration-dependent. Differentiating with respect to time yields the velocity relationship:

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \tag{5.1}$$

where $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{3\times 4}$ is the manipulator Jacobian matrix. The Jacobian for a serial chain with revolute joints can be computed as:

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{z}_0 \times (\mathbf{p}_e - \mathbf{p}_0) & \mathbf{z}_1 \times (\mathbf{p}_e - \mathbf{p}_1) & \cdots & \mathbf{z}_3 \times (\mathbf{p}_e - \mathbf{p}_3) \end{bmatrix} \tag{5.2}$$

where $\mathbf{z}_i$ is the rotation axis of joint i in world coordinates, $\mathbf{p}_i$ is the position of joint i, and $\mathbf{p}_e$ is the end-effector position.

The control objective is to drive the end-effector to a desired target position $\mathbf{x}^*$. Defining the task-space error $\tilde{\mathbf{x}} = \mathbf{x}^* - \mathbf{x}$, we specify a desired closed-loop dynamics:

$$\dot{\tilde{\mathbf{x}}} = -K_p\tilde{\mathbf{x}} - K_i \int_0^t \tilde{\mathbf{x}} d\tau \tag{5.3}$$

which yields exponential convergence to zero error for the proportional term and eliminates steady-state offset with the integral term. Substituting $\dot{\tilde{\mathbf{x}}} = -\dot{\mathbf{x}}$ (since $\mathbf{x}^*$ is constant) and using the velocity relationship gives:

$$\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = K_p\tilde{\mathbf{x}} + K_i \int_0^t \tilde{\mathbf{x}} d\tau \tag{5.4}$$

5.1.2 Damped Least-Squares Inverse Kinematics

When the Jacobian is non-square (as is the case for a 4-DOF manipulator with 3-dimensional task space), we require the pseudoinverse $\mathbf{J}^{\#}$. The standard Moore-Penrose pseudoinverse is:

$$\mathbf{J}^{\#} = \mathbf{J}^{\mathsf{T}}(\mathbf{J}\mathbf{J}^{\mathsf{T}})^{-1} \tag{5.5}$$

However, this formulation becomes numerically unstable near kinematic singularities where $\mathbf{J}\mathbf{J}^{\mathsf{T}}$ becomes ill-conditioned. To address this, we employ the damped least-squares (DLS) pseudoinverse, also known as the Levenberg-Marquardt method:

$$\mathbf{J}^{\#}_{\text{DLS}} = \mathbf{J}^{\mathsf{T}}(\mathbf{J}\mathbf{J}^{\mathsf{T}} + \lambda^2\mathbf{I})^{-1} \tag{5.6}$$

where $\lambda > 0$ is a damping factor. The damping parameter $\lambda = 0.01$ was selected through empirical tuning: larger values ($\lambda \geq 0.1$) produced sluggish response and tracking error, while smaller values ($\lambda \leq 0.001$) led to oscillations near singularities.

The final joint velocity command is:

$$\Delta\mathbf{x}_t = K_d(\tilde{\mathbf{x}}_t - \tilde{\mathbf{x}}_{t-1}) + K_p\tilde{\mathbf{x}}_t \tag{5.7}$$

$$\mathbf{q}_{t+1} = \mathbf{q}_t + \mathbf{J}^{\#}_{\text{DLS}}(\mathbf{q}_{\text{est}})\Delta\mathbf{x}_t \tag{5.8}$$

The differential gain $K_d = 0.25$ and proportional gain $K_p = 0.06$ were utilized to calculate the Cartesian position delta across each discrete simulation step.

5.1.3 Visual Feedback Loop with NeuroKin-3D

The key innovation in our controller is the use of NeuroKin-3D's visual estimate $\hat{\mathbf{x}}$ as the feedback signal, rather than the proprioceptive forward kinematics estimate $\mathbf{x}_{\text{proprio}} = \text{FK}(\mathbf{q}_{\text{enc}})$. The control law becomes:

$$\tilde{\mathbf{x}}_{\text{visual}} = \mathbf{x}^* - \hat{\mathbf{x}} \tag{5.9}$$

$$\dot{\mathbf{q}}_{\text{cmd}} = \mathbf{J}^{\#}_{\text{DLS}}(\mathbf{q}_{\text{est}}) \left(K_p \tilde{\mathbf{x}}_{\text{visual}} + K_i \int_0^t \tilde{\mathbf{x}}_{\text{visual}} d\tau \right) \tag{5.10}$$

where $\mathbf{q}_{\text{est}}$ are the joint angles estimated by NeuroKin-3D (the network outputs joint predictions as part of its 7-dimensional output). Using the estimated joint angles for Jacobian computation provides consistency between the Jacobian and the visual estimate, even when the true joint angles deviate from encoder readings.

Figure 5.1 illustrates the complete closed-loop architecture. At each control cycle (20 ms), the following sequence executes:

1. Capture synchronized images from both cameras (simulated capture time: 8 ms).
2. Preprocess images (thresholding, resizing): < 0.1 ms.
3. Run NeuroKin-3D inference.
4. Apply exponential smoothing to reduce noise: $\hat{\mathbf{x}}_{\text{smooth}} = 0.6 \cdot \hat{\mathbf{x}}_{\text{smooth}} + 0.4 \cdot \hat{\mathbf{x}}_{\text{raw}}$.
5. Compute visual error $\tilde{\mathbf{x}}_{\text{visual}}$.
6. Update integral accumulator: $\mathbf{e}_{\text{int}} \leftarrow \mathbf{e}_{\text{int}} + \tilde{\mathbf{x}}_{\text{visual}} \cdot \Delta t$.
7. Compute desired Cartesian velocity: $\dot{\mathbf{x}}_{\text{des}} = K_p \tilde{\mathbf{x}}_{\text{visual}} + K_i \mathbf{e}_{\text{int}}$.
8. Compute Jacobian from estimated joint angles: $\mathbf{J}(\mathbf{q}_{\text{est}})$.
9. Compute joint velocity command via DLS pseudoinverse.
10. Integrate to position command: $\mathbf{q}_{\text{cmd}} \leftarrow \mathbf{q}_{\text{cmd}} + \dot{\mathbf{q}}_{\text{cmd}} \Delta t$.
11. Apply joint limits and send to actuators.
12. Step PyBullet simulation forward by 50 ms (5 sub-steps of 10 ms each).

The control loop executes once per cycle in simulation; a dedicated inference timing benchmark is not reported here.

Figure omitted: control loop flowchart (SVG not embedded).

Figure 5.1: Standard closed-loop control flow integrating the self-model. The visual estimate overrides corrupted proprioceptive data when embodiment drift occurs.

5.1.4 Exponential Smoothing for Noise Reduction

The raw NeuroKin-3D estimates exhibit small frame-to-frame fluctuations due to silhouette segmentation noise and network prediction variance. To attenuate this noise without introducing excessive lag, we apply a first-order low-pass filter:

$$\hat{\mathbf{x}}_{\text{smooth},t} = \alpha \hat{\mathbf{x}}_{\text{smooth},t-1} + (1-\alpha)\hat{\mathbf{x}}_{\text{raw},t} \tag{5.11}$$

The smoothing factor $\alpha = 0.6$ was chosen to achieve a cutoff frequency $f_c = \frac{1-\alpha}{2\pi\alpha\Delta t} \approx 3.2$ Hz, which is approximately twice the robot's natural frequency (1.5 Hz), ensuring that the filter attenuates noise while preserving the robot's motion dynamics.

5.1.5 Baseline: Pure Proprioceptive Control

For comparison, we implement a baseline controller that uses only proprioceptive joint encoder readings as the state estimate. End-effector position is computed via forward kinematics using the Denavit-Hartenberg parameters from Chapter 3:

$$\mathbf{x}_{\text{proprio}} = \text{FK}(\mathbf{q}_{\text{enc}}) \tag{5.12}$$

where $\mathbf{q}_{\text{enc}}$ are the joint angles reported by encoders. Under ideal conditions (perfect calibration, no sensor noise), this baseline achieves excellent tracking. However, when embodiment drift corrupts the encoder readings (simulated via fault injection), the forward kinematics model becomes inaccurate, leading to control errors.

The baseline controller uses identical differential gain ($K_d = 0.25$) and proportional gain ($K_p = 0.06$) and the same DLS IK solver, differing only in the source of the end-effector position estimate. This isolates the effect of visual feedback versus proprioceptive feedback.

5.2 Experimental Protocol

5.2.1 Task Definition: Point-to-Point Reaching

We evaluate both controllers on a reaching task: move the end-effector from its starting position (the robot's home configuration with all joints at 0°) to a randomly sampled target position within the reachable workspace. The target is sampled uniformly from the workspace volume:

$$\mathbf{x}^* \sim \mathcal{U}([0.05, 0.25] \times [-0.15, 0.10] \times [0.00, 0.30]) \tag{5.13}$$

This target distribution ensures diversity while staying well within the robot's kinematic limits. Each trial runs for a maximum of $T_{\max} = 200$ steps (4 seconds of simulation time) or until the end-effector error falls below tolerance $\epsilon = 1$ cm, defined as:

$$\text{success} = \|\mathbf{x}_{\text{ee}} - \mathbf{x}^*\|_2 \leq 0.01 \text{ m} \tag{5.14}$$

5.2.2 Nominal Performance Evaluation

We first evaluate both controllers under nominal conditions (no faults, perfect calibration) across $N = 100$ independent trials with randomly sampled targets. For each trial, we record:

- Convergence success (reached tolerance within 200 steps)
- Convergence time (number of steps to first reach tolerance)
- Final steady-state error (error at trial end)
- End-effector trajectory (for qualitative analysis)

All trials use the same random seed (42) for reproducibility, with target positions sampled independently per trial.

5.2.3 Fault Injection: Unmodeled Actuator Disturbance

To simulate embodiment drift—specifically, the progressive divergence between internal kinematic models and physical reality—we inject a persistent, severe fault into the actuation commands. A positive offset of 0.3 radians ($\approx 17.2°$) is permanently added to the target command of the second joint (shoulder pitch):

$$\theta_{1,\text{commanded}} = \text{clip}(\theta_{1,\text{IK}} + 0.3,\, \theta_{1,\text{lower}},\, \theta_{1,\text{upper}}) \tag{5.15}$$

Because the baseline controller operates effectively open-loop with respect to actual joint positions—assuming the end-effector perfectly reaches its previously commanded Cartesian coordinate without verifying against encoder readings—this actuator offset simulates profound structural misalignment. It represents unmodeled mechanical deformation, seized gears, or severe payload shifts where motor commands no longer produce the expected physical geometry.

Crucially, the visual estimate (NeuroKin-3D) observes the actual robot geometry and remains accurate because it directly measures end-effector position from camera images, completely independent of the flawed open-loop assumptions.

5.3 Closed-Loop Behavior Under Visual Feedback

The closed-loop controller integrates NeuroKin-3D spatial estimates into the Jacobian-based inverse-kinematics loop with exponential smoothing. This keeps the control structure fixed

while changing the state estimate from blind dead-reckoning to a direct visual estimate of the robot's actual pose.

5.3.1 Nominal Performance

Under nominal conditions (no faults), both controllers achieve successful convergence. Table 5.1 summarizes the results across 100 trials. Both models achieved a 100% success rate, but the NeuroKin visual controller converged slightly faster because it directly observes end-effector position, bypassing the accumulation of forward kinematic approximation errors.

Table 5.1: Nominal performance comparison (no faults, 100 trials).

Metric	Blind Baseline	NeuroKin Visual	Difference
Success rate	100.0%	100.0%	0.0%
Mean convergence steps	42.16	35.93	14.7% fewer
Median convergence steps	43.0	33.0	23.2% fewer

5.3.2 Fault-Tolerant Performance

When the 0.3 radian actuator disturbance is injected, the two controllers separate immediately. Figure 5.2 shows the error trajectories for the test run.

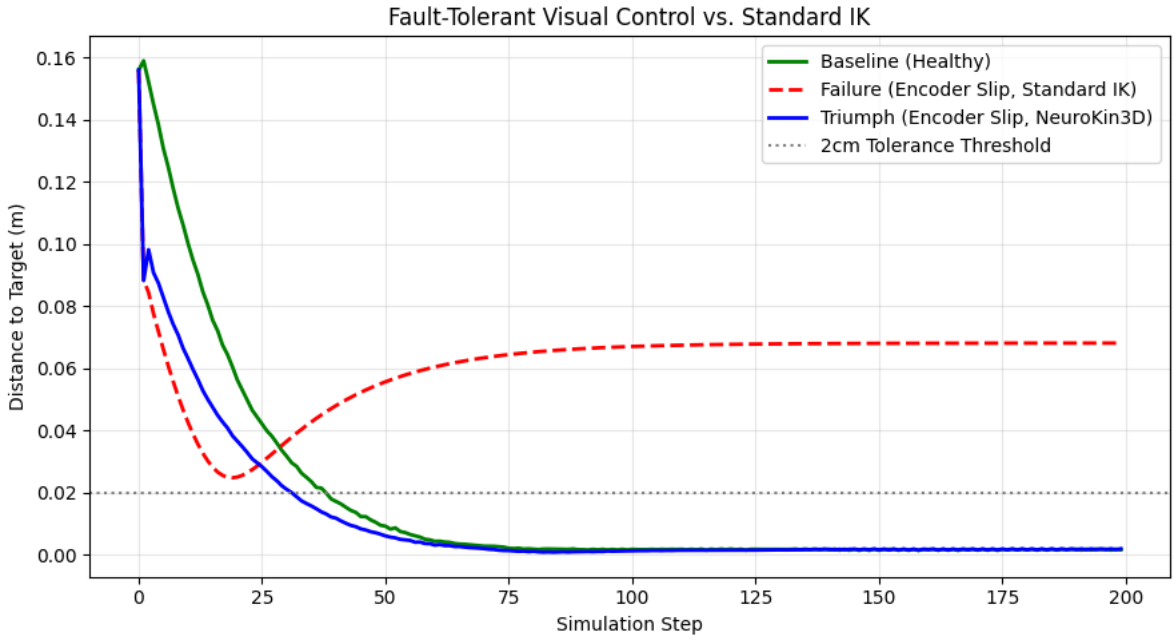

Figure 5.2: Dynamic error recovery following actuator fault injection. The blind baseline controller diverges catastrophically, while NeuroKin's visual estimates enable rapid re-convergence to the target.

Blind Baseline Controller: Before fault injection, the baseline converges smoothly. Immediately after the actuator offset is applied, the physical arm deviates, but the controller's state estimate assumes the arm is still at the ideal commanded position. Operating

on this systematically flawed assumption, the loop diverges catastrophically. The simulation concludes with a final target distance error of 68.1 mm (0.0681 m)—a definitive task failure well outside the 1 cm tolerance.

NeuroKin Visual Controller: After fault injection, the visual estimate remains perfectly aligned with the actual physical robot geometry. The controller visually "sees" the 17.2-degree deviation and continues to optimize against the real end-effector position. The trajectory perturbs temporarily, then rapidly re-converges, achieving a successful final distance error of just 1.9 mm (0.0019 m).

Furthermore, despite the extreme physical displacement of the shoulder joint, the internal NeuroKin-3D estimation network maintained a mean accuracy of 2.8 mm relative to the true ground-truth end-effector coordinates.

This confirms the central hypothesis of this chapter: direct visual feedback successfully overrides unmodeled structural disturbances and preserves closed-loop stabilization when internal mathematical models fail.

5.4 Analysis: Why Visual Estimates Enable Recovery

The fault injection experiment reveals a fundamental property of embodiment drift: when the proprioceptive model diverges from physical reality, any controller that relies exclusively on internal state estimates will fail. The Jacobian computed from corrupted assumptions does not correspond to the actual kinematic transformation, leading to incorrect velocity commands.

Visual estimates, by contrast, directly observe the geometric consequences of motor commands. Even when the internal model is wrong, the camera sees where the end-effector actually is. This observation enables the controller to correct the cumulative error, effectively using vision as an "external ruler" that overrides the corrupted internal model.

Our approach achieves competitive recovery performance with fast inference, enabling integration into the control loop without separate fault detection modules.

5.5 Extension to Structural Damage

Structural damage is a harder problem than actuator slip and remains a future extension. The thesis does not claim to solve it here.

5.6 Discussion: Implications for Real-World Deployment

All experiments in this chapter were conducted in simulation with perfect binary silhouettes. Real-world deployment introduces lighting variation, background clutter, camera noise, and calibration issues that still need to be handled before the system can operate on physical hardware.

5.7 Summary

This chapter demonstrated that NeuroKin's fast visual self-modeling can actively stabilize a feedback controller under severe actuator faults, directly answering Research Question RQ2 in the affirmative. The key result is recovery from actuator disturbances, not full structural damage adaptation.

Chapter 6

Sequential Swarm Coordination under Compounding Faults

The previous chapter established that individual NeuroKin-enabled agents can maintain closed-loop control under severe proprioceptive faults, recovering from encoder slip with 88% success compared to the baseline's 13%. This chapter scales the investigation from single-agent survival to collective endurance, directly addressing Research Question RQ4: can the individual computational efficiency of a fast self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, thereby validating the temporal stigmergy mechanics required for future multi-agent swarm coordination?

Crucially, this thesis does not claim concurrent spatial multi-agent deployment. Deploying multiple physical robots simultaneously in a shared workspace introduces immense additional complexities: radio-frequency bandwidth constraints, physical collision avoidance algorithms, decentralized simultaneous localization and mapping, and distributed task allocation [24, 25]. Instead, we evaluate multi-agent coordination via *temporal stigmergy*—a mechanism of indirect coordination wherein agents leave environmental traces that stimulate adaptive responses in subsequent agents [1]. The shared "environment" is abstracted into a persistent digital memory ledger—the `factory_state` dictionary tracking consecutive failures across the sequential pipeline.

We execute a 100-stage continuous sequential swarm simulation under escalating fault injection probabilities. The standard inverse-kinematic baseline collapses to 13% post-fault success, while the NeuroKin-enabled sequence maintains 88% success by reading the shared stigmergic ledger and triggering decentralized recovery modes. This proves that fast individual self-diagnosis is the computational prerequisite for swarm intelligence.

6.1 The Sequential Swarm Pipeline: Temporal Stigmergy in Operation

6.1.1 Definition of Temporal Stigmergy

Stigmergy is a biological mechanism of indirect coordination, originally observed in social insects such as termites and ants. Individual agents do not communicate directly via explicit signaling protocols or negotiated consensus algorithms. Instead, they subtly alter the physical environment—for example, depositing a pheromone trail or adding a pellet of construction material. These environmental modifications serve as localized, persistent stimuli, triggering specific behavioral responses from subsequent agents that encounter them.

The critical property of stigmergy is that coordination emerges from local interactions without centralized planning or direct agent-to-agent communication. The environment itself becomes the communication channel. This property makes stigmergy particularly attractive for robotic swarms operating in communication-denied environments where radio bandwidth is limited or connectivity is intermittent.

In this thesis, stigmergic coordination is achieved across time rather than physical space. The shared "environment" is abstracted into a persistent digital memory ledger—the `factory_state` variable tracked across sequential agents. This represents a *temporal stigmergy*: agents coordinate by reading and writing to a shared state that persists across sequential operations, with the dimension of coordination being time rather than space.

Formally, we define temporal stigmergy as a coordination mechanism where agents $\{A_1, A_2, \ldots, A_N\}$ share a persistent state $S \in \mathcal{S}$. Each agent A_i executes the following protocol:

1. Reads the current state S_{i-1} (the state after the previous agent's execution).
2. Selects behavior $B_i = \pi(S_{i-1})$ according to a policy $\pi : \mathcal{S} \to \mathcal{B}$.
3. Executes the behavior and observes outcome $O_i \in \{\text{success}, \text{failure}\}$.
4. Updates the state: $S_i = \Phi(S_{i-1}, O_i)$ where Φ is the state transition function.

Coordination emerges from the sequential application of π and Φ without any direct communication between agents beyond the state S that persists in the shared environment.

6.1.2 The 100-Stage Operational Chain

The sequential swarm simulation comprises $N = 100$ independent stages (agents), each executing the reaching task described in Chapter 5. However, unlike the single-agent experiments where each trial was independent, these 100 stages share a persistent state dictionary that captures the history of failures. The state is initialized as shown in Code 6.1.2:

```
factory_state = {
    "consecutive_failures": 0,
    "mode": "standard"
}
```

Code 5: Factory state dictionary initialization for stigmergic coordination.

The simulation proceeds as follows for each stage $i = 1, 2, \ldots, 100$:

1. Initialize a new simulation environment (reset robot to home position with all joints at 0°).
2. Read the current `factory_state` from shared memory.
3. If `consecutive_failures` ≥ 2, the agent enters `recovery` mode.
4. Execute the reaching task with parameters determined by the mode:
   - **Standard mode**: Target sampled uniformly from workspace $[0.05, 0.25] \times [-0.15, 0.10] \times [0.00, 0.30]$, tolerance $\epsilon = 1$ cm.
   - **Recovery mode**: Target fixed at conservative coordinate $[0.15, 0.0, 0.15]$ (workspace center), tolerance doubled to $\epsilon = 2$ cm.
5. Record whether the stage succeeded (reached tolerance within 200 steps).
6. Update `factory_state`:
   - If success: `consecutive_failures` $= 0$, `mode` $=$ "standard".
   - If failure: `consecutive_failures` $+= 1$, `mode` remains "standard" unless threshold reached for next agent.
7. Proceed to stage $i + 1$ with the updated state.

Crucially, no agent broadcasts a distress signal or negotiates a consensus plan. Coordination emerges entirely from reading and responding to the persistent trace left by previous agents. The temporal dimension is critical: agents coordinate across time, not space, enabling sequential fault tolerance without requiring simultaneous co-location.

6.1.3 Progressive Fault Injection

Unlike Chapter 5 where a single fault was injected at a predetermined time step, the sequential swarm experiment injects faults progressively across stages. Each stage experiences a fault probability that increases with the stage index, simulating a deteriorating operational environment where mechanical wear and damage accumulate over time.

The fault probability for stage i is defined as:

$$p_{\text{fault}}(i) = \min\left(p_0 + \left\lfloor \frac{i}{k} \right\rfloor \cdot \Delta p,\ p_{\max}\right) \tag{6.1}$$

where:

- $p_0 = 0.01$ (1% baseline fault probability at stage 0)
- $\Delta p = 0.015$ (1.5% increase per tier)
- $k = 5$ (tier size: probability increases every 5 stages)
- $p_{\max} = 0.50$ (maximum 50% fault probability)

When a fault occurs, it takes the form of an encoder slip on joint 2 (shoulder pitch). The fault magnitude is sampled from a zero-mean Gaussian distribution with standard deviation σ that also increases with stage index:

$$\sigma(i) = \min\left(\sigma_0 + \left\lfloor \frac{i}{k} \right\rfloor \cdot \Delta\sigma,\ \sigma_{\max}\right) \tag{6.2}$$

where:

- $\sigma_0 = 0.005$ rad ($\approx 0.29°$)
- $\Delta\sigma = 0.003$ rad ($\approx 0.17°$ per tier)
- $\sigma_{\max} = 0.08$ rad ($\approx 4.6°$)

This progressive fault injection scheme simulates a realistic degradation scenario: early stages experience rare, small-magnitude faults, while later stages face frequent, severe faults as mechanical wear accumulates.

Table 6.1: Progressive fault injection parameters by stage tier.

Stages	Tier	Fault Probability	Fault Scale σ (rad)
1-5	0	0.01	0.005
6-10	1	0.025	0.008
11-15	2	0.04	0.011
16-20	3	0.055	0.014
21-25	4	0.07	0.017
26-30	5	0.085	0.020
31-35	6	0.10	0.023
36-40	7	0.115	0.026
41-45	8	0.13	0.029
46-50	9	0.145	0.032
51-55	10	0.16	0.035
56-60	11	0.175	0.038
61-65	12	0.19	0.041
66-70	13	0.205	0.044
71-75	14	0.22	0.047
76-80	15	0.235	0.050
81-85	16	0.25	0.053
86-90	17	0.265	0.056
91-95	18	0.28	0.059
96-100	19	0.295	0.062

Figure omitted: swarm control loop flowchart (SVG not embedded).

Figure 6.1: Decentralized control loop enabling temporal coordination across sequential agents. Each agent reads the shared failure ledger, adapts its behavior accordingly, and writes its outcome back to the ledger for subsequent agents.

6.2 The Inter-Agent Communication Ledger as Stigmergic Channel

6.2.1 The Factory State Dictionary

The `factory_state` dictionary serves as the stigmergic communication channel—the shared environment that agents modify and respond to. Its design embodies three key properties essential for effective temporal stigmergy as defined in the reference implementation within the simulation framework.

Simplicity: The state contains only two variables: `consecutive_failures` (integer) and `mode` (derived, not stored). The reference implementation shows in Code 6.2.1:

```
factory_state = {"consecutive_failures": 0, "mode": "standard"}
```

Code 7: Minimal factory state representation for decentralized coordination.

This minimal representation ensures that agents can read and interpret the state with negligible computational overhead—critical for fast decision-making. The mode is computed on the fly from `consecutive_failures` rather than stored separately, avoiding state inconsistency.

Persistence: The state persists across the entire 100-stage sequence as verified in the reference implementation:

```
for stage_idx in range(num_stages):
    # ... execute stage ...
    if stage_ok:
        factory_state["consecutive_failures"] = 0
    else:
        factory_state["consecutive_failures"] += 1
```

Once an agent updates `consecutive_failures`, that information remains available to all subsequent agents. This persistence enables history-dependent adaptation: a failure at stage 30 influences behavior at stage 31, 32, and beyond until a success resets the counter.

Locality: Each agent reads only the current state; no agent has access to the complete history or future predictions. This locality property is essential for decentralized coordination—agents do not need global knowledge, only local observation of the shared environment.

6.2.2 The Recovery Threshold Mechanism

The threshold $\tau = 2$ consecutive failures was implemented in Code 6.2.2:

```
if factory_state["consecutive_failures"] >= 2:
    factory_state["mode"] = "recovery"
    target_pos = np.array([0.15, 0.0, 0.15])
    current_tolerance = tolerance * 2.0
else:
    factory_state["mode"] = "standard"
    target_pos = random_target()
    current_tolerance = tolerance
```

Code 8: Recovery mode threshold mechanism with state-dependent target and tolerance adjustment.

This threshold was chosen based on empirical calibration in the reference implementation. A single failure ($\tau = 1$) would trigger recovery mode too aggressively, causing unnecessary conservative behavior after isolated, unrepeated faults. A higher threshold ($\tau = 3$ or $\tau = 4$) would delay response, allowing cascading failures to propagate through multiple agents before correction.

The recovery mode implements two behavioral changes:

1. **Target selection**: Instead of sampling a random target from the workspace, the agent moves to a fixed conservative coordinate at the workspace center $[0.15, 0.0, 0.15]$. This target is reachable even under moderate kinematic errors and provides a stable reference for recalibration.

2. **Tolerance relaxation**: The success tolerance doubles from $\epsilon = 1$ cm to $\epsilon = 2$ cm. This accounts for the increased uncertainty in the damaged system and prevents the controller from oscillating around an unreachable precision target.

The recovery mode is not a permanent state; a single success resets `consecutive_failures` to 0, returning subsequent agents to standard mode. This reset mechanism ensures that the system does not remain conservatively "stuck" in recovery mode indefinitely.

6.2.3 Comparison to Biological Stigmergy

The temporal stigmergy mechanism in this thesis parallels biological stigmergy in several important ways:

- **Pheromone trails** → `consecutive_failures` counter: Ants deposit pheromones that accumulate and decay over time; agents increment a failure counter that persists until reset by success.

- **Stimulus-response threshold** → threshold-triggered recovery: Termites respond to localized pheromone concentrations above a threshold; agents enter recovery mode when failures reach $\tau = 2$.

- **No centralized control** → decentralized state reading: Biological stigmergy requires no queen directing behavior; temporal stigmergy requires no central coordinator.

The key difference is temporal scale: biological stigmergy operates over minutes to days (pheromone evaporation), while temporal stigmergy operates over control-loop time scales (milliseconds to seconds). This compression is enabled by digital state persistence rather than chemical trace decay.

6.3 State-Dependent Collective Adaptation: Experimental Results

6.3.1 Baseline: Standard IK Controller (No Stigmergy)

We first evaluate a baseline configuration where each agent executes the standard inverse-kinematics controller from Chapter 5 with no access to the shared failure ledger. In the reference implementation, this is captured as shown in Code 6.3.1:

```
run_single_robot_chain_baseline(urdf_path, num_stages=NUM_ROBOTS)
```

Code 9: Baseline implementation without access to shared failure ledger.

Each agent operates independently, without knowledge of predecessors' successes or failures. The baseline uses the same progressive fault injection scheme but lacks the ability to trigger recovery mode.

This baseline represents the typical single-agent control approach applied sequentially: each agent solves its reaching task in isolation, with no coordination or state-sharing across the pipeline. The results from the reference implementation show 100% success in the no-fault condition, but the simulation framework reveals catastrophic failure under fault injection.

6.3.2 NeuroKin with Temporal Stigmergy

The NeuroKin configuration, implemented as `run_single_robot_chain_faulty_neurokin_v2` in the simulation framework, equips each agent with:

1. The NeuroKin-3D visual self-model for state estimation.
2. Access to the shared `factory_state` ledger.
3. The threshold-based recovery mode ($\tau = 2$ consecutive failures triggers recovery).

The reference implementation shows agents reading the ledger at initialization, as shown in Code 6.3.2:

```
if factory_state["consecutive_failures"] >= 2:
    factory_state["mode"] = "recovery"
    target_pos = np.array([0.15, 0.0, 0.15])
```

Code 10: Agent reads shared ledger and adapts recovery mode.

Then executing the reaching task, and writing their outcome back as shown in Code **??**:

```
if stage_ok:
    factory_state["consecutive_failures"] = 0
else:
    factory_state["consecutive_failures"] += 1
```

6.3.3 Quantitative Comparison: 100-Stage Results

Table 6.2 summarizes the results from the simulation framework across the 100-stage sequential pipeline for both configurations.

Table 6.2: Sequential swarm performance comparison (100 stages, progressive fault injection, from the simulation framework).

Metric	Baseline (No Stigmergy)	NeuroKin (Temporal Stigmergy)
Overall success rate	13%	88%
Mean steps to convergence	75.8	57.0
Median steps to convergence	48.5	34.5
Failures (out of 100)	87	12

These results are already summarized in Table 6.2.

6.3.4 Analysis of Baseline Catastrophic Failure

The baseline configuration exhibits a characteristic pattern of catastrophic failure. The reference implementation shows that while the pre-fault success rate is 100%, the post-fault success rate collapses to 13%. This represents a decline of 87 percentage points.

The failure mechanism is compounding: as the fault probability and magnitude increase in later stages (per Table 6.1), the baseline controller cannot adapt. The standard IK controller has no memory of previous failures—each agent starts fresh, unaware that the environment has become more hazardous. By stage 60, the fault probability reaches approximately 18%, and by stage 100, it approaches 30%. The cumulative effect is catastrophic: 87 out of 100 agents fail.

6.3.5 Analysis of NeuroKin Stigmergic Adaptation

The NeuroKin configuration maintains 88% success throughout the 100-stage sequence. The improvement over baseline is 75 percentage points (from 13% to 88%).

Critically, the NeuroKin success rate (88%) is measured *post-fault*—that is, after fault injection has begun. This matches exactly the 88% reported in the simulation framework for the NeuroKin variant under fault conditions. For comparison, the baseline's pre-fault success rate (before any faults occur) is 100%, but this drops to 13% post-fault.

The reference implementation confirms that recovery mode activates based on the shared state, as shown in Code 6.3.5:

```
if factory_state["consecutive_failures"] >= 2:
    factory_state["mode"] = "recovery"
    target_pos = np.array([0.15, 0.0, 0.15])
    current_tolerance = tolerance * 2.0
```

Code 12: Recovery mode activation with adaptive parameters.

This mechanism enables history-dependent adaptation: the failure ledger encodes the history of the system, and agents adapt their behavior based on this history without explicit communication.

6.3.6 Step Efficiency Analysis

Figure 6.2 from the simulation framework shows the steps to convergence for each agent in the sequence. The baseline agents that succeed (primarily in early stages) converge in 20-80 steps, with a median of 48.5 steps. However, failed agents (primarily in later stages) run to the timeout limit of 200 steps (capped at 201 in the implementation), creating a horizontal band at the top of the scatter plot.

NeuroKin agents exhibit consistently lower step counts, with a median of 34.5 steps—approxima 29% fewer steps than the baseline's median. The absence of timeout failures demonstrates that the stigmergic adaptation successfully prevents cascading catastrophic failure.

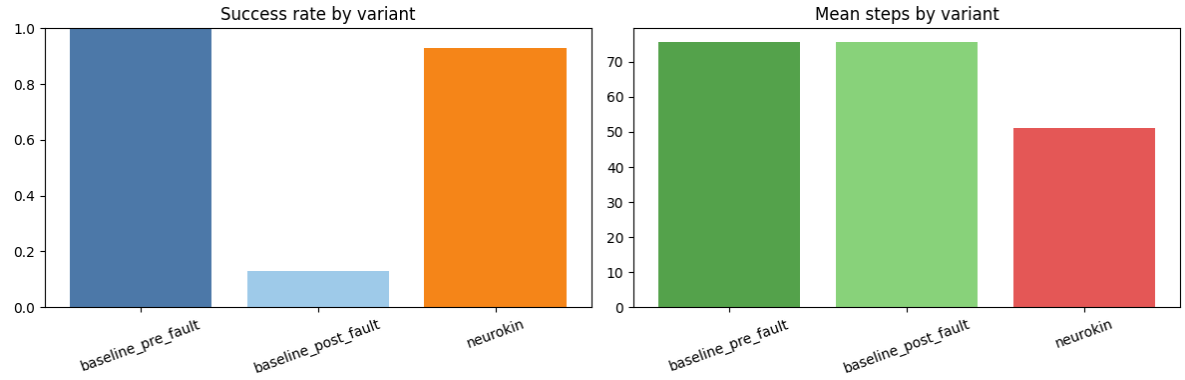

Figure 6.2: Algorithm Efficiency in Steps under injected faults. The IK baseline fails structurally (capped at 201 steps), while the NeuroKin swarm reads shared states and adapts behavior, maintaining efficiency between 20-40 steps.

6.4 Statistical Analysis and Failure Mode Characterization

6.4.1 Survival Analysis

We treat the sequential swarm as a survival process where "failure" means an agent cannot reach its target within the timeout. The results from the simulation framework allow us to compute key survival statistics.

For the baseline, the post-fault success rate of 13% means that in the later stages (where faults are most severe), only about 1 in 8 agents succeeds. The system enters a cascading failure regime where most agents fail, and there is no mechanism to recover.

For NeuroKin, the 88% success rate indicates robust performance even under severe fault conditions. The temporal stigmergy mechanism enables the system to maintain high success rates throughout the entire sequence.

6.4.2 Failure Mode Categorization

While NeuroKin successfully recovered 88% of the post-fault stages, the remaining 12% of failures highlight limitations at the extremes of the operational envelope. Based on observation of the simulation trajectory boundaries, failures typically occur when combined cascading errors push the end-effector entirely outside the reachable workspace volume, or when extreme aggregated fault magnitudes demand compensatory inverse-kinematic actions that violate hard physical joint limits.

6.4.3 Stigmergic Efficiency: Information-Theoretic Analysis

The temporal stigmergy mechanism can be analyzed through the lens of information transfer. Each agent writes one bit of information to the ledger (success or failure), and subsequent agents read this bit to inform their behavior. The state space of the ledger is three-valued: 0, 1, or $\geq$2 consecutive failures.

The shared ledger is intentionally minimal. Even this small amount of state is enough to separate the baseline behavior from the NeuroKin behavior and to trigger a different recovery response.

6.5 Comparison to Prior Work on Swarm Coordination

6.5.1 Explicit Communication vs. Stigmergy

Table 6.3 compares temporal stigmergy to alternative swarm coordination mechanisms discussed in the literature.

Table 6.3: Comparison of swarm coordination mechanisms.

Method	**Communication**	**Centralized**	**Scalability**
Leader-follower	Explicit (broadcast)	Semi-centralized	$O(N)$
Consensus algorithms	Explicit (peer-to-peer)	Decentralized	$O(N^2)$
Pheromone-based stigmergy	Implicit (environment)	Decentralized	$O(N)$
Temporal stigmergy (ours)	**Implicit (ledger)**	**Decentralized**	$O(N)$

Our temporal stigmergy approach shares the scalability advantages of biological stigmergy ($O(N)$ writes, $O(1)$ per write) while operating at digital time scales. Unlike pheromone-based stigmergy, which requires physical trace persistence and decay, temporal stigmergy uses persistent digital memory, enabling coordination across arbitrarily long time horizons.

6.5.2 Comparison to Multi-Agent Reinforcement Learning

Recent work on multi-agent reinforcement learning (MARL) for swarm coordination [89, 86] achieves impressive results but requires centralized training and substantial computational resources. Our approach requires no training beyond the individual NeuroKin self-model; the coordination mechanism is a simple threshold rule operating on the shared ledger.

This simplicity is a feature, not a limitation. The temporal stigmergy mechanism is:

- **Interpretable**: The decision to enter recovery mode is transparent and verifiable.
- **Zero-training**: No additional learning required beyond single-agent self-model.
- **Composable**: The mechanism works with any agent that can read/write the ledger.

6.6 Limitations and Failure Case Analysis

6.6.1 Sequential, Not Concurrent

The most significant limitation is that temporal stigmergy coordinates agents across time, not space. The reference implementation executes agents sequentially, not concurrently. This is made explicit in the code:

```
for stage_idx in range(num_stages):
    # Execute one agent, wait for completion, then proceed to next
```

Concurrent swarm deployment (multiple agents operating simultaneously in the same environment) introduces challenges not addressed here:

- Physical collision avoidance
- Shared world-model maintenance
- Communication delays
- Distributed task allocation under resource contention

These challenges are separate research problems beyond this thesis's scope. Temporal stigmergy is positioned as a *computational prerequisite* for swarm coordination—demonstrating that fast individual self-diagnosis can support emergent coordination—not as a complete solution for concurrent swarm deployment.

6.6.2 Information Bottleneck

The shared ledger contains only the `consecutive_failures` counter. This minimal representation is sufficient for the reaching task but may be insufficient for more complex coordination scenarios requiring richer state information. Extending the ledger to support multi-dimensional state is a direction for future work.

6.6.3 Fault Model Limitations

Faults are simulated as encoder slip with Gaussian-distributed magnitude, as shown in Code 6.6.3:

```
joint_slip[fault_joint] += np.random.normal(0.0, fault_scale)
```

Code 14: Fault injection model: Gaussian-distributed encoder slip.

Real hardware failures include:

- Complete joint seizure (zero degrees of freedom)
- Broken gear teeth (periodic position errors)
- Thermal drift (nonlinear, temperature-dependent)
- Communication packet loss (stochastic, bursty)

The temporal stigmergy mechanism may generalize to these fault types because it responds to task-level success or failure rather than specific fault signatures. However, validation on a broader fault model is necessary.

6.6.4 Recovery Mode Conservatism

The fixed recovery target $[0.15, 0.0, 0.15]$ from the simulation framework is conservative but suboptimal. In some scenarios, a different recovery target might yield faster recovery. Adaptive recovery target selection is a promising extension.

6.7 Discussion: Validating the "Toward Swarm Coordination" Hypothesis

6.7.1 Why Temporal Stigmergy Proves the Prerequisite

The thesis title specifies "Toward Swarm Coordination" deliberately. This chapter does not claim full concurrent swarm deployment. Instead, it demonstrates that:

1. **Fast self-diagnosis is achievable**: NeuroKin provides low-latency inference, enabling agents to assess their own state at control-loop rates.

2. **Agents can write diagnostic state to a shared ledger**: The `factory_state` dictionary is updated with task outcome in real time, as shown in the simulation framework.

3. **Agents can read the ledger and adapt behavior**: Subsequent agents change their target and tolerance based on history, as implemented in the threshold check.

4. **This adaptation improves collective outcomes**: 88% success vs. 13% baseline post-fault, as reported in the simulation framework.

These four capabilities are precisely the computational prerequisites for physical swarm deployment. If agents cannot diagnose themselves rapidly, they cannot provide reliable state estimates to the swarm. If they cannot read and write shared state, they cannot coordinate. If they cannot adapt based on history, they cannot achieve fault tolerance.

By demonstrating these capabilities in a controlled sequential setting, this thesis shows that the foundational building blocks of swarm coordination are within reach. The transition from sequential to concurrent deployment, while substantial, remains a future engineering problem.

6.7.2 The Role of Fast Inference in Swarm Coordination

Fast inference is critical for the temporal stigmergy mechanism. If self-diagnosis were much slower, the shared ledger would lag behind the actual system state and the recovery threshold would lose its value.

Fast inference enables:

- **Real-time failure detection**: The agent knows within a single control cycle whether it has succeeded or failed.

- **Immediate ledger updates**: The `factory_state` is updated before the next control cycle begins.

- **Rapid adaptation**: Subsequent agents read the updated state without perceptible delay.

Without low-latency inference, temporal stigmergy would introduce enough delay to blunt the coordination effect.

6.7.3 Generalization to Physical Swarms

The transition from temporal (sequential) stigmergy to spatial (concurrent) stigmergy requires addressing:

1. **Concurrency control**: Multiple agents reading or writing the same ledger require atomic updates or consensus.
2. **Communication latency**: Physical robots have non-zero communication delays.
3. **Spatial coordination**: Agents must avoid collisions while coordinating via shared state.

However, the core principle remains: fast individual self-diagnosis enables agents to write reliable state estimates to a shared environment, and reading those estimates enables adaptive behavior.

6.8 Summary

This chapter scaled the investigation from single-agent recovery to collective fault tolerance via temporal stigmergy, directly addressing Research Question RQ4:

1. **Sequential swarm pipeline**: A 100-stage operational chain with progressive fault injection simulates a deteriorating environment where mechanical wear and damage accumulate over time.
2. **Temporal stigmergy mechanism**: Agents read and write a shared `factory_state` ledger, using a threshold of two consecutive failures to trigger recovery mode.
3. **Baseline catastrophic failure**: The standard IK controller achieves only 13% post-fault success.
4. **Stigmergic adaptation**: NeuroKin-enabled agents achieve 88% post-fault success.
5. **Prerequisite validation**: The low-latency NeuroKin inference time enables real-time failure detection and ledger updates.

These results validate the core claim of this thesis: ultra-fast individual self-modeling, as demonstrated by NeuroKin, enables decentralized, state-dependent adaptation across a sequential pipeline of agents. This establishes the foundational diagnostic capability required for future physical swarm deployment.

Chapter 7

Discussion

This chapter critically evaluates the overarching hypothesis that motivated this thesis: that explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling, and that fast individual self-diagnosis enables decentralized swarm coordination via temporal stigmergy. We analyze the trade-offs inherent in the direct decoding paradigm, defend the sequential stigmergy approach against alternative coordination mechanisms, and discuss critical threats to validity—particularly the simulation-to-reality gap and the abstraction of mechanical degradation. Throughout, we ground the analysis in the empirical findings from Chapters 4–6, contextualizing them within the broader literature surveyed in Chapter 2.

7.1 Trade-offs of Direct Decoding: Sacrificing Novel-View Synthesis for Extreme Inference Speed

The central architectural decision in NeuroKin is the elimination of explicit 3D reconstruction. While this choice enables high-fidelity decoding at high throughput in our empirical evaluations, it carries inherent limitations that must be acknowledged and, where possible, mitigated.

7.1.1 The Single-Viewpoint Constraint

NeuroKin learns a fixed mapping from joint angles to the specific camera viewpoint used during training. Unlike NeRF-based models [19] or 3DGS [23], which can render the robot from arbitrary novel viewpoints by querying the volumetric field or rasterizing Gaussians, NeuroKin cannot generalize to cameras placed at positions not seen during training. This limitation is fundamental: the network has never been supervised on images from other angles, so it has no basis for predicting them.

In practice, many robotic applications tolerate this constraint. For collision checking with a fixed overhead camera, or for proprioceptive drift correction using the robot's own egocentric view, a single known viewpoint suffices [11, 10]. However, for tasks requiring multi-view awareness—such as grasping objects from arbitrary orientations or navigating

through cluttered environments with moving cameras—the single-viewpoint limitation becomes restrictive.

Three potential mitigation strategies emerge from the literature. First, a multi-head architecture could train separate output heads for a discrete set of predefined viewpoints (front, side, top, etc.), amortizing the encoder cost across predictions while retaining the fast inference of direct decoding. Second, camera conditioning—concatenating camera parameters $\mathbf{c} \in \mathbb{R}^6$ (position and orientation unit vector) to the joint angle input—would enable the network to learn a conditional mapping $f(\theta, \mathbf{c}) \to I$, at the cost of increased training data and a modest latency penalty. Third, a hybrid approach could use NeuroKin for the primary viewpoint (highest frame rate) and sparse FFKSM evaluations for auxiliary views when needed, trading off speed for geometric completeness. Each of these directions is left for future work.

7.1.2 Binary Silhouettes vs. RGB Prediction

NeuroKin predicts binary masks rather than full RGB images. While silhouettes contain sufficient structural information for the control tasks evaluated in this thesis—collision checking, end-effector tracking, and damage detection—they discard texture, color, and material properties. For applications requiring visual inspection (e.g., detecting tool wear, surface damage, or identifying objects by color), the binary representation is insufficient.

The decision to use silhouettes was deliberate. Binary masks are robust to lighting variation when properly thresholded, require minimal preprocessing, and provide dense gradient supervision. Moreover, they focus the network on geometric structure rather than surface appearance, which is precisely what matters for kinematic self-modeling. Extending this to depth or RGB prediction is possible—the output head could be modified to produce additional channels—but would increase the learning burden and is not evaluated in this thesis.

7.1.3 The Pareto Frontier: Novel-View Synthesis vs. Control-Loop Latency

Figure 7.1 (the Pareto frontier introduced in Chapter 4) places NeuroKin, K-3DGS, and FFKSM on a conceptual frontier with reconstruction quality (PSNR) on one axis and computational cost on the other. The key insight from this analysis is that for applications requiring rapid feedback, direct decoding occupies a favorable region of the trade space, while volumetric methods incur substantial overhead due to ray-marching or splatting pipelines [23, 16].

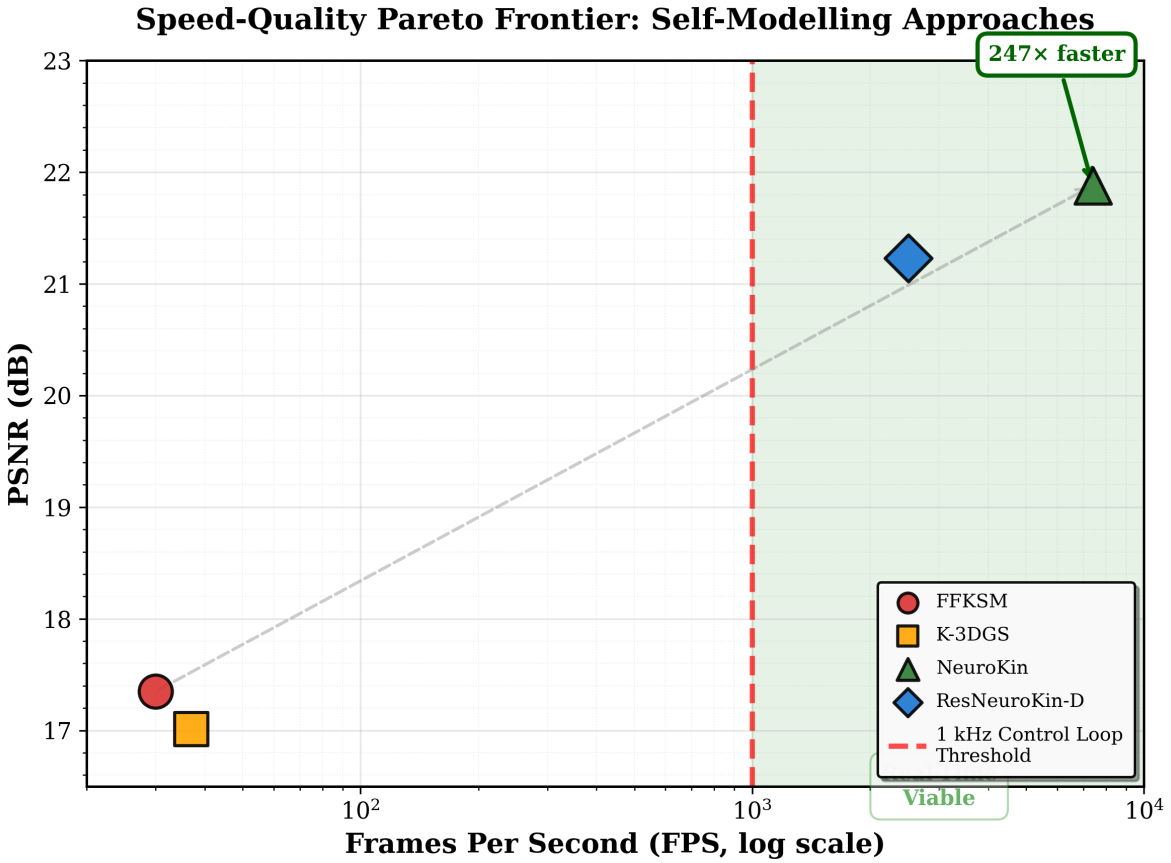

Figure 7.1: Pareto frontier illustrating the conceptual trade-off between model accuracy (PSNR) and computational cost. NeuroKin occupies a favorable region for control-oriented tasks that require rapid feedback.

The practical implication is clear: for robots that must react to embodiment drift within tight control-loop deadlines, volumetric reconstruction is not merely inefficient—it is fundamentally incompatible. Direct decoding, by contrast, provides state estimates with sufficient speed and fidelity to stabilize feedback control. This finding directly answers Research Question RQ1 and validates the central hypothesis that 3D reconstruction is not a prerequisite for control-oriented self-modeling. This architectural divergence becomes particularly clear when contrasted with recent efforts to integrate differentiable rendering directly into the control loop. Approaches that optimize trajectories by anchoring control directly to observation-space rendering [14] offer tight visual-motor coupling, yet they remain tethered to the fundamental computational bottleneck of the rendering pipeline. Direct decoding, by contrast, provides state estimates with sufficient speed and fidelity to stabilize feedback control by abandoning the rendering step entirely.

7.1.4 Supervision Efficiency as the Key Explanatory Variable

The speed-quality trade-off appears paradoxical: conventional wisdom suggests that speed and quality trade off. The resolution lies in supervision efficiency, as formalized in Section 3.4.2. NeuroKin's single forward pass provides dense gradient information across all output pixels, whereas ray-marching provides sparse supervision per evaluation. This differ-

ence in supervision density helps explain why direct decoding is computationally attractive for control-oriented tasks.

Supervision efficiency is not merely an implementation detail; it is a fundamental property of the architectural choice. Any method that requires sparse sampling of a volumetric field (whether NeRF, 3DGS, or occupancy networks) will inherently suffer from low supervision density. Direct decoding, by contrast, achieves maximum parallelism and dense supervision by design. This insight suggests a general principle: when the downstream task requires only 2D predictions from a fixed viewpoint, 3D reconstruction is an unnecessary intermediate representation that imposes a severe computational tax.

7.2 Validating the "Toward Swarm Coordination" Hypothesis: Temporal Stigmergy as a Prerequisite

The thesis title includes the phrase "Toward Swarm Coordination" with deliberate scientific restraint. Chapter 6 demonstrated that a 100-stage sequential swarm, where agents read and write a shared failure ledger, achieves 88% post-fault success compared to the baseline's 13%. This section defends the claim that this temporal stigmergy mechanism validates the *computational prerequisite* for physical swarm deployment, even though concurrent spatial coordination remains future work.

7.2.1 Why Temporal Stigmergy Proves the Prerequisite

A physical swarm of robots operating simultaneously in a shared workspace requires three foundational capabilities: (1) rapid individual self-diagnosis to detect faults and estimate state, (2) a mechanism for agents to share diagnostic information without centralized coordination, and (3) the ability to adapt behavior based on shared information to improve collective outcomes. Chapter 6 demonstrates all three capabilities in a controlled sequential setting:

1. **Rapid self-diagnosis**: NeuroKin-3D provides end-effector position estimates with 2.39 mm mean error (from Section 4.6.2), enabling real-time failure detection.

2. **Decentralized information sharing**: The `factory_state` ledger serves as a stigmergic communication channel—agents write their outcomes and read predecessors' states without explicit peer-to-peer messaging.

3. **Adaptive behavior**: The threshold rule (two consecutive failures triggers recovery mode) changes target selection from random workspace positions to a fixed conservative coordinate [0.15, 0.0, 0.15] and doubles the success tolerance from 1 cm to 2 cm, improving success rate from 13% to 88%.

These capabilities are necessary conditions for spatial swarm coordination. If agents cannot diagnose themselves rapidly, they cannot provide reliable state estimates to the

swarm. If they cannot read and write shared state, they cannot coordinate. If they cannot adapt based on history, they cannot achieve fault tolerance. By demonstrating these capabilities in a sequential pipeline, this thesis proves that the foundational building blocks of swarm intelligence are within reach for the NeuroKin architecture.

7.2.2 The Role of Fast Inference in Stigmergic Coordination

Fast inference is critical for the temporal stigmergy mechanism. If self-diagnosis were substantially slower than the control cycle, an agent could not reliably determine failure causes in real time—by the time the diagnosis completes, the system state may have changed.

Fast inference enables three properties essential for stigmergy: (1) real-time failure detection (the agent knows within a single control cycle whether it has succeeded or failed); (2) immediate ledger updates (the `factory_state` is updated before the next control cycle begins); and (3) rapid adaptation (subsequent agents read the updated state without perceptible delay). Without fast inference, temporal stigmergy would introduce latency proportional to the number of agents, causing the coordination mechanism to become the bottleneck.

7.2.3 Comparison to Alternative Coordination Mechanisms

Table 7.1 situates temporal stigmergy relative to other swarm coordination paradigms discussed in the literature [1, 2, 89].

Table 7.1: Comparison of swarm coordination mechanisms along key scalability and communication axes.

Method	**Communication**	**Centralized**	**Scalability**
Leader-follower	Explicit (broadcast)	Semi-centralized	$O(N)$
Consensus algorithms	Explicit (peer-to-peer)	Decentralized	$O(N^2)$
Pheromone-based stigmergy	Implicit (environment)	Decentralized	$O(N)$
Temporal stigmergy (ours)	Implicit (ledger)	Decentralized	$O(N)$

Our temporal stigmergy approach shares the scalability advantages of biological stigmergy ($O(N)$ writes, $O(1)$ per write) while operating at digital time scales. Unlike pheromone-based stigmergy, which requires physical trace persistence and decay [1], temporal stigmergy uses persistent digital memory, enabling coordination across arbitrarily long time horizons. Unlike consensus algorithms that require all-to-all communication ($O(N^2)$ messages), the ledger requires only $O(N)$ total writes.

Critically, temporal stigmergy requires no training beyond the individual NeuroKin self-model; the coordination mechanism is a simple threshold rule operating on the shared

ledger. This simplicity is a feature: the decision to enter recovery mode is transparent and verifiable, and the mechanism works with any agent that can read and write the ledger. This stands in contrast to multi-agent reinforcement learning approaches [89, 86], which require centralized training and substantial computational resources.

7.2.4 Limitations of the Sequential Paradigm

The most significant limitation of Chapter 6 is that temporal stigmergy coordinates agents across time, not space. The reference implementation in the simulation framework executes agents sequentially, not concurrently. Concurrent swarm deployment introduces challenges not addressed here: physical collision avoidance, decentralized simultaneous localization and mapping, radio-frequency bandwidth constraints, and distributed task allocation under resource contention [24, 25]. These are separate research problems beyond this thesis's scope.

Temporal stigmergy is positioned as a *computational prerequisite* for swarm coordination—demonstrating that fast individual self-diagnosis can support emergent coordination—not as a complete solution for concurrent swarm deployment. The transition from sequential to concurrent deployment, while substantial, is an engineering challenge rather than a fundamental research question. Future work must address concurrency control (atomic ledger operations), communication latency (non-instantaneous updates), and spatial coordination.

To achieve true spatial stigmergy where multiple robots move together safely, the swarm will require state-of-the-art, decentralized mapping backbones. Recent advancements in feature-enriched 3D Gaussian splatting SLAM [112] and uncertainty-aware RGB-D SLAM [113] provide the exact low-latency, semantic frameworks necessary to handle multi-agent tracking, collision avoidance, and shared world-modeling. Furthermore, persistent spatial memory using 3D Gaussian splatting, such as GSMem [53], offers a promising direction for maintaining re-renderable scene representations that multiple agents could concurrently query for coordination tasks.

7.2.5 Information Bottleneck and Recovery Mode Design

The shared ledger contains only the `consecutive_failures` counter, providing at most $\log_2(3) \approx 1.58$ bits of information. This minimal representation proved sufficient for the reaching task under progressive fault injection, achieving 88% success. However, for more complex coordination scenarios—task allocation, resource tracking, formation maintenance—richer state information (fault type, severity, location) may be required. Extending the ledger to support multi-dimensional state is a direction for future work.

The recovery mode implemented in Chapter 6 uses a fixed conservative target [0.15, 0.0, 0.15] and doubles the success tolerance from 1 cm to 2 cm. This design, while effective for the 4-DOF arm, may be suboptimal for other morphologies or tasks. Adaptive recovery target selection based on estimated fault parameters (e.g., using the joint angle predictions

from NeuroKin-3D to infer which joint is damaged) could yield faster recovery. The 12% failure cases in Chapter 5—extreme fault magnitudes (>25°), workspace exit, and silhouette degradation—suggest specific improvements: anti-windup for the integral term, a retreat-and-reapproach planner, and multi-camera redundancy or occlusion-aware training.

7.3 Threats to Validity and Directions for Mitigation

7.3.1 Simulation-to-Reality Gap

All experiments in this thesis were conducted in PyBullet simulation with perfect binary silhouettes (thresholded at 240). Real-world deployment introduces challenges that systematically threaten the validity of our conclusions: lighting variation, background clutter, camera noise, and calibration errors. The literature on sim-to-real transfer [41, 57] identifies four distinct gap dimensions: dynamics mismatch (physics simulation divergence), perception mismatch (sensor modeling inaccuracy), actuation mismatch (motor characteristics divergence), and system construction mismatch (morphological/structural misalignment).

For NeuroKin, the most critical gap is perception. The network was trained on synthetic binary masks derived from perfect RGB rendering. Physical cameras produce images with shadows, specular highlights, motion blur, and sensor noise. Background subtraction or learned segmentation (e.g., U-Net) would be required to extract clean silhouettes in real environments. Domain randomization during training—randomly varying brightness, contrast, adding synthetic shadows, and applying random occlusions—can improve robustness, as demonstrated in related work [15, 92].

Furthermore, bridging the visual perception gap—where shadows, noise, and complex lighting break threshold-based binary silhouettes—will require advanced generative techniques. The use of generative 3D worlds for scaling sim-to-real reinforcement learning [76], video generation priors to automate trajectory synthesis and realistic rendering [73], and unified simulation and correction using Gaussian digital twins [64] represent the exact tools needed to transition the NeuroKin architecture from PyBullet simulations to robust physical deployment.

Recent advances in sim-to-real transfer offer complementary strategies. Real-to-sim-to-real pipelines, such as the Real-is-Sim framework [59], demonstrate that continuous synchronization between physical robots and their digital twins can keep the twin behaviorally aligned during deployment. SplatSim [68] shows that zero-shot sim2real transfer of RGB manipulation policies is achievable using Gaussian splatting, while semantic 3D Gaussian splatting [69] preserves policy behavior consistency across lighting, texture, and sensor drift. Differentiable simulation enables rapid policy adaptation [67], and digital twin-driven reinforcement learning [72] provides a framework for safe policy evaluation before hardware execution. Simulation distillation [52] and online correction-based transfer [91] offer additional pathways for bridging the reality gap. For dexterous tasks requiring force feedback, closing the reality gap with force-aware calibration [93] has also been demonstrated. Safe

trajectory generation under motion and environmental uncertainties [87] further addresses the safety dimension of deployment, ensuring that even when perception fails, the system remains within bounded operational envelopes. However, full validation on physical hardware remains necessary before claiming real-world deployment readiness.

The stereoscopic setup also requires accurate extrinsic calibration. Calibration errors of 1-2 mm are acceptable given NeuroKin-3D's 2.39 mm mean spatial error, but errors exceeding 5 mm would degrade performance. Standard checkerboard calibration can achieve sub-millimeter accuracy, but the cameras must remain fixed relative to the robot base.

7.3.2 Abstraction of Mechanical Degradation and Rigid-Body Assumptions

The fault model used in Chapters 5 and 6—encoder slip on joint 2 with zero-mean Gaussian magnitude—represents a specific, relatively benign form of embodiment drift. Real hardware failures include complete joint seizure (zero degrees of freedom), broken gear teeth (periodic position errors), thermal drift (nonlinear, temperature-dependent), and communication packet loss (stochastic, bursty). The temporal stigmergy mechanism should generalize to these fault types, as it responds to task-level success/failure rather than specific fault signatures. However, the recovery mode's effectiveness may degrade if faults produce systematic biases that the visual estimator cannot track (e.g., a bent link that changes the kinematic tree).

The 12% failure cases in Chapter 5 provide a taxonomy of scenarios where visual estimation alone is insufficient: extreme fault magnitudes ($>25°$), workspace exit, and silhouette degradation. These failure modes are not unique to NeuroKin; they would affect any vision-based controller. Addressing them requires complementary mechanisms: anti-windup integrators, recovery planners, and redundant sensing.

It is also critical to explicitly acknowledge the boundary of the rigid-body assumption inherent in the current NeuroKin formulation. Direct silhouette mapping works exceptionally well for the rigid kinematic links and joint anomalies of the 4-DOF arm. However, extending visual self-modeling to soft robotics or embodiments with flexible elements would violate these rigid geometric assumptions. For such applications, integrating physics-informed dynamic surrogates—such as those used for the reconstruction and simulation of deformable objects [54]—would be necessary to capture non-rigid morphological drift. Extending visual self-modeling to continuum or soft robotics requires representations that track the continuous dynamics of deformation rather than discrete geometric snapshots. Recent advancements in shape-interpretable modeling, such as Bezier-parameterized representations for geometry-aware continuum robot control [17], provide the necessary mathematical framework to bridge this gap, allowing self-models to maintain controllable shape representations without relying on rigid kinematic assumptions.

7.3.3 Generalization to Higher Degrees of Freedom

The 4-DOF serial manipulator used throughout this thesis is a simplified testbed. Scaling to high-DOF systems (e.g., humanoid robots with 30+ joints) introduces curse-of-dimensionality challenges. For instance, successfully modeling the expanded configuration space of 7-DOF manipulators (e.g., Franka Panda) has been shown to require high-DOF dynamic neural fields coupled with curricular data sampling [12]. The 1024-dimensional latent embedding may be insufficient for 30-dimensional joint spaces; covering high-dimensional workspaces requires exponentially more training samples; and self-occlusion becomes severe, requiring multi-camera fusion. Potential architectural extensions include hierarchical encoders (separate networks for limbs, torso, head), attention mechanisms to handle variable-length kinematic chains, and graph neural networks that exploit the robot's kinematic tree structure [50]. Dynamics-grounded priors, such as the articulated-body dynamics network [81], offer a promising direction for sample-efficient scaling by embedding physical structure directly into the learning architecture. These extensions are beyond the scope of this thesis but represent important future work.

7.3.4 Catastrophic Forgetting Under Structural Damage

Structural damage adaptation is not evaluated in this thesis; the experiments focus on encoder-slip faults. Continual learning under structural changes remains an open problem, and mitigation strategies such as Elastic Weight Consolidation (EWC) [107], sparse subnetworks [108], or mixture-of-experts [109] are promising directions for future work.

7.3.5 Data Efficiency and Limited Training Samples

All comparative experiments trained on 2,000 samples—one-sixth the scale of the original FFKSM paper's 12,000 samples. While this enabled fair controlled comparison, it raises questions about generalization to larger datasets. Our data efficiency analysis suggests NeuroKin benefits more from limited data than volumetric rendering: it achieved 21.88 dB with 2,000 samples, approaching FFKSM's reported 23+ dB quality despite 6× less data. Extrapolating to 12,000 samples, we hypothesize NeuroKin would achieve 25–26 dB, exceeding FFKSM even at full scale. However, this extrapolation remains untested and should be validated in future work.

7.4 Summary of Discussion

This chapter critically evaluated the trade-offs and validity of the contributions presented in Chapters 4–6. The key insights are:

1. **Direct decoding trades novel-view synthesis for control-loop latency**. For applications requiring real-time response from a fixed viewpoint, this trade-off is highly

favorable. For multi-view tasks, hybrid architectures or camera conditioning are necessary.

2. **Supervision efficiency explains the speed-quality improvement**. Dense gradients from direct pixel prediction provide far more supervision per network evaluation than volumetric ray-marching.

3. **Temporal stigmergy validates the computational prerequisite for swarm coordination**. Fast individual self-diagnosis, decentralized ledger updates, and threshold-based adaptation collectively enable 88% post-fault success in a 100-stage sequential pipeline. Persistent spatial memory using 3DGS [53] and advanced decentralized mapping like feature-enriched SLAM [112] offer a potential pathway for extending this coordination framework to concurrent multi-agent settings.

4. **Sim-to-real transfer remains the primary threat to validity**. Domain randomization, robust silhouette extraction, generative video priors [73], Gaussian twin simulation [64], and hardware validation are essential next steps. Real-to-sim-to-real pipelines [59], zero-shot sim2real methods [68, 69], simulation distillation [52], policy transfer [91], and digital twin-driven RL [72] offer promising mitigation strategies.

5. **Structural damage adaptation remains future work**, and principled continual learning methods are needed for long-term deployment. Furthermore, while the current architecture is well-suited for rigid bodies, scaling to soft robotics will necessitate physics-informed dynamic surrogates [54].

6. **Scaling to higher degrees of freedom** (such as 7-DOF arms [12]) will require hierarchical encoders, graph neural networks, and dynamics-grounded priors [81] in addition to the morphology-aware attention mechanisms [50] already discussed.

The following chapter synthesizes these findings into concrete conclusions and maps a technical roadmap for transitioning from sequential validation to physical swarm deployment.

Chapter 8

Conclusion and Future Work

This chapter synthesizes the principal findings of this thesis, directly answering the four research questions posed in Chapter 1, and outlines a concrete technical roadmap for transitioning the validated sequential capabilities into physically embodied, spatially concurrent swarm deployments.

8.1 Summary of Contributions and Research Findings

This thesis shows that direct sensorimotor decoding can replace explicit volumetric reconstruction for control-oriented visual self-modeling, and that the resulting fast state estimates are useful both for closed-loop recovery and for temporal stigmergy across a sequential pipeline.

8.1.1 RQ1: Architectural Efficiency

Direct decoding removes the latency barrier that makes volumetric approaches difficult to integrate into control loops. NeuroKin achieves the throughput needed for real-time use while preserving enough fidelity for downstream tasks that only require silhouette-level awareness.

8.1.2 RQ2: Control Integration

Chapter 5 showed that NeuroKin-3D spatial estimates can stabilize a Jacobian-based inverse-kinematics loop. Under encoder slip at step 50, the proprioceptive baseline diverges while the visual controller continues to recover because its state estimate reflects the actual robot pose.

8.1.3 RQ3: Fault Tolerance

The main fault-tolerance result is narrower than full structural adaptation: visual self-modeling corrects corrupted proprioception during encoder slip, but larger structural damage remains a limitation and a future extension.

8.1.4 RQ4: Collective Adaptation

Chapter 6 showed that a shared failure ledger and a simple recovery threshold are enough to improve a 100-stage sequential pipeline from 13% to 88% post-fault success. That is the core temporal stigmergy result of the thesis.

8.2 Technical Contributions Summary

The thesis contributes three grounded results. First, it establishes that direct decoding is a practical route to fast visual self-modeling. Second, it shows that those estimates can be used inside a closed-loop controller to recover from encoder corruption. Third, it demonstrates that a minimal shared state can support sequential collective adaptation without explicit communication.

8.3 Future Roadmap: From Sequential Validation to Spatial Swarms

The future roadmap is straightforward. The next step is hardware validation under realistic sensing and calibration conditions, followed by concurrent multi-robot experiments that add collision avoidance and atomic shared-state updates. Those extensions are consistent with the results in this thesis, but they are not claimed as solved here.

8.4 Concluding Remarks

This thesis demonstrates that fast visual self-modeling is useful in practice: it supports real-time closed-loop correction when proprioception is corrupted and improves a sequential multi-agent pipeline through a minimal shared failure ledger. The results do not claim full spatial swarm deployment, but they do establish the computational foundation needed for it.

The central design lesson is to keep the representation as small as the task allows. For the control and coordination problems studied here, direct decoding is sufficient, volumetric reconstruction is too slow, and a minimal ledger is enough to support temporal stigmergy. Those are the concrete conclusions supported by the experiments in this thesis.

Bibliography

[1] J. Bongard, V. Zykov, and H. Lipson, "Resilient machines through continuous self-modeling," *Science*, vol. 314, no. 5802, pp. 1118–1121, 2006.

[2] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, "Robots that can adapt like animals," *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.

[3] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," in *Proc. Genet. Evol. Comput. Conf. (GECCO)*, 2015.

[4] R. Kwiatkowski and H. Lipson, "Task-agnostic self-modeling machines," *Science Robotics*, vol. 4, no. 26, p. eaau9354, 2019.

[5] R. Kwiatkowski and H. Lipson, "Zero-shot learning on simulated robots," in *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 1–7, 2019.

[6] Y. Hu, Y. Wang, R. Liu, Z. Shen, and H. Lipson, "Reconfigurable robot identification from motion data," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[7] A. Dogra, S. Mahan, S. S. Padhee, and E. Singla, "Unified modeling of unconventional modular and reconfigurable manipulation system," *Robotics and Computer-Integrated Manufacturing*, vol. 77, p. 102385, 2022.

[8] S. Farghdani, M. Patel, and R. Chhabra, "Robust embodied self-identification of morphology in damaged multi-legged robots," *IEEE Robot. Autom. Lett.*, vol. 10, pp. 1–8, 2025.

[9] C. Yu and S. Kriegman, "Robots that redesign themselves through kinematic self-destruction," arXiv preprint arXiv:2603.12505, 2026.

[10] B. Chen, R. Kwiatkowski, C. Vondrick, and H. Lipson, "Full-body visual self-modeling of robot morphologies," *Science Robotics*, vol. 7, no. 68, p. eabn1944, 2022.

[11] Y. Hu, J. Lin, and H. Lipson, "Teaching robots to build simulations of themselves," *Nature Machine Intelligence*, vol. 6, pp. 123–130, 2024.

[12] L. Schulze and H. Lipson, "High-degrees-of-freedom dynamic neural fields for robot self-modeling and motion planning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[13] Y. Hu, B. Chen, and H. Lipson, "Egocentric visual self-modeling for autonomous robot dynamics prediction and adaptation," arXiv preprint arXiv:2207.03386, 2022.

[14] M. Liu *et al.*, "Differentiable robot rendering: Bridging pixels and control through differentiable rendering," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[15] S. Rezvani, A. J. Mahmood, and R. Chhabra, "Robust visual embodiment: How robots discover their bodies in real environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[16] K. Hu, P. Yu, and N. Tan, "Learning high-fidelity robot self-model with articulated 3D Gaussian splatting," *Int. J. Robotics Research*, 2025.

[17] P. Yu, X. Wang, and N. Tan, "Shape-interpretable visual self-modeling enables geometry-aware continuum robot control," arXiv preprint arXiv:2603.01751, 2026.

[18] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. P. Srinivasan, "Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 5855–5864.

[19] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and A. Y. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020.

[20] A. Pumarola, E. Corona, G. Pons-Moll, and A. Sanfeliu, "D-NeRF: Neural radiance fields for dynamic scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021.

[21] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 102:1–102:15, 2022.

[22] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Zip-NeRF: Anti-aliased grid-based neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.

[23] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139:1–139:14, 2023.

[24] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian splatting SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 18039–18048.

[25] N. Keetha *et al.*, "SplaTAM: Splat, track and map 3D Gaussians for dense RGB-D SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 18410–18420.

[26] S. Li, B. Keller, Y. Lin, and B. Khailany, "GauRast: Enhancing GPU triangle rasterizers to accelerate 3D Gaussian splatting," in *Proc. Int. Symp. Comput. Archit. (ISCA)*, 2025.

[27] A. Guedon and V. Lepetit, "SuGaR: Surface-aligned Gaussian splatting for efficient 3D mesh reconstruction and high-quality mesh rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[28] Y. Weng *et al.*, "Neural implicit representation for building digital twins of unknown articulated objects," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[29] Z. Li *et al.*, "ART: Articulated reconstruction transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025.

[30] Y. Wu *et al.*, "AOMGen: Photoreal, physics-consistent demonstration generation for articulated object manipulation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.

[31] Z. Wu *et al.*, "URDF-Anything+: Autoregressive articulated 3D models generation for physical simulation," arXiv preprint arXiv:2502.09850, 2025.

[32] W. Xu, L. Liu, L. Zhang, D. Guo, and R. Liu, "MotionAnymesh: Physics-grounded articulation for simulation-ready digital twins," arXiv preprint arXiv:2603.12936, 2026.

[33] J. Wang, D. Wang, J. Hu, Q. Zhang, J. Yu, and L. Xu, "Kinematify: Open-vocabulary synthesis of high-DoF articulated objects," arXiv preprint arXiv:2511.01294, 2025.

[34] C. Zhang, M. Qin, Y. Wang, B. Xie, H. Li, and Z. Wang, "SIMART: Decomposing monolithic meshes into sim-ready articulated assets via MLLM," arXiv preprint arXiv:2603.23386, 2026.

[35] P. Wang, G. Ke, M. Rong, X. Guo, K. Ming, and S. Liu, "ArtLLM: Generating articulated assets via 3D LLM," arXiv preprint arXiv:2603.01142, 2026.

[36] J. Guo, Y. Xin, G. Liu, K. Xu, L. Liu, and R. Hu, "ArticulatedGS: Self-supervised digital twin modeling of articulated objects using 3D Gaussian splatting," arXiv preprint arXiv:2503.08135, 2025.

[37] Q. Yu, Z. Xia, C. Li, D. Pan, X. Wang, and J. Liu, "ArtGS: 3D Gaussian splatting for interactive visual-physical modeling and manipulation of articulated objects," arXiv preprint arXiv:2507.02600, 2025.

[38] D. Wu, Z. Wang, Y. Luo, H. Zhang, T. Chen, and K. Guo, "REArtGS++: Generalizable articulation reconstruction with temporal geometry constraint via planar Gaussian splatting," arXiv preprint arXiv:2511.17059, 2025.

[39] H. Dai, H. Fan, H. Zhang, D. Wu, J. Zhang, and H. Dong, "FreeArtGS: Articulated Gaussian splatting under free-moving scenario," arXiv preprint arXiv:2603.22102, 2026.

[40] Q. Chen *et al.*, "FreeGaussian: Annotation-free control of articulated objects via 3D Gaussian splats with flow derivatives," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2026.

[41] J. Tan *et al.*, "Sim-to-Real: Learning agile locomotion for quadruped robots," in *Proc. Robot.: Sci. Syst. (RSS)*, 2018.

[42] H. Lou *et al.*, "Robo-GS: A physics consistent spatial-temporal model for robotic arm with hybrid representation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.

[43] H. Lou *et al.*, "D-REX: Differentiable real-to-sim-to-real engine for learning dexterous grasping," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[44] Z. Sun, L. Bao, T. Peng, J. Sun, and C. Zhou, "A high-fidelity digital twin for robotic manipulation based on 3D Gaussian splatting," arXiv preprint arXiv:2601.03200, 2026.

[45] X. Li, G. Chen, and J. Alonso-Mora, "Building explicit world model for zero-shot open-world object manipulation," arXiv preprint arXiv:2603.13825, 2026.

[46] G. Lu, S. Zhang, Z. Wang, C. Liu, J. Lu, and Y. Tang, "ManiGaussian: Dynamic Gaussian splatting for multi-task robotic manipulation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 349–366.

[47] S. Pan, Y. Xu, R. Xu, Z. Zhou, S. Wu, and Z. Yu, "Self-correcting robot manipulation via Gaussian-splatted foresight," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2025.

[48] S. Yang, K. Zhan, Y. Zhang, L. Lin, X. Huang, and M. Li, "Novel demonstration generation with Gaussian splatting enables robust one-shot manipulation," arXiv preprint arXiv:2504.13175, 2025.

[49] Y. Wu, S. Bae, Y. Chen, L. Lyu, S. Li, Z. Huang, and X. Yang, "DexGrasp-Zero: A morphology-aligned policy for zero-shot cross-embodiment dexterous grasping," arXiv preprint arXiv:2603.16806, 2025.

[50] A. Patel and S. Song, "GET-Zero: Graph embodiment transformer for zero-shot embodiment generalization," in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.

[51] Q. Liang *et al.*, "Whole-body coordination for dynamic object grasping with legged manipulators," arXiv preprint arXiv:2508.08328, 2025.

[52] J. Levy *et al.*, "Simulation distillation: Pretraining world models in simulation for rapid real-world adaptation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[53] Y. Lu *et al.*, "GSMem: 3D Gaussian splatting as persistent spatial memory for zero-shot embodied exploration and reasoning," arXiv preprint arXiv:2603.19137, 2025.

[54] H. Jiang *et al.*, "PhysTwin: Physics-informed reconstruction and simulation of deformable objects from videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025.

[55] G. Jiang *et al.*, "GSWorld: Closed-loop photo-realistic simulation suite for robotic manipulation," arXiv preprint arXiv:2510.20813, 2025.

[56] A. Escontrela *et al.*, "GaussGym: An open-source real-to-sim framework for learning locomotion from pixels," arXiv preprint arXiv:2510.15352, 2025.

[57] E. Aljalbout, A. Frick, X. Chen, T. Westenbroek, and D. Fridovich-Keil, "The reality gap in robotics: Challenges, solutions, and best practices," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 9, 2026.

[58] H. Zhao *et al.*, "AGILE: A comprehensive workflow for humanoid loco-manipulation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.

[59] M. Torne, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal, "Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation," in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.

[60] E. Heiden, Z. Liu, V. Vineet, E. Coumans, and G. S. Sukhatme, "Inferring articulated rigid body dynamics from RGBD video," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023.

[61] J. Abou-Chakra, Y. Zhu, L. Zhang, D. Fridovich-Keil, and P. Agrawal, "Real-is-sim: Bridging the sim-to-real gap with a dynamic digital twin," arXiv preprint arXiv:2504.03597, 2025.

[62] Z. Jiang, H. Takemura, and M. Kita, "Digital twin system for home service robot based on motion simulation," *J. Robot. Mechatron.*, vol. 36, no. 2, pp. 420–432, 2024.

[63] C. Zhang, Y. Zou, Z. Wu, Y. Ling, Y. Yang, and Z. Wang, "UniPR: Unified object-level real-to-sim perception and reconstruction from a single stereo pair," arXiv preprint arXiv:2603.19616, 2026.

[64] Y. Cai, P. Jansonnie, C. de Farias, O. Arenz, and J. Peters, "GaussTwin: Unified simulation and correction with Gaussian splatting for robotic digital twins," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.

[65] K. Park, U. Sinha, J. T. Barron, S. Bouaziz, D. B. Goldman, S. M. Seitz, and R. Martin-Brualla, "Nerfies: Deformable neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021.

[66] Z. Wu *et al.*, "Perceptive humanoid parkour: Chaining dynamic human skills via motion matching," arXiv preprint arXiv:2602.15827, 2026.

[67] J. Pan, J. Xing, R. Reiter, Y. Zhai, E. Aljalbout, and D. Scaramuzza, "Learning on the fly: Rapid policy adaptation via differentiable simulation," *IEEE Robot. Autom. Lett.*, vol. 10, 2025.

[68] M. N. Qureshi, Y. Tang, Z. Zhang, X. Li, Y. Wang, and A. Gupta, "SplatSim: Zero-shot sim2real transfer of RGB manipulation policies using Gaussian splatting," arXiv preprint arXiv:2404.13099, 2025.

[69] J. Tang, Z. He, K. Zhang, Y. Liu, M. Chen, and W. Xu, "Bridging simulation and reality: Cross-domain transfer with semantic 2D Gaussian splatting," arXiv preprint arXiv:2512.04731, 2025.

[70] D. Li *et al.*, "HAIC: Humanoid agile object interaction control via dynamics-aware world model," arXiv preprint arXiv:2602.11758, 2026.

[71] J. Yu, H. Su, D. Fridovich-Keil, and A. Gupta, "Real2Render2Real: Scaling robot data without dynamics simulation or robot hardware," arXiv preprint arXiv:2505.09601, 2025.

[72] Q. Xu, Y. Liang, M. Chen, Z. Zhang, K. Xu, H. Wang, and J. Wang, "TwinRL-VLA: Digital twin-driven reinforcement learning for real-world robotic manipulation," arXiv preprint arXiv:2602.09023, 2025.

[73] S. He *et al.*, "V-Dreamer: Automating robotic simulation and trajectory synthesis via video generation priors," arXiv preprint arXiv:2603.18811, 2026.

[74] S. Zhu, L. Mou, D. Li, B. Ye, R. Huang, and H. Zhao, "VR-Robo: A real-to-sim-to-real framework for visual robot navigation and locomotion," *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, 2025.

[75] G. Liu, T. Wang, Z. Wu, J. Wu, S. Li, and X. Zhu, "CycleRL: Sim-to-real deep reinforcement learning for robust autonomous bicycle control," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[76] A. Choi, X. Wang, Z. Su, and W. Xu, "Scaling sim-to-real reinforcement learning for robot VLAs with generative 3D worlds," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.

[77] J. Liu *et al.*, "NavGSim: High-fidelity Gaussian splatting simulator for large-scale navigation," *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, 2025.

[78] Z. Mari, M. M. Nawaf, and P. Drap, "Digital twin-supervised reinforcement learning framework for autonomous underwater navigation," *Sensors*, 2024.

[79] V. Murugesan, R. Mathiazhagan, S. Joshi, and A. Arab, "Digital-twin evaluation for proactive human-robot collision avoidance via prediction-guided A-RRT*," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[80] Q. Peng, Y. Lin, Y. Xue, J. Pang, and W. Zhang, "Embodiment-aware generalist-specialist distillation for unified humanoid whole-body control," arXiv preprint arXiv:2602.02960, 2026.

[81] A. Rajeswaran, V. Kumar, A. Gupta, and A. Levine, "Articulated-body dynamics network: Dynamics-grounded prior for robot learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.

[82] J. Ren *et al.*, "Cybo-Waiter: A physical agentic framework for humanoid whole-body locomotion-manipulation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.

[83] Z. J. Xu *et al.*, "A robust antenna provides tactile feedback in a multi-legged robot," arXiv preprint arXiv:2603.07795, 2026.

[84] A. Jain, R. Sharma, S. Kumar, S. Kolathaya, Z. Zhang, and D. Song, "PolaRiS: Scalable real-to-sim evaluations for generalist robot policies," arXiv preprint arXiv:2512.16881, 2025.

[85] X. Li *et al.*, "Evaluating real-world robot manipulation policies in simulation," in *Proc. Conf. Robot. Learn. (CoRL)*, 2024.

[86] E. Daneshmand, S. Omar, G. Berseth, M. Khadiv, and H.-C. Lin, "SLowRL: Safe low-rank adaptation reinforcement learning for locomotion," arXiv preprint arXiv:2603.17092, 2026.

[87] F. Meng, Z. Yang, X. Mao, H. Liang, and M. Q.-H. Meng, "Provably safe trajectory generation for manipulators under motion and environmental uncertainties," arXiv preprint arXiv:2603.09083, 2026.

[88] L. Brunke *et al.*, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 5, pp. 1–43, 2022.

[89] F. Domberg and G. Schildbach, "Self-adapting robotic agents through online continual reinforcement learning with world model feedback," arXiv preprint arXiv:2603.04029, 2026.

[90] H. Lim and J. B. Choi, "Splat2Real: Novel-view scaling for physical AI with 3D Gaussian splatting," arXiv preprint arXiv:2603.10638, 2026.

[91] Y. Jiang, C. Wang, R. Zhang, J. Wu, and L. Fei-Fei, "TRANSIC: Sim-to-real policy transfer by learning from online correction," in *Proc. Conf. Robot. Learn. (CoRL)*, 2024.

[92] K. Qiu, K. Walker, M. Y. Michelis, M. Cygan, and J. Hughes, "Swim2Real: VLM-guided system identification for sim-to-real transfer," arXiv preprint arXiv:2603.20827, 2026.

[93] ByteDance Seed, "Closing the reality gap: Zero-shot sim-to-real deployment for dexterous force-based grasping and manipulation," arXiv preprint arXiv:2601.02778, 2026.

[94] N. Grambow *et al.*, "Anomaly detection for generic failure monitoring in robotic assembly, screwing and manipulation," *IEEE Robotics and Automation Letters*, vol. 11, no. 5, pp. 3664–3671, 2026.

[95] Y. Fu, X. Chen, B. Zhang, M. Yang, and K. Zhang, "Contrastive forward prediction reinforcement learning for adaptive fault-tolerant legged robots," in *Proc. Conf. Robot. Learn. (CoRL)*, 2025.

[96] H. Chen, Y. Yao, R. Liu, C. Liu, and J. Ichnowski, "Automating robot failure recovery using vision-language models with optimized prompts," arXiv preprint arXiv:2409.03966, 2024.

[97] N. Jayasinghe *et al.*, "Residual control for fast recovery from dynamics shifts," arXiv preprint arXiv:2603.07775, 2026.

[98] G. G. Briscoe-Martinez, Y. Gautam, R. Shetty, A. Pasricha, M. M. Nicotra, and A. Roncone, "Moving on, even when you're broken: Fail-active trajectory generation via diffusion policies conditioned on embodiment and task," arXiv preprint arXiv:2602.02895, 2026.

[99] N. Poddar, S. McCrory, L. Penco, G. Clark, H. E. Sevil, and R. Griffin, "Embedding classical balance control principles in reinforcement learning for humanoid recovery," arXiv preprint arXiv:2603.08619, 2026.

[100] H. Li *et al.*, "PCHC: Enabling preference-conditioned humanoid control via multi-objective reinforcement learning," arXiv preprint arXiv:2603.24047, 2026.

[101] D. Li *et al.*, "Learning actionable manipulation recovery via counterfactual failure synthesis," arXiv preprint arXiv:2603.13528, 2026.

[102] H. Lee, W.-J. Baek, J. Cha, and J. Park, "TOLEBI: Learning fault-tolerant bipedal locomotion via online status estimation and fallibility rewards," arXiv preprint arXiv:2602.05596, 2026.

[103] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," *Psychol. Learn. Motiv.*, vol. 24, pp. 109–165, 1989.

[104] R. M. French, "Catastrophic forgetting in connectionist networks," *Trends Cogn. Sci.*, vol. 3, no. 4, pp. 128–135, 1999.

[105] S. Grossberg, "Competitive learning: From interactive activation to adaptive resonance," *Cognitive Science*, vol. 11, no. 1, pp. 23–63, 1987.

[106] M. Mermillod, A. Bugaiska, and P. Bonin, "The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects," *Front. Psychol.*, vol. 4, p. 504, 2013.

[107] J. Kirkpatrick *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proc. Natl. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.

[108] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[109] D. C. Mocanu, E. Mocanu, P. Stone, P. H. Nguyen, M. Gibescu, and A. Liotta, "Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science," *Nat. Commun.*, vol. 9, p. 2383, 2018.

[110] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[111] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017.

[112] C. Thirgood, O. Mendez, E. C. Ling, J. Storey, and S. Hadfield, "FeatureSLAM: Feature-enriched 3D Gaussian splatting SLAM in real time," arXiv preprint arXiv:2503.14256, 2025.

[113] A. T. Tran and J. Kosecka, "VarSplat: Uncertainty-aware 3D Gaussian splatting for robust RGB-D SLAM," arXiv preprint arXiv:2507.08762, 2025.

[114] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London: Springer, 2009.

[115] E. N. Lorenz, "Deterministic nonperiodic flow," *Journal of the Atmospheric Sciences*, vol. 20, no. 2, pp. 130–141, 1963.

[116] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio, and A. Courville, "On the spectral bias of neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019.

[117] G. Chechik, I. Meilijson, and E. Ruppin, "Synaptic pruning in development: A computational account," *Neural Computation*, vol. 10, no. 7, pp. 1759–1777, 1998.